

分层设计



ODS层尽量保持和原始数据相同；DWD层负责清洗转换，数据粒度与ODS一致；只要是甲方提供的表和字段，都要在这两层进行处理（课程中无关的表不会处理）。

DWB层，是数据降维后生成的明细宽表，作为中间数据使用。可以只保留一段周期内的有效数据。从DWB层开始形成宽表，可以将频繁关联的表合并在一起。

DWS层，按照主题划分的日统计宽表，基于DWB上的基础数据，整合汇总成分析某一个主题域的服务数据。从DWS层开始按照主题进行聚合分析。

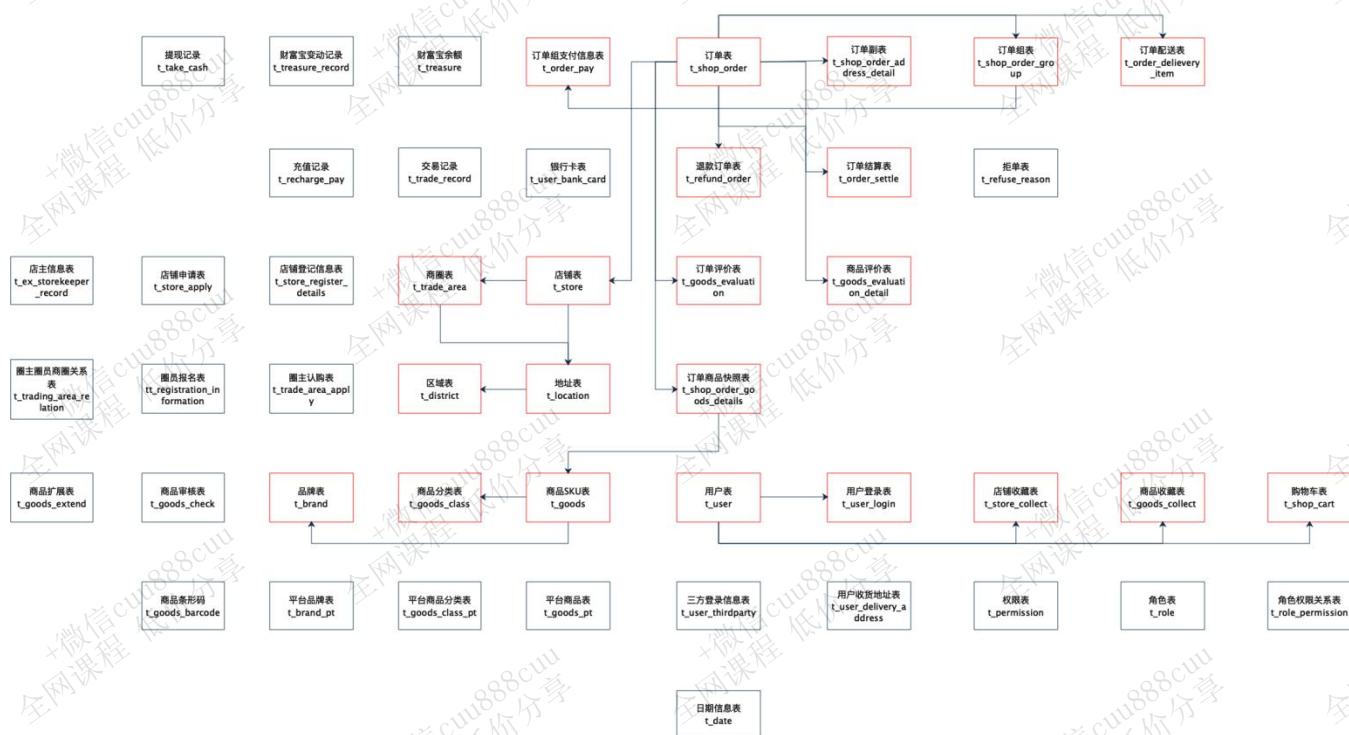
DM层，数据集市层，主要职责是建设宽表模型、汇总表模型，比如用户主题宽表、销售主题宽表等。主要作用是支撑数据分析查询以及支持应用所需数据。DM层在DWS日统计宽表基础上，进一步进行汇总统计，将可能涉及到的时间维度都包含进来。

RPT层，同ADS层、APP层。根据报表、专题分析的需求而计算生成的个性化数据。RPT层是直接面向需求的，数据大多来源于DM数据集市。



业务系统表结构

1. 表关系图



2. 表结构

2.1 订单相关

2.1.1 订单表! (t_shop_order)





Table:		Comment:	
t_shop_order		订单表	
Columns (18)	Keys (1)	Indexes (1)	Foreign Keys
id	varchar(64)	-- part of primary key	/*根据一定规则生成的订单编号*/
order_num	varchar(64)		/*订单序号*/
buyer_id	varchar(64)		/*买家的userId*/
store_id	varchar(64)		/*店铺的id*/
order_from	tinyint(4)		/*是来自于app还是小程序,或者pc 1.安卓; 2.ios; 3.小程序H5; 4.PC*/
order_state	int(4) default 1		/*订单状态:1.已下单; 2.已付款; 3. 已确认; 4.配送; 5.已完成; 6.退款; 7.已取消*/
create_time	datetime		/*下单时间*/
finshed_time	timestamp		/*订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需要根据订单完成时间,进行自动收货以及自动销
is_settlement	tinyint(4) default 0		/*是否结算;0.待结算订单; 1.已结算订单;*/
is_delete	tinyint(4) default 0		/*订单评价的状态:0.未删除; 1.已删除;(默认0)*/
evaluation_state	tinyint(4) default 0		/*订单评价的状态:0.未评价; 1.已评价;(默认0)*/
way	varchar(64) default 'SHOP'		/*取货方式:SELF自提;SHOP店铺负责配送*/
is_stock_up	int(4) default 0		/*是否需要备货 0: 不需要 1: 需要 2:平台确认备货 3:已完成备货 4平台已经将货物送至店铺 */
create_user	varchar(64)		
create_time	datetime		
update_user	varchar(64)		
update_time	datetime		
is_valid	tinyint(4) default 1		/*是否有效 0: false; 1: true; 订单是否有有效的标志*/

2.1.2 订单详情表! (t_shop_order_address_detail)





Table:				Comment:
t_shop_order_address_detail				订单副表
Columns (16)	Keys (1)	Indexes	Foreign Keys	
id	varchar(64)	-- part of primary key /*关联订单的id*/		
order_amount	decimal(11,2)	/*订单总金额:购买总金额-优惠金额*/		
discount_amount	decimal(11,2)	/*优惠金额*/		
goods_amount	decimal(11,2)	/*用户购买的商品的总金额+运费*/		
is_delivery	varchar(255) default '1'	/*0.自提; 1.配送*/		
buyer_notes	varchar(255)	/*买家备注留言*/		
pay_time	datetime	/*支付时间*/		
receive_time	datetime	/*接单时间*/		
delivery_begin_time	datetime	/*开始配送时间*/		
arrive_store_time	datetime	/*配送员到达店铺时间*/		
arrive_time	datetime	/*订单完成时间,当配送员点击确认送达时,进行更新订单完成时间*/		
create_user	varchar(64)			
create_time	datetime			
update_user	varchar(64)			
update_time	datetime			
is_valid	tinyint(4) default 1	/*是否有效 0: false; 1: true; 订单是否有效的标志*/		

2.1.3 订单组表! (t_shop_order_group)



Table:

t_shop_order_group

Columns (9)	Keys (1)	Indexes (2)	Foreign Keys
id	varchar(64) -- part of primary key		
order_id	varchar(32) /*订单id*/		
group_id	varchar(32) /*订单分组id*/		
is_pay	tinyint(4) default 0 /*是否已支付,0未支付,1已支付*/		
create_user	varchar(64)		
create_time	datetime		
update_user	varchar(64)		
update_time	datetime		
is_valid	tinyint(4) default 1		

2.1.4 退款订单表! (t_refund_order)

Table:

t_refund_order

Comment:

退款订单表

Columns (15)	Keys (1)	Indexes (1)	Foreign Keys
id	varchar(64) -- part of primary key /*退款单号*/		
order_id	varchar(64) /*订单的id*/		
apply_date	datetime /*用户申请退款的时间*/		
modify_date	datetime /*退款订单更新时间*/		
refund_reason	varchar(255) /*买家退款原因*/		
refund_amount	decimal(11,2) /*订单退款的金额*/		
refund_state	tinyint(4) default 1 /*1.申请退款;2.拒绝退款;3.同意退款,配送员配送;4.商家同意退款,用户亲自送货;5.退款完成*/		
refuse_refund_reason	varchar(255) /*商家拒绝退款原因*/		
refund_goods_type	varchar(255) /*1.上门取货(买家承担运费);2.买家送达;*/		
refund_shipping_fee	decimal(11,2) default 0.00 /*配送员的运费(如果退货方式为1:则买家支付配送费)*/		
create_user	varchar(64)		
create_time	datetime		
update_user	varchar(64)		
update_time	datetime		
is_valid	tinyint(4) default 1 /*是否有效 0: false; 1: true; 订单是否有效的标志*/		

2.1.5 订单结算表! (t_order_settle)



Table:	Comment:		
t_order_settle	订单结算表,如果发生退款,则结算的金额 = 订单的总金额 - 退款的金额		
Columns (22)	Keys (1)	Indexes (1)	Foreign Keys
id	varchar(64) default '' -- part of primary key	/*结算单号*/	
order_id	varchar(64)		
settlement_create_date	datetime	/*用户申请结算的时间*/	
settlement_amount	decimal(11,2)	/*如果发生退款,则结算的金额 = 订单的总金额 - 退款的金额*/	
dispatcher_user_id	varchar(64)	/*配送员id*/	
dispatcher_money	decimal(11,2)	/*配送员的配送费(配送员的运费(如果退货方式为1:则买家支付配送费))*/	
circle_master_user_id	varchar(64)	/*圈主id*/	
circle_master_money	decimal(11,2)	/*圈主分润的金额*/	
plat_fee	decimal(11,2)	/*平台应得的分润*/	
store_money	decimal(11,2)	/*商家应得的订单金额*/	
status	tinyint(4) default 0	/*0.待结算; 1.待审核; 2.完成结算; 3.拒绝结算*/	
note	varchar(200)	/*原因*/	
settle_time	datetime	/* 结算时间*/	
create_user	varchar(64)		
create_time	datetime		
update_user	varchar(64)		
update_time	datetime		
is_valid	tinyint(4) default 1	/*是否有效 0: false; 1: true; 订单是否有有效的标志*/	
first_commission_user_id	varchar(64)	/*一级分佣用户*/	
first_commission_money	decimal(11,2)	/*一级分佣金额*/	
second_commission_user_id	varchar(64)	/*二级分佣用户*/	
second_commission_money	decimal(11,2)	/*二级分佣金额*/	

2.1.6 订单商品快照表! (t_shop_order_goods_details)





Table:		Comment:	
t_shop_order_goods_details		订单和商品的中间表	
Columns (22)	Keys (1)	Indexes (1)	Foreign Keys
id	varchar(64) -- part of primary key	/*id主键*/	
order_id	varchar(64)	/*对应订单表的id*/	
shop_store_id	varchar(64)	/*卖家店铺ID*/	
buyer_id	varchar(64)	/*购买用户ID*/	
goods_id	varchar(64)	/*购买商品的id*/	
buy_num	int(11)	/*购买商品的数量*/	
goods_price	decimal(11,2)	/*购买商品的价格*/	
total_price	decimal(11,2)	/*购买商品的价格 = 商品的数量 * 商品的单价*/	
goods_name	varchar(64)	/*商品的名称*/	
goods_image	varchar(255)	/*商品的图片*/	
goods_specification	varchar(255)	/*商品规格*/	
goods_weight	int(11) default 0		
goods_unit	varchar(50)	/*商品计量单位*/	
goods_type	varchar(64)	/*商品分类 ytgj:进口商品 ytsc:普通商品 hots:爆品*/	
refund_order_id	varchar(64)	/*退款订单的id*/	
goods_brokerage	decimal(11,2) default 0.00	/*商家设置的商品分润的金额*/	
is_refund	tinyint(4) default 0	/*0.不退款; 1.退款*/	
create_user	varchar(64)		
create_time	datetime		
update_user	varchar(64)		
update_time	datetime		
is_valid	tinyint(4) default 1	/*是否有效 0: false; 1: true*/	

2.1.7 订单评价表! (t_goods_evaluation)





Table: **t_goods_evaluation**

Columns (13)	Keys (1)	Indexes	Foreign Keys
id	varchar(64) -- part of primary key		
user_id	varchar(64) /*评论人id*/		
store_id	varchar(64) /*店铺id*/		
order_id	varchar(64) /*订单id*/		
geval_scores	int(4) default 0 /*综合评分*/		
geval_scores_speed	int(4) default 0 /*送货速度评分0-5分(配送评分)*/		
geval_scores_service	int(4) default 0 /*服务评分0-5分*/		
geval_isanony	tinyint(4) default 1 /*0-匿名评价, 1-非匿名*/		
create_user	varchar(64)		
create_time	datetime		
update_user	varchar(64)		
update_time	datetime		
is_valid	tinyint(4) default 1 /*0：失效, 1：开启*/		

2.1.8 商品评价表! (t_goods_evaluation_detail)





Table:

t_goods_evaluation_detail

Columns (23)	Keys (1)	Indexes	Foreign Keys
id	varchar(64)	-- part of primary key	
user_id	varchar(64)	/*评论人id*/	
store_id	varchar(64)	/*店铺id*/	
goods_id	varchar(64)	/*商品id*/	
order_id	varchar(64)	/*订单id*/	
order_goods_id	varchar(64)	/*订单商品表id*/	
geval_scores_goods	int(4) default 0	/*商品评分0-10分*/	
geval_content	varchar(1000)		
geval_content_superaddition	varchar(1000)	/*追加评论*/	
geval_addtime	datetime	/*评论时间*/	
geval_addtime_superaddition	datetime	/*追加评论时间*/	
geval_state	tinyint(4) default 1	/*评价状态 1-正常 0-禁止显示*/	
geval_remark	varchar(255)	/*管理员对评价的处理备注*/	
revert_state	tinyint(4) default 0	/*回复状态0未回复1已回复*/	
geval_explain	varchar(1000)	/*管理员回复内容*/	
geval_explain_superaddition	varchar(1000)	/*管理员追加回复内容*/	
geval_explaintime	datetime	/*管理员回复时间*/	
geval_explaintime_superaddition	datetime	/*管理员追加回复时间*/	
create_user	varchar(64)		
create_time	datetime		
update_user	varchar(64)		
update_time	datetime		
is_valid	tinyint(4) default 1	/*0：失效，1：开启*/	

2.1.9 订单配送信息表! (t_order_delievery_item)





Table:		Comment:
t_order_delivery_item		包含配送员的信息,以及配送的商品信息,
Columns (22)		
Keys (1)		
Indexes (1)		
Foreign Keys		
id	varchar(64) -- part of primary key /*主键id*/	
shop_order_id	varchar(64) /*订单表ID*/	
refund_order_id	varchar(64)	
dispatcher_order_type	tinyint(4) default 1 /*配送订单类型:1.支付单; 2.退款单*/	
shop_store_id	varchar(64) /*卖家店铺ID*/	
buyer_id	varchar(64) /*购买用户ID*/	
circle_master_user_id	varchar(64) /*圈主ID*/	
dispatcher_user_id	varchar(64) /*配送员ID*/	
dispatcher_order_state	tinyint(4) default 0 /*配送订单状态:0.待接单.1.已接单.2.已到店.3.配送中 4.商家普通提货码完成订单.5.商家万能提货码	
order_goods_num	tinyint(4) /*订单商品的个数*/	
delivery_fee	decimal(11,2) /*配送员的运费*/	
distance	int(11) default 0 /*配送距离*/	
dispatcher_code	varchar(25) /*收货码*/	
receiver_name	varchar(64) /*收货人姓名*/	
receiver_phone	varchar(64) /*收货人电话*/	
sender_name	varchar(64) /*发货人姓名*/	
sender_phone	varchar(64) /*发货人电话*/	
create_user	varchar(64)	
create_time	datetime	
update_user	varchar(64)	
update_time	datetime	
is_valid	tinyint(4) default 1 /*是否有效 0: false; 1: true*/	

2.1.10 拒单表 (t_refuse_reason)

略

2.2 支付相关

2.2.1 订单组支付信息表! (t_order_pay)



Table:	Comment:		
t_order_pay	订单支付表		
Columns (9)	Keys (1)	Indexes (1)	Foreign Keys
id	varchar(64) -- part of primary key		
group_id	varchar(64) /*关联shop_order_group的group_id,一对多订单*/		
order_pay_amount	decimal(11,2) /*订单总金额;*/		
create_date	datetime /*订单创建的时间,需要根据订单创建时间进行判断订单是否已经失效*/		
create_user	varchar(64)		
create_time	datetime		
update_user	varchar(64)		
update_time	datetime		
is_valid	tinyint(4) default 1 /*是否有效 0: false; 1: true; 订单是否有效的标志*/		

2.2.2 财富宝余额 (t_treasure)

略

2.2.3 交易记录表! (t_trade_record)

Table:	Comment:		
t_trade_record	所有交易记录信息		
Columns (20)	Keys (1)	Indexes (1)	Foreign Keys
id	varchar(64) -- part of primary key		/*交易单号*/
external_trade_no	varchar(64)		/*(支付,结算,退款)第三方交易单号*/
relation_id	varchar(64)		/*关联单号*/
trade_type	tinyint(4)		/*1.支付订单; 2.结算订单; 3.退款订单; 4.充值单; 5.提现单; 6.分销单; 7.缴
status	tinyint(4) default 1		/*1.成功;2.失败;3.进行中*/
finnshed_time	datetime		/*订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需要根
fail_reason	varchar(255)		/*交易失败的原因*/
payment_type	varchar(255)		/*支付方式:小程序,app微信,支付宝,快捷支付,钱包, 银行卡,消费券*/
trade_before_balance	decimal(11,2)		/*交易前余额*/
trade_true_amount	decimal(11,2)		/*交易实际支付金额,第三方平台扣除优惠以后实际支付金额*/
trade_after_balance	decimal(11,2)		/*交易后余额*/
note	varchar(200)		/*业务说明*/




```
user_card    varchar(64) /*第三方平台账户标识/多钱包用户钱包id*/  
user_id      varchar(64) /*用户id*/  
aip_user_id  varchar(64) /*钱包id*/  
create_user  varchar(64)  
create_time  datetime  
update_user  varchar(64)  
update_time  datetime  
is_valid     tinyint(4) default 1 /*是否有效 0: false; 1: true; 订单是否有效的标志*/
```

2.2.4 财富宝余额变动记录 (t_treasure_record)

略

2.2.5 提现记录表 (t_take_cash)

略

2.2.6 充值记录表 (t_recharge_pay)

略

2.2.7 银行卡 (t_user_bank_card)

略

2.3 店铺相关

2.3.1 店铺表! (t_store)





Table:

Comment:

t_store

Columns (37)

Keys (1)

Indexes (1)

Foreign Keys

id	varchar(64)	-- part of primary key /*主键*/
user_id	varchar(64)	default ''
store_avatar	varchar(255)	/*店铺头像*/
address_info	varchar(500)	/*店铺详细地址*/
name	varchar(255)	/*店铺名称*/
store_phone	varchar(64)	/*联系电话*/
province_id	int(11)	/*店铺所在省份ID*/
city_id	int(11)	/*店铺所在城市ID*/
area_id	int(11)	/*店铺所在县ID*/
mb_title_img	varchar(255)	/*手机店铺 页头背景图*/
store_description	text	/*店铺描述*/
notice	varchar(255)	/*店铺公告*/
is_pay_bond	tinyint(4)	/*是否有交过保证金 1: 是 0: 否*/
trade_area_id	varchar(64)	/*归属商圈ID*/
delivery_method	tinyint(4)	/*配送方式 1: 自提; 3: 自提加配送均可; 2: 商家配送*/
origin_price	decimal(10)	
free_price	decimal(20)	
store_type	int(11)	/*店铺类型 22天街网店 23实体店 24直营店铺 33会员专区店*/
store_label	varchar(255)	/*店铺logo*/
search_key	varchar(255)	/*店铺搜索关键字*/
end_time	varchar(20)	/*营业结束时间*/
start_time	varchar(20)	/*营业开始时间*/
operating_status	tinyint(4)	/*营业状态 0: 未营业; 1: 正在营业*/

create_user	varchar(64)
create_time	datetime
update_user	varchar(64)
update_time	datetime
is_valid	tinyint(4) default 1 /*0关闭, 1开启, 3店铺申请中*/
state	varchar(64) default 'MONEY' /*可使用的支付类型:MONEY金钱支付;CASHCOUPON现金券支付*/
idCard	varchar(64) /*身份证*/
deposit_amount	decimal(11,2) /*商圈认购费用总额*/
delivery_config_id	varchar(64) /*配送配置表关联ID*/
aip_user_id	varchar(64) /*通联支付标识ID*/
search_name	varchar(128) /*模糊搜索名称字段:名称_+真实名称*/
automatic_order	tinyint(4) default 0 /*是否开启自动接单功能 1: 是 0: 否*/
is_primary	tinyint(4) default 1 /*是否是总店 1: 是 2: 不是*/
parent_store_id	varchar(64) /*父级店铺的id, 只有当is_primary类型为2时有效*/





2.3.2 商圈表! (t_trade_area)

Table:		Comment:
t_trade_area		
Columns (21)	Keys (1)	Indexes
Foreign Keys		
id	varchar(64) -- part of primary key /*主键*/	
user_id	varchar(64) /*用户ID*/	
user_allinpay_id	varchar(64) /*通联用户表id*/	
trade_avatar	varchar(255) /*商圈logo*/	
name	varchar(64) /*商圈名称*/	
notice	varchar(255) /*商圈公告*/	
distric_province_id	int(8) /*商圈所在省份ID*/	
distric_city_id	int(11) /*商圈所在城市ID*/	
distric_area_id	int(11) /*商圈所在县ID*/	
address	varchar(255) /*商圈地址*/	
radius	double /*半径*/	
mb_title_img	varchar(255) /*手机商圈 页头背景图*/	
deposit_amount	decimal(11,2) default 0.00 /*商圈认购费用总额*/	
hava_deposit	int(11) /*是否有交过保证金 1: 是 0: 否*/	
state	tinyint(4) default -1 /*申请商圈状态 -1: 未认购; 0: 申请中; 1: 已认购; */	
search_key	varchar(64) /*商圈搜索关键字*/	
create_user	varchar(64)	
create_time	datetime	
update_user	varchar(64)	
update_time	datetime	
is_valid	tinyint(4) default 1 /*是否有效 0: false; 1: true*/	

2.3.3 地址表! (t_location)





Table:		Comment:
t_location		
Columns (18)	Keys (1)	Indexes (2)
Foreign Keys		
id	varchar(64) -- part of primary key	/*主键*/
type	int(4)	/*类型 1: 商圈地址; 2: 店铺地址; 3.用户地址管理;4.订单买家地址信息;5.订单卖家地址信息*/
correlation_id	varchar(64)	/*关联表id*/
address	varchar(64)	/*地图地址详情*/
latitude	double	/*纬度*/
longitude	double	/*经度*/
street_number	varchar(100)	/*门牌*/
street	varchar(100)	/*街道*/
district	varchar(50)	/*区县*/
city	varchar(50)	/*城市*/
province	varchar(50)	/*省份*/
business	varchar(200)	/*百度商圈字段，代表此点所属的商圈*/
create_user	varchar(64)	
create_time	datetime	
update_user	varchar(64)	
update_time	datetime	
is_valid	tinyint(4) default 1	/*是否有效 0: false; 1: true*/
adcode	char(6) default '0'	/*百度adcode,对应区县code*/

2.3.4 区域表! (t_district)

Table:	
t_district	
Columns (5)	Keys (1)
Indexes (3)	
Foreign Keys	
id	varchar(64) -- part of primary key
code	char(6)
name	varchar(48)
pid	int(11)
alias	varchar(48)

2.3.5 店铺登记信息表 (t_store_register_details)

略





2.3.6 店铺申请表 (t_store_apply)

略

2.3.7 店主信息表 (t_ex_storekeeper_record)

略

2.3.8 圈主认购表 (t_trade_area_apply)

略

2.3.9 圈员报名表 (t_registration_information)

略

2.3.10 圈主圈员商圈关系表 (t_trading_area_relation)

略

2.4 商品相关

2.4.1 商品表! (t_goods)



Table:

Comment:

t_goods

Columns (30)	Keys (1)	Indexes (2)	Foreign Keys
id	varchar(64)	-- part of primary key	
store_id	varchar(64)	/*所属商店ID*/	
class_id	varchar(64)	/*分类id:只保存最后一层分类id*/	
store_class_id	varchar(64)	/*店铺分类id*/	
brand_id	varchar(64)	/*品牌id*/	
goods_name	varchar(255)	/*商品名称*/	
goods_specification	varchar(255)	/*商品规格*/	
search_name	varchar(128)	/*模糊搜索名称字段:名称_+真实名称*/	
goods_sort	int(11) default 0	/*商品排序*/	
goods_market_price	decimal(11,2) default 0.00	/*商品市场价*/	
goods_price	decimal(11,2) default 0.00	/*商品销售价格(原价)*/	
goods_promotion_price	decimal(11,2) default 0.00	/*商品促销价格(售价)*/	
goods_storage	int(11) default 100	/*商品库存*/	
goods_limit_num	int(11) default 100	/*购买限制数量*/	
goods_unit	varchar(50)	/*计量单位*/	
goods_state	tinyint(4) default 1	/*商品状态 1正常, 2下架,3违规 (禁售) */	
goods_verify	tinyint(4) default 3	/*商品审核状态: 1通过, 2未通过, 3审核中*/	
activity_type	tinyint(4) default 0	/*活动类型:0无活动1促销2秒杀3折扣*/	
discount	int(4) default 0	/*商品折扣(%)*/	
seckill_begin_time	datetime	/*秒杀开始时间*/	
seckill_end_time	datetime	/*秒杀结束时间*/	
seckill_total_pay_num	int(11) default 0	/*已秒杀数量*/	
seckill_total_num	int(11)	/*秒杀总数限制*/	
seckill_price	decimal(11,2)	/*秒杀价格*/	

top_it	tinyint(4) default 0	/*商品置顶: 1-是, 0-否*/
create_user	varchar(64)	
create_time	datetime	
update_user	varchar(64)	
update_time	datetime	
is_valid	tinyint(4) default 1	/*0 : 失效, 1 : 开启*/



2.4.2 商品分类表! (t_goods_class)

Table:

t_goods_class	
Columns (20)	Keys (1) Indexes Foreign Keys
id	varchar(100) -- part of primary key
store_id	varchar(64) /*店铺id*/
class_id	varchar(64) /*对应的平台分类表id*/
name	varchar(255) /*店铺内分类名字*/
parent_id	varchar(64) /*父id*/
level	tinyint(4) /*分类层级*/
is_parent_node	tinyint(4) default 0 /*是否为父节点:1是0否*/
background_img	varchar(255) /*背景图片*/
img	varchar(255) /*分类图片*/
keywords	varchar(255) /*关键词*/
title	varchar(255) /*搜索标题*/
sort	int(11) /*排序*/
note	varchar(255) /*类型描述*/
url	varchar(255) /*分类的链接*/
is_use	tinyint(4) default 1 /*是否使用:0否,1是*/
create_user	varchar(64)
create_time	datetime
update_user	varchar(64)
update_time	datetime
is_valid	tinyint(4) default 1 /*0：失效，1：开启*/

2.4.3 品牌表! (t_brand)



Table: Comme

t_brand

Columns (14)	Keys (1)	Indexes	Foreign Keys
id	varchar(64) -- part of primary key		
store_id	varchar(64) /*店铺id*/		
brand_pt_id	varchar(64) /*平台品牌库品牌Id*/		
brand_name	varchar(255) /*品牌名称*/		
brand_image	varchar(255) /*品牌图片*/		
initial	varchar(10) /*品牌首字母*/		
sort	int(11) default 0 /*排序*/		
is_use	tinyint(4) default 1 /*0禁用1启用*/		
goods_state	tinyint(4) default 1 /*商品品牌审核状态 1 审核中,2 通过,3 拒绝*/		
create_user	varchar(64)		
create_time	datetime		
update_user	varchar(64)		
update_time	datetime		
is_valid	tinyint(4) default 1 /*0：失效，1：开启*/		

2.4.4 平台商品表 (t_goods_pt)

略

2.4.5 商品审核表 (t_goods_check)

略

2.4.6 商品扩展表 (t_goods_extend)

略

2.4.7 平台商品分类表 (t_goods_class_pt)

略





2.4.8 平台品牌表 (t_brand_pt)

略

2.4.9 商品条形码 (t_goods_barcode)

略

2.5 用户相关

2.5.1 登录日志表! (t_user_login)

Table:	Comment:
t_user_login	
Columns (7)	Keys (1) Indexes Foreign Keys
id	varchar(64) -- part of primary key
login_user	varchar(64)
login_type	varchar(255) /*登录类型（登陆时使用）*/
client_id	varchar(125) /*推送标示id(登录、第三方登录、注册、支付回调、给用户推送消息时使用)*/
login_time	datetime
login_ip	varchar(64)
logout_time	datetime

2.5.2 店铺收藏表! (t_store_collect)





Table:

t_store_collect

Columns (8)

Keys (1)

Indexes

Foreign Keys

```
id          varchar(64) -- part of primary key
user_id     varchar(64) /*收藏人id*/
store_id    varchar(64) /*店铺id*/
create_user varchar(64)
create_time datetime
update_user varchar(64)
update_time datetime
is_valid    tinyint(4) default 1 /*0：失效，1：开启*/
```

2.5.3 商品收藏表! (t_goods_collect)

Table:

t_goods_collect

Columns (9)

Keys (1)

Indexes (1)

Foreign Keys

```
id          varchar(64) -- part of primary key
user_id     varchar(64) /*收藏人id*/
goods_id    varchar(64) /*商品id*/
store_id    varchar(64) /*通过哪个店铺收藏的（因主店分店概念存在需要）*/
create_user varchar(64)
create_time datetime
update_user varchar(64)
update_time datetime
is_valid    tinyint(4) default 1 /*0：失效，1：开启*/
```

2.5.4 购物车表! (t_shop_cart)



Table:	Comment:		
t_shop_cart	购物车,当用户进行提交订单之后,将购物车相关的删除		
Columns (10)	Keys (1)	Indexes (2)	Foreign Keys
id	varchar(64)	-- part of primary key	/*主键id*/
shop_store_id	varchar(64)		/*卖家店铺ID*/
buyer_id	varchar(64)		/*购买用户ID*/
goods_id	varchar(64)		/*购买商品的id*/
buy_num	int(11)		/*购买商品的数量*/
create_user	varchar(64)		
create_time	datetime		
update_user	varchar(64)		
update_time	datetime		
is_valid	tinyint(4)	default 1	/*是否有效 0: false; 1: true*/

2.5.5 用户表 (t_user)

略

2.5.6 三方登录信息表 (t_user_thirdparty)

略

2.5.7 用户收货地址表 (t_user_delivery_address)

略

2.5.8 权限表 (t_permission)

略

2.5.9 角色表 (t_role)

略

2.5.10 角色权限关系表 (t_role_permission)

略





2.6 系统配置相关

2.6.1 时间维度表 (t_date)





Table:

dim_date

Columns (40)	Keys	Indexes	Foreign Keys
dim_date_id	varchar(max)	/*日期*/	
date_code	varchar(max)	/*日期编码*/	
lunar_calendar	varchar(max)	/*农历*/	
year_code	varchar(max)	/*年code*/	
year_name	varchar(max)	/*年名称*/	
month_code	varchar(max)	/*月份编码*/	
month_name	varchar(max)	/*月份名称*/	
quanter_code	varchar(max)	/*季度编码*/	
quanter_name	varchar(max)	/*季度名称*/	
year_month	varchar(max)	/*年月*/	
year_week_code	varchar(max)	/*一年中第几周*/	
year_week_name	varchar(max)	/*一年中第几周名称*/	
year_week_code_cn	varchar(max)	/*一年中第几周（中国）*/	
year_week_name_cn	varchar(max)	/*一年中第几周名称（中国）*/	
week_day_code	varchar(max)	/*周几code*/	
week_day_name	varchar(max)	/*周几名称*/	
day_week	varchar(max)	/*周*/	
day_week_cn	varchar(max)	/*周(中国)*/	
day_week_num	varchar(max)	/*一周第几天*/	
day_week_num_cn	varchar(max)	/*一周第几天（中国）*/	
day_month_num	varchar(max)	/*一月第几天*/	
day_year_num	varchar(max)	/*一年第几天*/	
date_id_wow	varchar(max)	/*与本周环比的上周日期*/	
date_id_mom	varchar(max)	/*与本月环比的上月日期*/	
date_id_wyw	varchar(max)	/*与本周同比的上年日期*/	



```
date_id_mym      varchar(max) /*与本月同比的上年日期*/
first_date_id_month  varchar(max) /*本月第一天日期*/
last_date_id_month  varchar(max) /*本月最后一天日期*/
half_year_code    varchar(max) /*半年code*/
half_year_name     varchar(max) /*半年名称*/
season_code        varchar(max) /*季节编码*/
season_name        varchar(max) /*季节名称*/
is_weekend         varchar(max) /*是否周末（周六和周日）*/
official_holiday_code varchar(max) /*法定节假日编码*/
official_holiday_name varchar(max) /*法定节假日*/
festival_code       varchar(max) /*节日编码*/
festival_name       varchar(max) /*节日*/
custom_festival_code varchar(max) /*自定义节日编码*/
custom_festival_name varchar(max) /*自定义节日*/
update_time        varchar(max) /*更新时间*/
```

2.7 广告相关

略

2.8 促销相关

略

2.9 配送相关

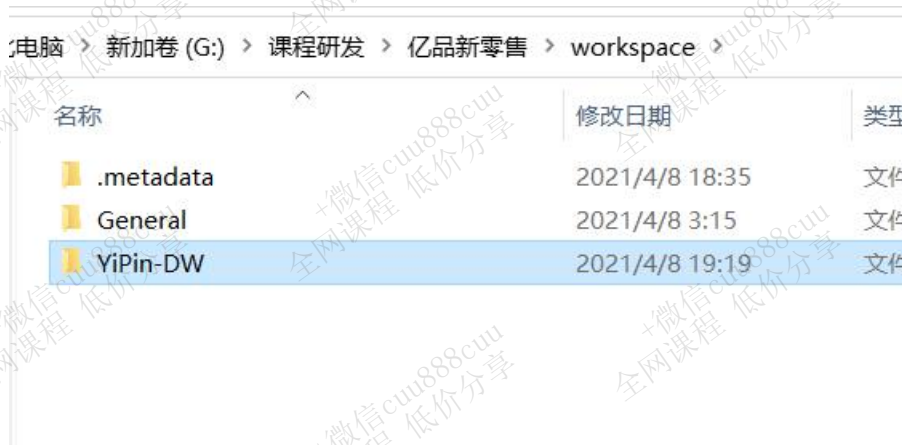
略



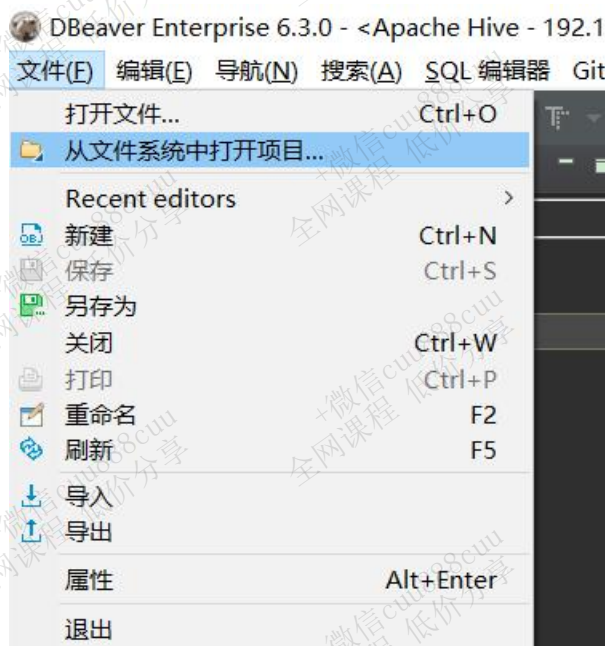
3. 导入测试数据

3.1 DBeaver创建项目

1. 新建自己的项目目录

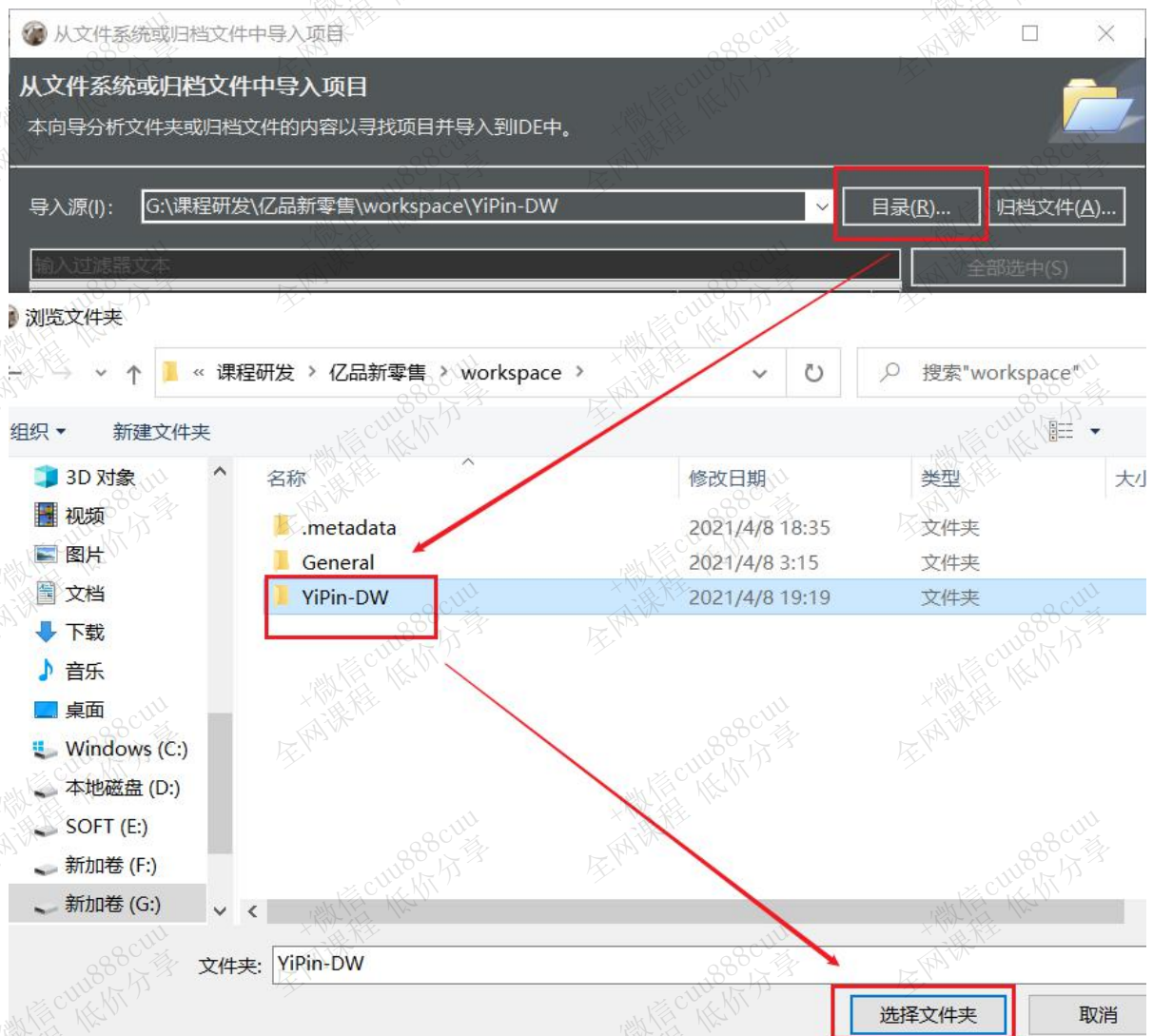


2. 【文件】->【从文件系统中打开项目】



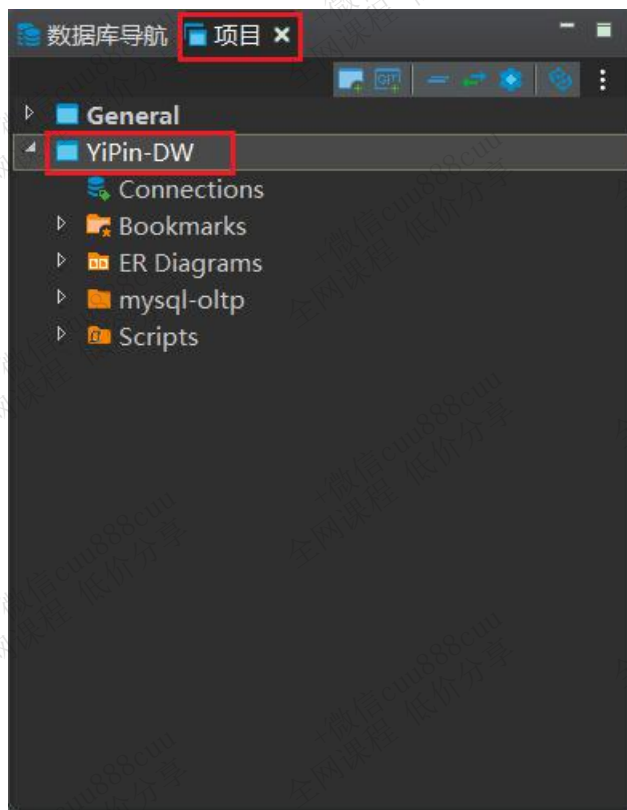
3. 选中项目目录



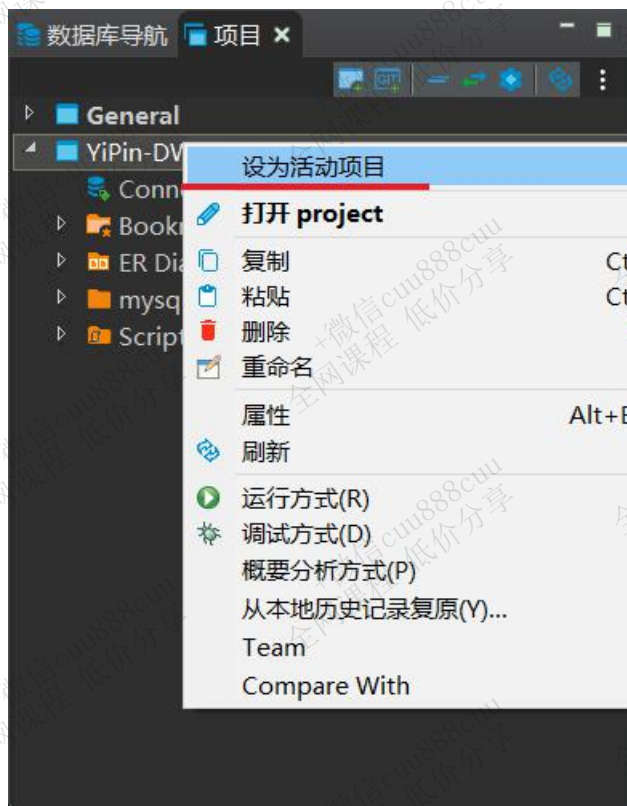


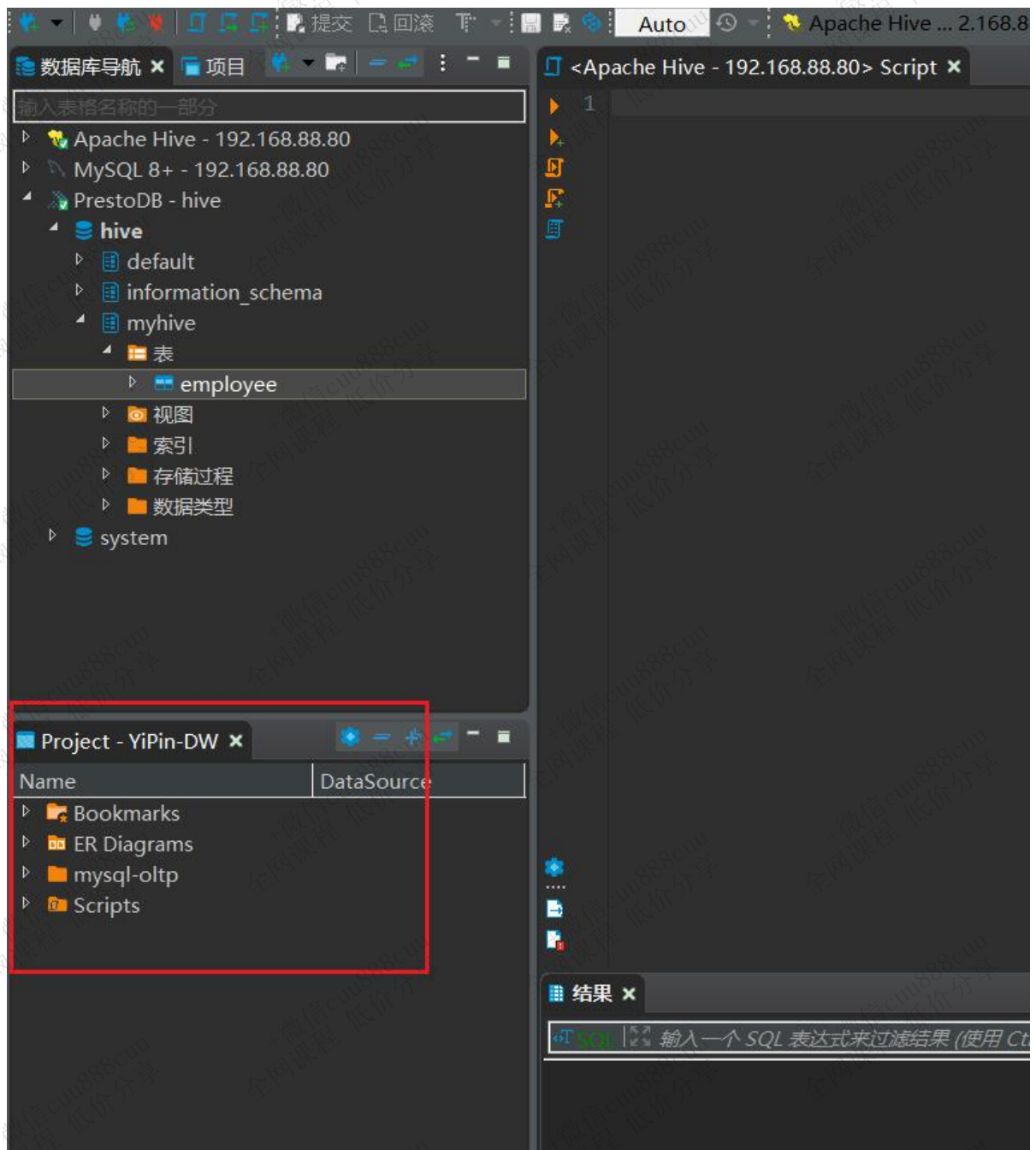
4. 【确定】后，在【项目】中查看





5. 设为【活动项目】



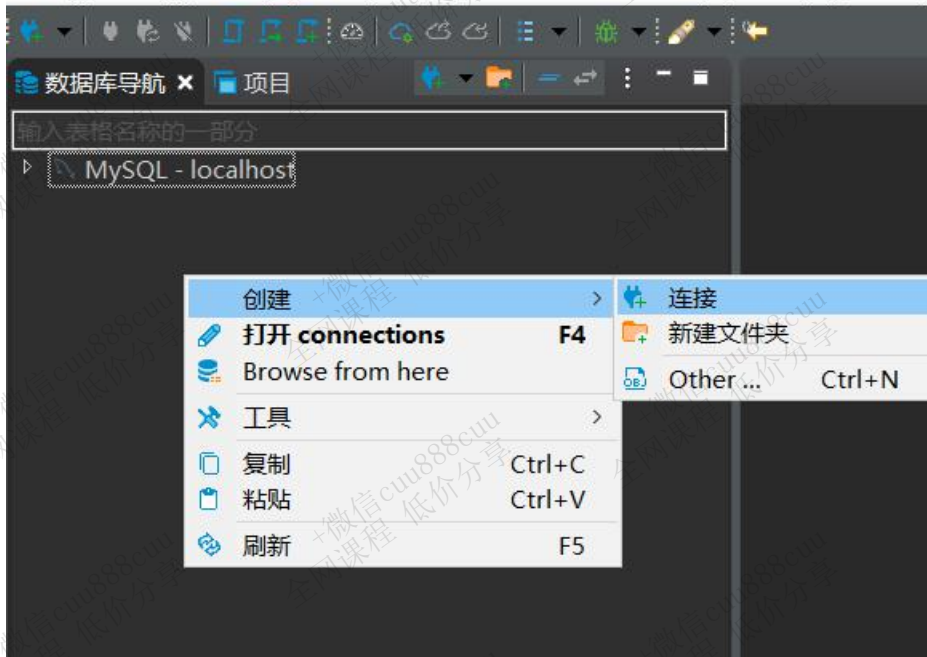


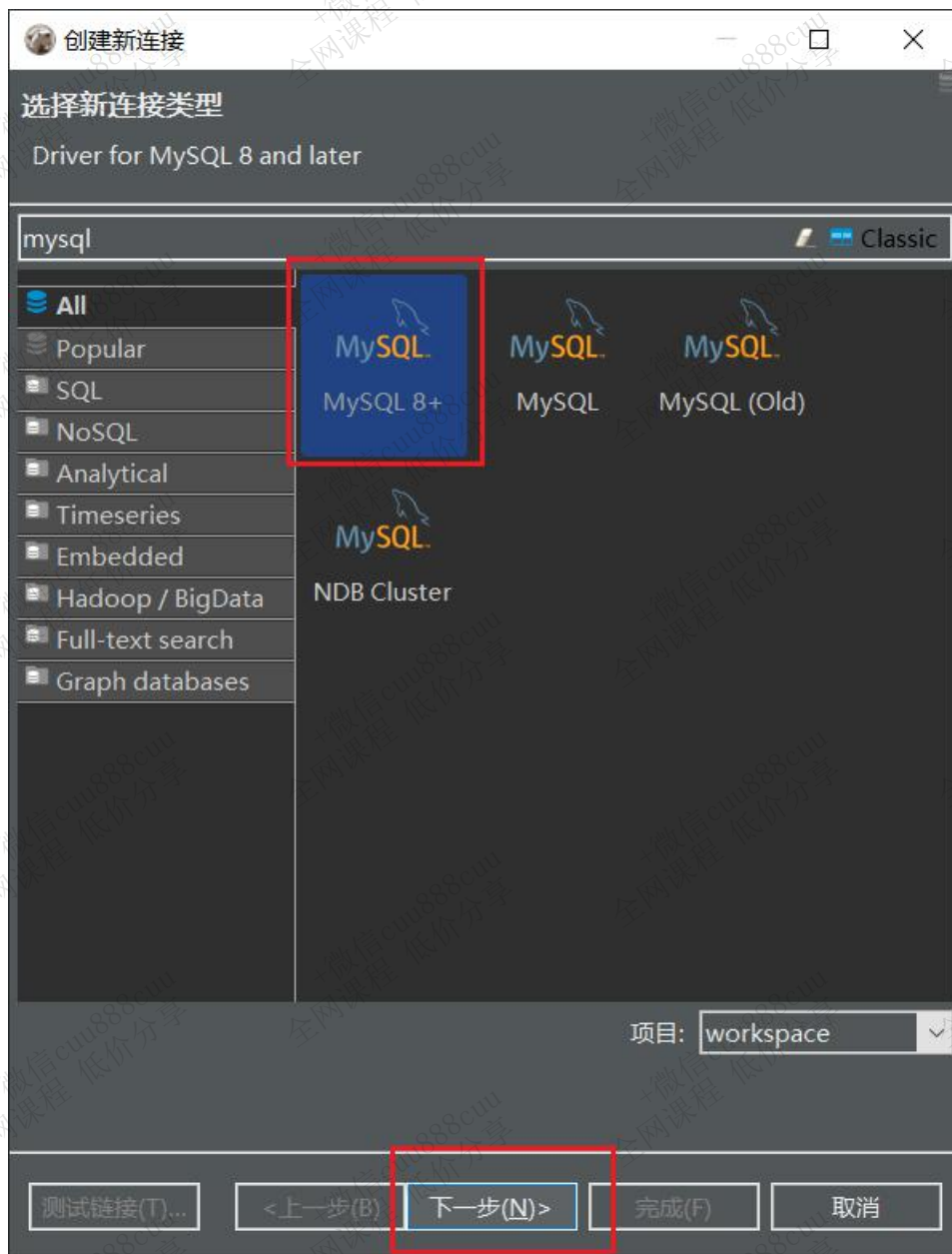


3.2 连接Mysql

DBeaver Enterprise 6.3.0

文件(E) 编辑(E) 导航(N) 搜索(A) SQL 编辑器 Git 数据库(D) 窗口(W) 帮助(H)







常规

驱动属性

SSH

Proxy

SSL

服务器地址:

localhost

端口:

3306

数据库:

用户名:

root

密码:

☐ Save password locally

服务器时区:

自动检测

本地客户端:

MySQL Server 5.7

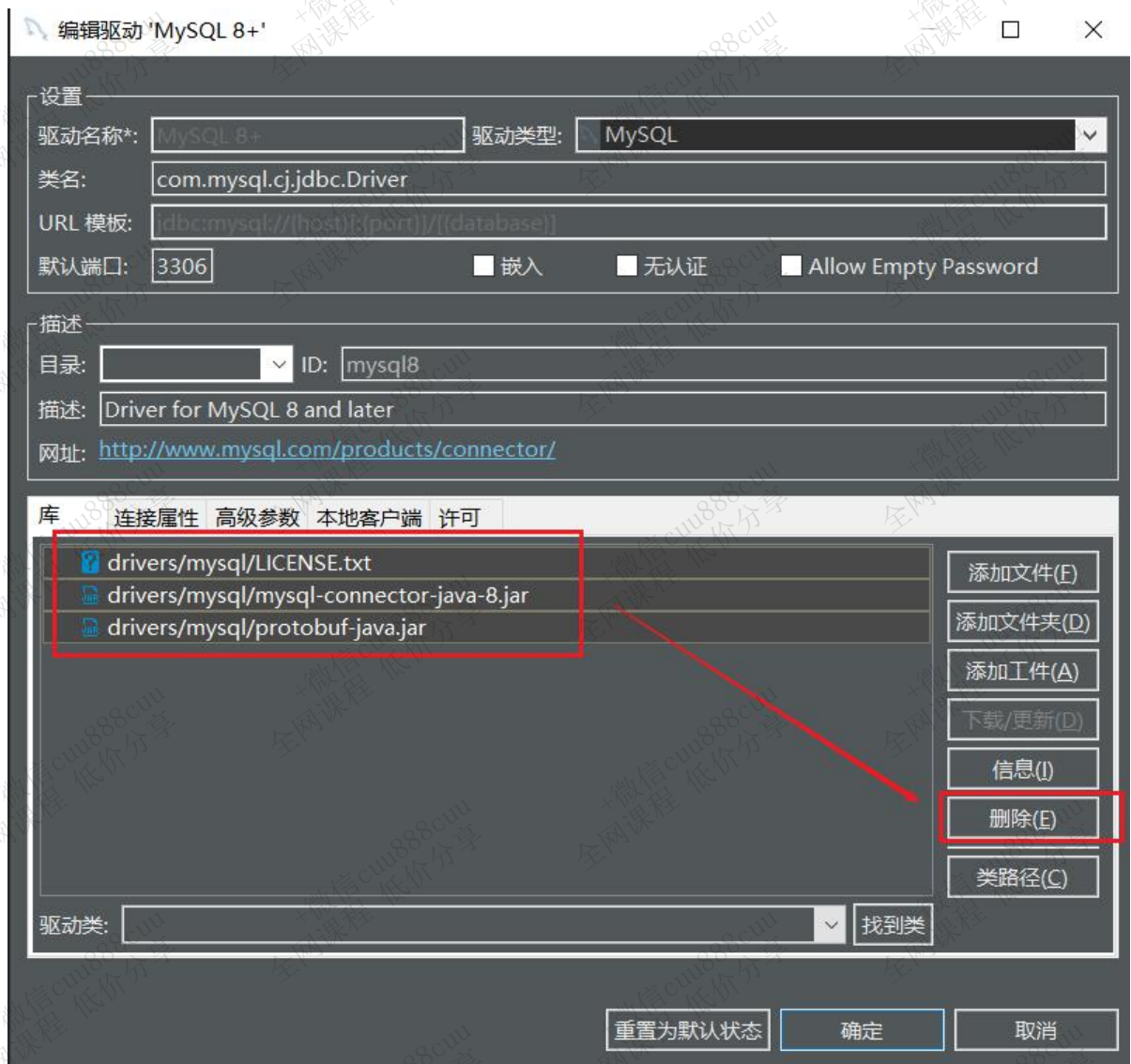
Advanced settings:

连接详情(名称、类型 ...)

驱动名称: MySQL 8+

编辑驱动设置





设置

驱动名称*: 驱动类型:

类名:

URL 模板:

默认端口: ☐ 嵌入 ☐ 无认证 ☐ Allow Empty Password

描述

目录: ID:

描述:

网址: <http://www.mysql.com/products/connector/>

库 连接属性 高级参数 本地客户端 许可

添加文件(E) 添加文件夹(D) 添加工件(A) 下载/更新(D) 信息(I) 删除(E) 类路径(C)

驱动类: 找到类

课程研发 > 亿品新零售 > 资料 > 驱动包 >

搜索"驱动包"

组织 新建文件夹

SOFT (E:) 新加卷 (F:) 新加卷 (G:) Data (H:) 新加卷 (I:) Windows (C:) 本地磁盘 (D:) OneDrive WPS网盘 此电脑 网络

名称	修改日期	类型	大小
hive	2021/4/26 14:10	文件夹	
mysql-connector-java-5.1.49.jar	2020/4/20 5:10	Executable Jar File	984 KB
mysql-connector-java-5.1.49.zip	2021/5/6 16:28	360压缩 ZIP 文件	3,635 KB
presto-jdbc-0.245.1.jar	2021/1/28 11:57	Executable Jar File	9,182 KB

文件名(N): 打开(O) 取消



编辑驱动 'MySQL 8+' □ ×

设置

驱动名称*: 驱动类型:

类名:

URL 模板:

默认端口: ☐ 嵌入 ☐ 无认证 ☐ Allow Empty Password

描述

目录: ID:

描述:

网址: <http://www.mysql.com/products/connector/>

库 连接属性 高级参数 本地客户端 许可

驱动类:



创建新连接

连接设置

MySQL 8+ 连接设置

常规 驱动属性 SSH Proxy SSL

服务器地址: 192.168.88.80

端口: 3306

数据库:

用户名: root

密码: ●●●●● ☒ Save password locally

服务器时区: 自动检测

本地客户端: MySQL Server 5.7

Advanced settings:

连接详情(名称、类型...)

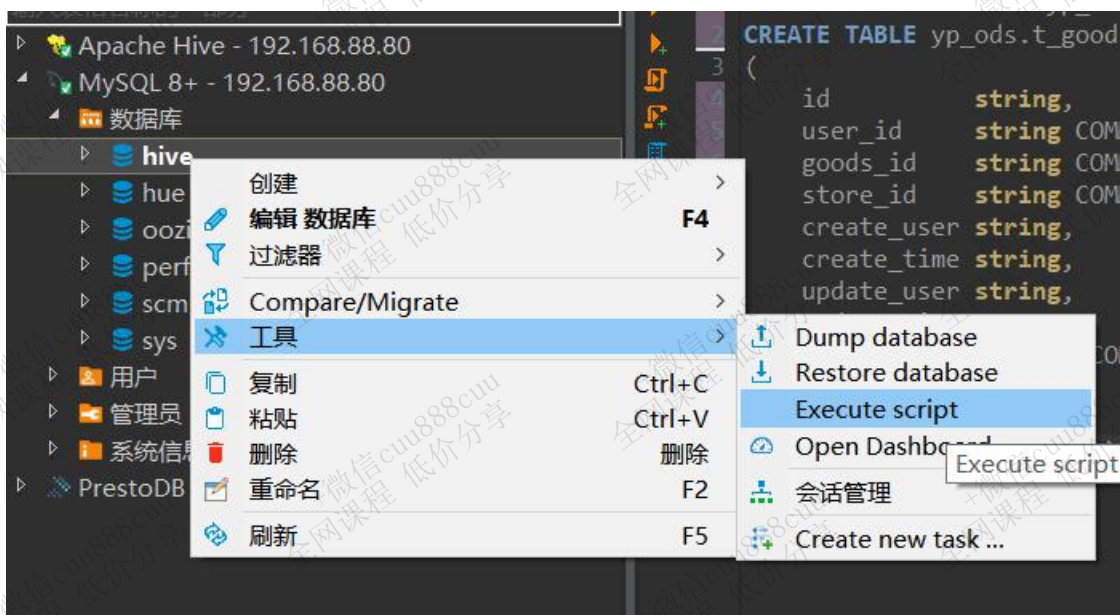
驱动名称: MySQL 8+ 编辑驱动设置

测试链接(I)... <上一步(B) 下一步(N)> 完成(F) 取消

3.3 导入SQL

亿品新零售\workspace\YiPin-DW\mysql-oltp\dump-yipin.sql





ODS层搭建

原始数据层，ODS层是原始数据的完整备份，不做任何修改。为了方便DWD层使用，一般会在ODS层增加抽取数据的日期字段。

1. 首次执行和循环执行

1.1 首次执行

首次建库时，需要对OLTP应用中的全量数据进行采集、清洗和统计计算，因此所有表都使用全量同步。

历史数据量可能会非常大，远远超出了增量过程。在执行时需要进行针对性的优化配置并采用分批执行。



1.2 循环执行

后续的循环执行大多采用的是**T+1模式**。

首次初始化以后，对OLTP每天的新增数据和更新数据要进行同步，如果还是对全量数据进行分析，效率会非常低下。每次**数据只有一天的量**，采集、清洗和统计的效率相对于全量计算会有很大提升。

什么是T+1？

这种说法来源于股票交易：

T+0，是国际上普遍使用的一种证券度（或期货）交易制度。凡在证券（或期货）成交日当天办理好证券（或期货）和价款清算交割手续的交易制度，就称为T+0交易。通俗说，就是当天买入的证道券（或期货）在当天就可以卖出。

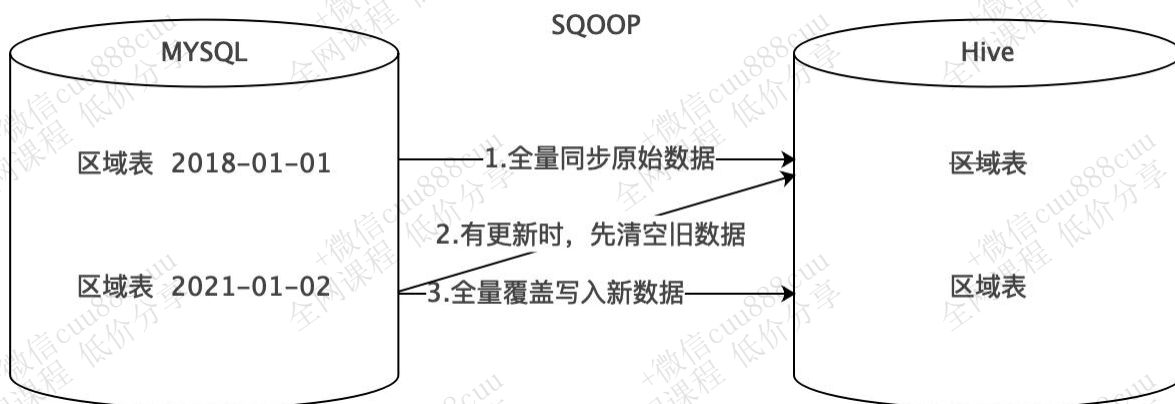
T+1是一种股票交易制度，即**当日买进的股票**，要到**下一个交易日才能卖出**。“T”指交易登记日，“T+1”指登记日的次日。

2. 数据同步方式

2.1 全量覆盖

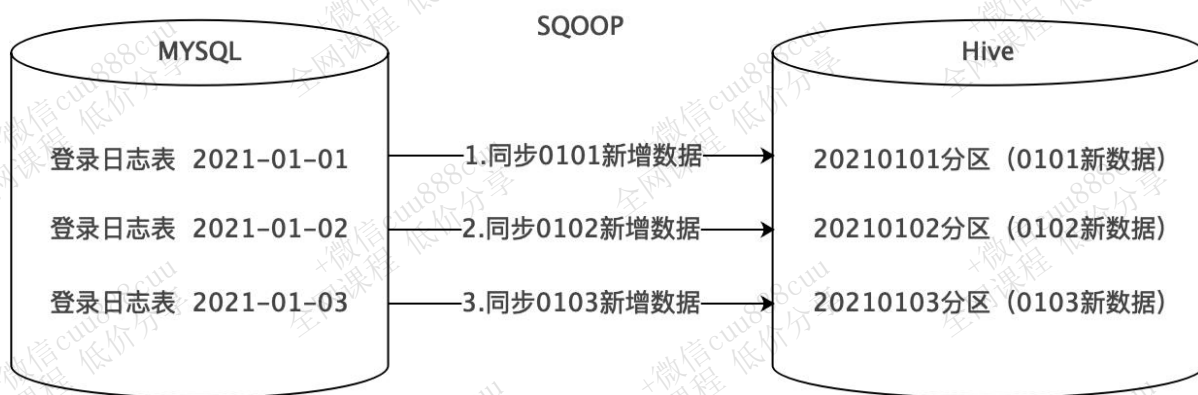
不需要分区，同步时直接覆盖插入。

适用于数据不会有任何新增和变化的情况。比如地区、时间、性别等维度数据，不会变更或变更不影响业务，可以只保留最新值。



2.2 仅新增同步

每天新增一个日期分区，同步并存储当天的新增数据。比如登录记录表、访问日志表、交易记录表、商品评价表等。



2.3 新增及更新同步

每天新增一个日期分区，同步并存储当天的新增和更新数据。

适用于既有新增又有更新的数据，比如用户表、订单表等。



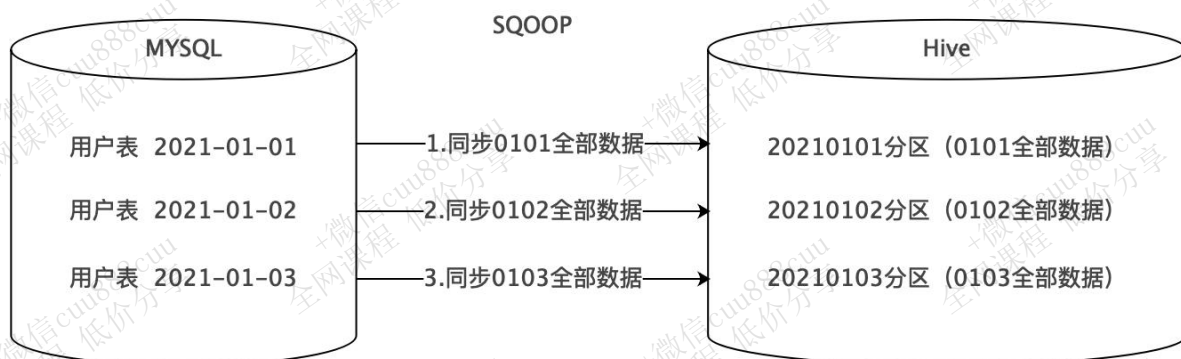
2.4 全量同步

每天新增一个日期分区，同步并存储当天的全量数据，历史数据定期删除。

适用于数据会有新增和更新，但是数据量较少，且历史快照不用保存很久的情况。

首次执行时都是全量同步。





3. Hive参数优化（基础）

此课程中关于Hive的优化，皆是基于Hive2.x的版本，对于Hive1.x旧版本的优化机制不再复述（新版本已改善或变更）。另外新版本中默认为开启状态的优化配置项，在工作中无需修改，也不再复述。

3.1 HDFS副本数

dfs.replication (HDFS)

文件副本数，通常设为3，不推荐修改。

如果测试环境只有二台虚拟机（2个datanode节点），此值要修改为2。

复制因子

dfs.replication

HDFS（服务范围）

3

3.2 Yarn基础配置

3.2.1 NodeManager配置

3.2.1.1 CPU配置

配置项：yarn.nodemanager.resource.cpu-vcores

表示该节点服务器上yarn可以使用的虚拟CPU个数，默认值是8，推荐将值配置与物理CPU线程数相同，如果节点CPU核心不足8个，要调小这个值，yarn不会智能的去检测物理核心数。



容器虚拟 CPU 内核

yarn.nodemanager.resource.cpu-vcores

编辑单个值

NodeManager Default Group ...and 2 others

16

容器虚拟 CPU 内核

yarn.nodemanager.resource.cpu-vcores

编辑相同值

NodeManager Default Group

16

NodeManager Group 1

16

NodeManager Group 2

16

查看 CPU 线程数：

```
grep 'processor' /proc/cpuinfo | sort -u | wc -l
```

Cloudera Manager 集群 主机 诊断 审核 图表 管理

所有主机

配置 Add Hosts 重新运行升级向导 检查所有主机 Inspect Network Performance

搜索

筛选器

- 状态
 - 运行状况良好 3
- 集群
- 内核
- 授权状态
- 上一检测信号
- 加载 (1 分钟)
- 加载 (5 分钟)
- 加载 (15 分钟)
- 维护模式
- 机架
- 服务
- 运行状况测试

主机配置 角色 主机模板 磁盘概述

名称	IP	角色	授权状态	上一检测信号	平均负载	内核	磁盘使用情况	物理内存	交换空间
hadoop01	172.17.0.200	7 Role(s)	已授权	3.18s 之前	0.09 0.07 0.11	16	23.2 GiB / 1 TiB	4.1 GiB / 31.3 GiB	0 B / 1.5 GiB
hadoop02	172.17.0.201	8 Role(s)	已授权	6.27s 之前	0.15 0.10 0.12	16	20.2 GiB / 1 TiB	5.3 GiB / 31.3 GiB	0 B / 1.5 GiB
hadoop03	172.17.0.202	18 Role(s)	已授权	8.11s 之前	0.19 0.22 0.23	16	47.3 GiB / 1 TiB	16 GiB / 62.8 GiB	0 B / 1.5 GiB

列: 11 已选定

zz-bd-cloudera-web.itheima.net/cm/hardware/hosts

3.2.1.2 内存配置

配置项：yarn.nodemanager.resource.memory-mb

设置该nodemanager节点上可以为容器分配的总内存，默认为8G，如果节点内存资源不足8G，要减少这个值，yarn不会智能的去检测内存资源，一般按照服务器剩余可用内存资源进行配置。生产上根据经验一般要预留15-20%的内存，那么可用内存就是实际内存*0.8，比如实际内存是64G，那么64*0.8=51.2G，我们设置成50G就可以了（固定经验值）。



容器内存

yarn.nodemanager.resource.memory-mb

NodeManager Default Group

46 吉字节

NodeManager Group 1

27 吉字节

NodeManager Group 2

26 吉字节

Cloudera Manager 集群 主机 诊断 审核 图表 管理

Itcast_Cluster

YARN (MR2 Included)

状态 实例 配置 命令 应用程序 资源池 图表库 审核 Web UI 快速链接

搜索

9月2, 凌晨1点27分 CST

筛选器

- 状态
 - 运行状况良好 5
- 授权状态
- 维护模式
- 机架
- 角色组
- 角色类型
- 状态
- 运行状况测试

已选定的操作 添加角色实例 角色组

角色类型	状态	主机	授权状态	角色组
JobHistory Server	已启动	hadoop03	已授权	JobHistory Server Default Group
NodeManager	已启动	hadoop01	已授权	NodeManager Group 1
NodeManager	已启动	hadoop03	已授权	NodeManager Default Group
NodeManager	已启动	hadoop02	已授权	NodeManager Group 2
ResourceManager (活动)	已启动	hadoop03	已授权	ResourceManager Default Group

通过CM所有主机查看剩余内存：

可以看到第一台剩余内存为 $31.3 - 4.1 = 27.2\text{G}$ 。



注意，要同时设置yarn.scheduler.maximum-allocation-mb为一样的值，
yarn.app.mapreduce.am.command-opts (JVM内存) 的值要同步修改为略小的值 (-Xmx1024m)。



3.2.1.3 本地目录

yarn.nodemanager.local-dirs (Yarn)

NodeManager 存储中间数据文件的本地文件系统中的目录列表。

如果单台服务器上有多个磁盘挂载，则配置的值应当是分布在各个磁盘上目录，这样可以充分利用节点的IO读写能力。

NodeManager 本地目录

yarn.nodemanager.local-dirs
编辑相同值

NodeManager Default Group

/yarn/nm

NodeManager Group 1

/yarn/nm

NodeManager Group 2

/yarn/nm

3.2.2 MapReduce内存配置

当MR内存溢出时，可以根据服务器配置进行调整。

mapreduce.map.memory.mb

为作业的每个 Map 任务分配的物理内存量(MiB)，默认为0，自动判断大小。

mapreduce.reduce.memory.mb

为作业的每个 Reduce 任务分配的物理内存量(MiB)，默认为0，自动判断大小。

mapreduce.map.java.opts、mapreduce.reduce.java.opts

Map和Reduce的JVM配置选项。

注意：

mapreduce.map.java.opts一定要小于mapreduce.map.memory.mb；

mapreduce.reduce.java.opts一定要小于mapreduce.reduce.memory.mb，格式-Xmx4096m。

注意：

此部分所有配置均不能大于Yarn的NodeManager内存配置。





Map 任务内存

mapreduce.map.memory.mb

Gateway Default Group

10

吉字节

Reduce 任务内存

mapreduce.reduce.memory.mb

Gateway Default Group

15

吉字节

Map 任务 Java 选项库

mapreduce.map.java.opts

Gateway Default Group

-Djava.net.preferIPv4Stack=true -Xmx4096m

Reduce 任务 Java 选项库

mapreduce.reduce.java.opts

Gateway Default Group

-Djava.net.preferIPv4Stack=true -Xmx10240m

3.3 Hive基础配置

3.3.1 HiveServer2 的 Java 堆栈

Hiveserver2异常退出，导致连接失败的问题。



Itcast_Cluster / Hive / HiveServer2 / hadoop03

进程状态

6月9, 凌晨2点54分 CST

测试该角色的进程状态是否符合预期。

不良：该角色的进程已退出。该角色的预期状态为已启动。

操作

- 为 HiveServer2 Default Group 角色组中的所有角色更改 HiveServer2 进程运行状况检查
- 为此角色实例更改 HiveServer2 进程运行状况检查
- 在执行运行状况测试时查看此角色实例的日志

建议

该 HiveServer2 运行状况测试用于检查 HiveServer2 主机上的 Cloudera Manager Agent 是否运行正常，以及与 HiveServer2 角色相关的进程是否处于 Cloudera Manager 预期的状态。

该运行状况测试失败可能表示 HiveServer2 进程出现问题，无法连接到 HiveServer2 主机上的 Cloudera Manager Agent 或 Cloudera Manager Agent 出现问题。HiveServer2 崩溃或 HiveServer2 未及时启动或停止都会导致该测试失败。请查看 HiveServer2 日志了解更多信息。如果因为与 HiveServer2 主机上的 Cloudera Manager Agent 通信问题而使测试失败，请通过运行 HiveServer2 主机上的 `/etc/init.d/cloudera-scm-agent` 状态来检查 Cloudera Manager Agent 的状态，或查看 HiveServer2 主机上的 Cloudera Manager Agent 日志了解更多信息。

可使用 **HiveServer2 进程运行状况检查** HiveServer2 监控设置启用或禁用该测试。

解决方法：修改HiveServer2 的 Java 堆栈大小。

HiveServer2 的 Java 堆栈大小 (字节)

HiveServer2 Default Group

3

吉字节

3.3.2 Hive并行编译

Hive默认同时只能编译一段HiveQL，并上锁。

将hive.driver.parallel.compilation设置为true，各个会话可以同时编译查询，提高团队工作效率。否则如果在UDF中执行了一段HiveQL，或者多个用户同时使用的话，就会锁住。

修改hive.driver.parallel.compilation.global.limit的值，0或负值为无限制，可根据团队人员和硬件进行修改，以保证同时编译查询。

Enable Parallel Compilation of Queries

hive.driver.parallel.compilation

☒ HiveServer2 Default Group

Query Compilation Degree of Parallelism

hive.driver.parallel.compilation.global.limit

HiveServer2 Default Group

10

3.3.3 动态生成分区的线程数

hive.load.dynamic.partitions.thread





用于加载**动态生成的分区的线程数**。加载需要将文件重命名为它的最终位置，并更新关于新分区的一些元数据。默认值为15。

当有大量动态生成的分区时，增加这个值可以提高性能。根据服务器配置修改。

Load Dynamic Partitions Thread Count
hive.load.dynamic.partitions.thread

HiveServer2 Default Group

15

3.3.4 监听输入文件线程数

hive.exec.input.listing.max.threads

Hive用来监听输入文件的最大线程数。默认值：15。

当需要读取大量分区时，增加这个值可以提高性能。根据服务器配置进行调整。

Input Listing Max Threads
hive.exec.input.listing.max.threads

HiveServer2 Default Group

15

3.4 压缩配置

3.4.1 Map输出压缩

除了创建表时指定保存数据时压缩，在查询分析过程中，Map的输出也可以进行压缩。由于map任务的输出需要写到磁盘并通过网络传输到reducer节点，所以通过使用LZO、LZ4或者Snappy这样的快速压缩方式，是可以获得性能提升的，因为需要传输的数据减少了。

MapReduce配置项：

- mapreduce.map.output.compress
设置是否**启动map输出压缩**，默认为false。在需要减少网络传输的时候，可以设置为true。
- mapreduce.map.output.compress.codec





设置map输出压缩编解码器，默认为org.apache.hadoop.io.compress.DefaultCodec，**推荐使用SnappyCodec**：org.apache.hadoop.io.compress.SnappyCodec。

压缩 Map 输出

mapreduce.map.output.compress

☒ Gateway Default Group

MapReduce Map 输出的压缩编解码器

mapreduce.map.output.compress.codec

Gateway Default Group

org.apache.hadoop.io.compress.SnappyCodec

3.4.2 Reduce结果压缩

是否对**任务输出结果压缩**，默认值false。对传输数据进行压缩，既可以减少文件的存储空间，又可以加快数据在网络不同节点之间的传输速度。

配置项：

1. mapreduce.output.fileoutputformat.compress

是否**启用 MapReduce 作业输出压缩**。

2. mapreduce.output.fileoutputformat.compress.codec

指定要使用的**压缩编解码器**，推荐**SnappyCodec**。

3. mapreduce.output.fileoutputformat.compress.type

指定MapReduce作业输出的**压缩方式**，默认值RECORD，可配置值有：NONE、RECORD、BLOCK。**推荐使用BLOCK**，即针对一组记录进行批量压缩，压缩效率更高。

压缩 MapReduce 作业输出

mapreduce.output.fileoutputformat.compress

☒ Gateway Default Group

MapReduce 作业输出的压缩编解码器

mapreduce.output.fileoutputformat.compress.codec

Gateway Default Group

org.apache.hadoop.io.compress.SnappyCodec

MapReduce 作业输出的压缩类型

mapreduce.output.fileoutputformat.compress.type

Gateway Default Group

☐ NONE

☒ BLOCK

☐ RECORD





3.4.3 Hive压缩

3.4.3.1 多个Map-Reduce中间数据压缩

控制 Hive 在多个map-reduce作业之间生成的中间文件是否被压缩。压缩编解码器和其他选项由上面Hive通用压缩mapreduce.output.fileoutputformat.compress.*确定。

```
set hive.exec.compress.intermediate=true;
```

3.4.3.2 Hive最终结果压缩

控制是否压缩查询的最终输出(到 local/hdfs 文件或 Hive table)。压缩编解码器和其他选项由上面Hive通用压缩mapreduce.output.fileoutputformat.compress.*确定。

```
set hive.exec.compress.output=true;
```

3.5 其他

3.5.1 JVM重用（不再支持）

随着Hadoop版本的升级，已自动优化了JVM重用选项，MRv2开始不再支持JVM重用。（旧版本配置项：mapred.job.reuse.jvm.num.tasks、mapreduce.job.jvm.numtasks）

3.5.2 Hive执行引擎（了解）

CDH支持的引擎包括MapReduce和Spark两种，可自由选择，Spark不一定比MR快，Hive2.x和Hadoop3.x经过多次优化，Hive-MR引擎的性能已经大幅提升。

配置项：hive.execution.engine

默认执行引擎

hive.execution.engine

HiveServer2 Default Group

☒ MapReduce

☐ Spark



CDH默认不支持Tez引擎：

https://docs.cloudera.com/documentation/enterprise/6/release-notes/topics/rg_cdh_620_unsupported_features.html

Apache Hive Unsupported Features

The following Hive features are not supported in CDH 6.2.x:

- AccumuloStorageHandler (HIVE-7068)
- ACID (HIVE-5317)
- Built-in `version()` function is not supported (CDH-40979)
- Cost-based Optimizer (CBO) and gathering column statistics required by CBO
- Explicit Table Locking
- HCatalog - HBase plugin
- Hive Authorization (Instead, use Apache Sentry.)
- Hive on Apache Tez
- Hive Local Mode Execution
- Hive Metastore - Derby
- Hive Web Interface (HWI)
- HiveServer1 / JDBC 1
- HiveServer2 Dynamic Service Discovery (HS2 HA) (HIVE-8376)
- HiveServer2 - HTTP Mode (Use THRIFT mode.)

4. 中文乱码

表名:	t_brand
表类型:	TABLE
模式:	yp_ods
表描述:	??????

列	字段名	#	数据类型	长度	标度	非空	自动生成	自动递增	缺省	描述
索引	id	1	STRING							??id
DDL	store_id	2	STRING							??...
Virtual	brand_pt...	3	STRING							????
	brand_na...	4	STRING							????
	brand_im...	5	STRING							??...
	initial	6	STRING							??
	sort	7	INT	10						0?...
	is_use	8	TINYINT	3						??...
	goods_st...	9	TINYINT	3						
	create_us...	10	STRING							
	create_ti...	11	STRING							
	update_u...	12	STRING							
	update_ti...	13	STRING							
	is_valid	14	TINYINT	3						0...
	dt	15	STRING							





在mysql的hive数据库中，运行以下脚本：

```
alter table COLUMNS_V2 modify column COMMENT varchar(256) character set utf8;  
alter table TABLE_PARAMS modify column PARAM_VALUE varchar(4000) character set utf8;  
alter table PARTITION_PARAMS modify column PARAM_VALUE varchar(4000) character set utf8;  
alter table PARTITION_KEYS modify column PKEY_COMMENT varchar(4000) character set utf8;  
alter table INDEX_PARAMS modify column PARAM_VALUE varchar(4000) character set utf8;
```

5. 建库

-- 建库

```
create database yp_ods;
```

库命名规范：业务简拼_ods，亿品新零售业务的ods层，可以命名为yp_ods。

表命名规范：ods层保持与原始数据一致，因此表名可以和原始表名一致，比如t_shop_order表，可以命名为yp_ods.t_shop_order。

6. 全量覆盖

6.1 数据格式和压缩格式

6.1.1 数据格式



6.1.1.1 列式存储和行式存储

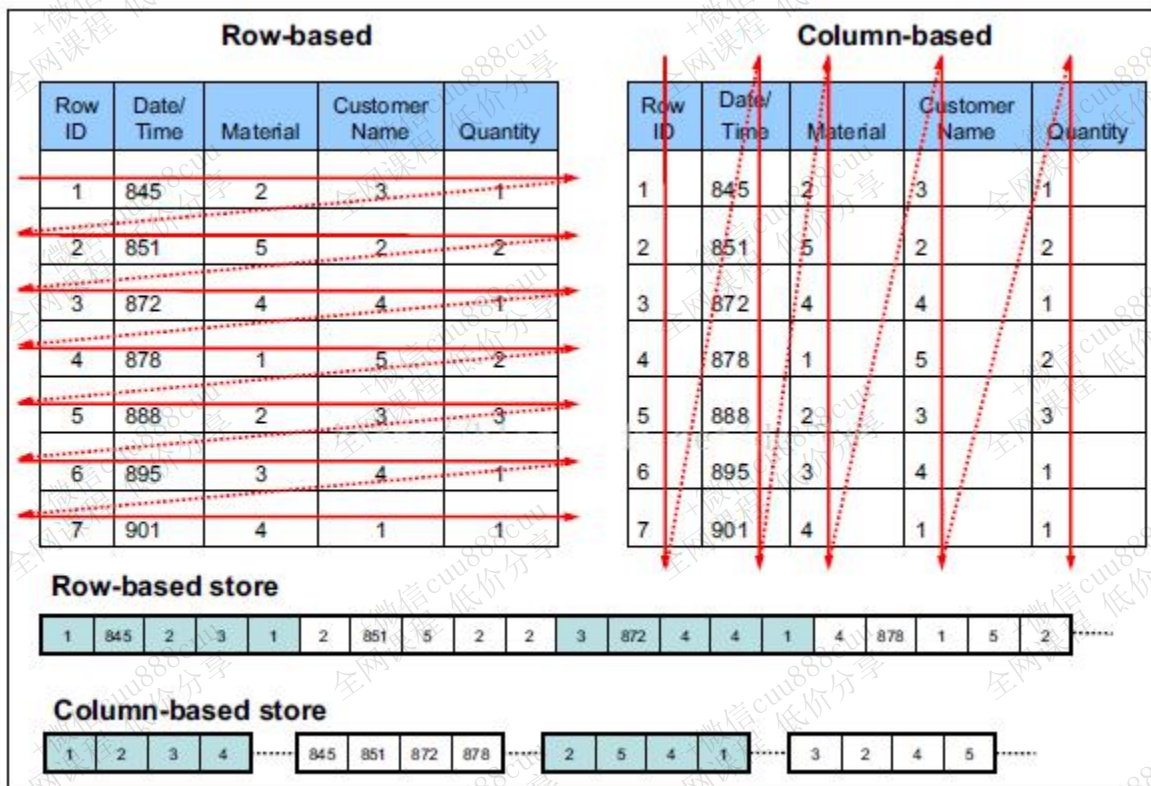


Figure 1-4 Row-based and column-based storage models

行存储的特点： 查询满足条件的一整行（所有列）数据的时候，列存储则需要去每个聚集的字段找到对应的每个列的值，行存储只需要找到其中一个值，其余的值都在相邻地方，所以此时行存储查询的速度更快。

列存储的特点： 因为每个字段的数据聚集存储，在查询只需要少数几个字段的时候，能大大减少读取的数据量；每个字段的数据类型一定是相同的，列式存储可以针对性的设计更好的设计压缩算法。

6.1.1.2 TEXTFILE

默认格式，**行式存储**。可结合Gzip、Bzip2使用(系统自动检查，执行查询时自动解压)，但使用这种方式，hive**不会对数据进行切分**，从而无法对数据进行并行操作。并且反序列化过程中，必须逐个字符判断是不是分隔符和行结束符，**性能较差**。

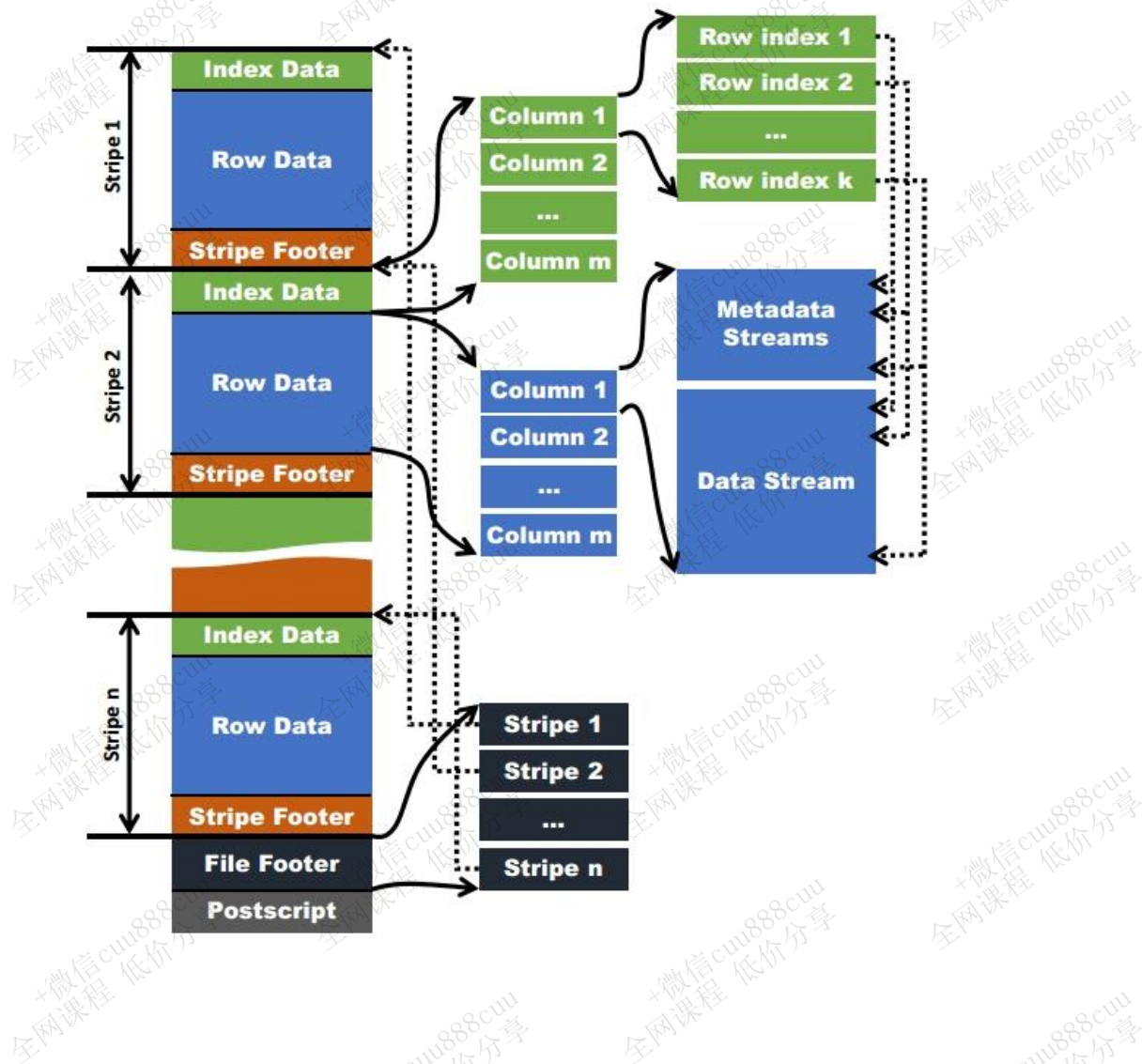
6.1.1.3 ORCFILE

使用ORC文件格式可以**提高hive读、写和处理数据的能力**。ORCFile是RCFile的升级版。



在ORC格式的hive表中，数据按行分块，每块按列存储。**结合了行存储和列存储的优点**。记录首先会被横向的切分为多个stripes，然后在每一个stripe内数据以列为单位进行存储，所有列的内容都保存在同一个文件中。

每个stripe的默认大小为256MB，相对于RCFile每个4MB的stripe而言，**更大的stripe**使ORC可以**支持索引**，数据读取更加高效。



6.1.2 存储和压缩结合

6.1.2.1 zlib压缩

优点：**压缩率比较高**；hadoop本身支持，在应用中处理gzip格式的文件就和直接处理文本一样。



缺点：压缩性能一般。

6.1.2.2 snappy压缩

优点：高速压缩速度和合理的压缩率。

缺点：压缩率比zlib要低；hadoop本身不支持，需要安装（CDH版本已自动支持，可忽略）。

6.1.3 系统采用的格式

因为ORCFILE的压缩快、存取快，而且拥有特有的查询优化机制，所以系统采用ORCFILE存储格式（RCFILE升级版），压缩算法采用orc支持的ZLIB和SNAPPY。

在ODS数据源层，因为数据量较大，可以采用orcfile+ZLIB的方式，以节省磁盘空间；

而在计算的过程中（DWD、DWM、DWS、APP），为了不影响执行的速度，采用orcfile+SNAPPY的方式，提升hive和presto的执行速度。

存储空间足够的情况下，推荐采用SNAPPY压缩。

6.2 区域表 t_district

6.2.1 SQL

```
DROP TABLE if exists yp_ods.t_district;
CREATE TABLE yp_ods.t_district
(
  `id` string COMMENT '主键ID',
  `code` string COMMENT '区域编码',
  `name` string COMMENT '区域名称',
  `pid` string COMMENT '父级ID',
  `alias` string COMMENT '别名'
)
comment '区域字典表'
row format delimited fields terminated by '\t' stored as orc tblproperties ('orc.compress'='ZLIB');
```



6.2.2 Sqoop

```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select * from t_district where l=1 and \$CONDITIONS" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_district \  
-m 1
```

6.3 日期表 t_date

6.3.1 SQL

```
drop table yp_ods.t_date;  
CREATE TABLE yp_ods.t_date  
(  
    dim_date_id          string COMMENT '日期',  
    date_code            string COMMENT '日期编码',  
    lunar_calendar       string COMMENT '农历',  
    year_code            string COMMENT '年code',  
    year_name            string COMMENT '年名称',  
    month_code           string COMMENT '月份编码',  
    month_name           string COMMENT '月份名称',  
    quarter_code         string COMMENT '季度编码',  
    quarter_name         string COMMENT '季度名称',  
    year_month           string COMMENT '年月',  
    year_week_code       string COMMENT '一年中第几周',  
    year_week_name       string COMMENT '一年中第几周名称',  
    year_week_code_cn    string COMMENT '一年中第几周 (中国)',  
    year_week_name_cn    string COMMENT '一年中第几周名称 (中国)',  
    week_day_code        string COMMENT '周几code',  
    week_day_name        string COMMENT '周几名称',  
    day_week             string COMMENT '周',  
    day_week_cn          string COMMENT '周 (中国)',
```





```

day_week_num      string COMMENT '一周第几天',
day_week_num_cn    string COMMENT '一周第几天 (中国)',
day_month_num      string COMMENT '一月第几天',
day_year_num       string COMMENT '一年第几天',
date_id_wow        string COMMENT '与本周环比的上周日期',
date_id_mom        string COMMENT '与本月环比的上月日期',
date_id_wyw        string COMMENT '与本周同比的上年日期',
date_id_mym        string COMMENT '与本月同比的上年日期',
first_date_id_month string COMMENT '本月第一天日期',
last_date_id_month  string COMMENT '本月最后一天日期',
half_year_code      string COMMENT '半年code',
half_year_name      string COMMENT '半年名称',
season_code         string COMMENT '季节编码',
season_name         string COMMENT '季节名称',
is_weekend          string COMMENT '是否周末 (周六和周日)',
official_holiday_code string COMMENT '法定节假日编码',
official_holiday_name string COMMENT '法定节假日',
festival_code       string COMMENT '节日编码',
festival_name       string COMMENT '节日',
custom_festival_code string COMMENT '自定义节日编码',
custom_festival_name string COMMENT '自定义节日',
update_time         string COMMENT '更新时间'
)
COMMENT '时间维度表'
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'ZLIB');

```

6.3.2 Sqoop

```

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select * from t_date where l=1 and \$CONDITIONS" \
--hcatalog-database yp_ods \
--hcatalog-table t_date \

```





-m 1

7. 增量同步

7.1 建表SQL

7.1.1 订单评价表 t_goods_evaluation

```
DROP TABLE if exists yp_ods.t_goods_evaluation;
CREATE TABLE yp_ods.t_goods_evaluation
(
    `id`                string,
    `user_id`           string COMMENT '评论人id',
    `store_id`          string COMMENT '店铺id',
    `order_id`          string COMMENT '订单id',
    `geval_scores`       INT COMMENT '综合评分',
    `geval_scores_speed` INT COMMENT '送货速度评分0-5分(配送评分)',
    `geval_scores_service` INT COMMENT '服务评分0-5分',
    `geval_isanony`      TINYINT COMMENT '0-匿名评价, 1-非匿名',
    `create_user`        string,
    `create_time`        string,
    `update_user`        string,
    `update_time`        string,
    `is_valid`           TINYINT COMMENT '0 : 失效, 1 : 开启'
)
comment '商品评价表'
partitioned by (dt string)
row format delimited fields terminated by '\t' stored as orc tblproperties ('orc.compress'='ZLIB');
```

7.1.2 Hive的分区

我们知道传统的OLTP数据库一般都具有索引和表分区的功能，通过表分区能够在特定的区域检索数据，**减少扫描成本**，在一定程度上**提高查询效率**，我们还可以通过建立索引进一步提升查询效率。在Hive数仓中也有索引和分区概念。



为了对表进行合理的管理以及提高查询效率，Hive可以将表组织成“分区”。

分区是表的部分列的集合，可以为频繁使用的数据建立分区，这样查找分区中的数据时就不需要扫描全表，这对于提高查找效率很有帮助。

分区是一种根据“分区列”（partition column）的值对表进行粗略划分的机制。Hive中每个分区对应着表很多的子目录，将所有数据按照分区列放入到不同的子目录中去。

7.1.2.1 为什么要分区

庞大的数据集可能需要耗费大量的时间去处理。在许多场景下，可以通过分区的方法减少每一次扫描总数据量，这种做法可以显著地改善性能。

数据会依照单个或多个列进行分区，通常按照时间、地域或者是商业维度进行分区。

比如电影表，分区的依据可以是电影的种类和评级，另外，按照拍摄时间划分可能会得到均匀的结果。

为了达到性能表现的一致性，对不同列的划分应该让数据尽可能均匀分布。最好的情况下，分区的划分条件总是能够对应where语句的部分查询条件，这样才能充分利用分区带来的性能优势。

主页 / user / hive / warehouse / itcast_dwd.db / itcast_intention_dwd

☐	名称	大小	用户	组
☐	名称		hue	hive
☐			hue	hive
☐	.hive-staging_hive_2020-06-20_04-47-39_078_8794509621116027026-1		hue	hive
☐	yearinfo=2011		hue	hive
☐	yearinfo=2012		hue	hive
☐	yearinfo=2013		hue	hive
☐	yearinfo=2014		hue	hive
☐	yearinfo=2015		hue	hive
☐	yearinfo=2016		hue	hive
☐	yearinfo=2017		hue	hive
☐	yearinfo=2018		hue	hive
☐	yearinfo=2019		hue	hive

Hive的分区使用HDFS的子目录功能实现。每一个子目录包含了分区对应的列名和每一列的值。但是由于HDFS并不支持大量的子目录，这也给分区的使用带来了限制。我们有必要对表中的分区数量进行预估，从而避免因分区数量过大带来一系列问题。

Hive查询通常使用分区的列作为查询条件。这样的做法可以指定MapReduce任务在HDFS中指定的子目录下完成扫描的工作。HDFS的文件目录结构可以像索引一样高效利用。

Hive(Inceptor)分区包括静态分区和动态分区。





7.1.2.2 静态分区

根据插入时是否需要手动指定分区可以分为：**静态分区**：导入数据时需要手动指定分区。
动态分区：导入数据时，系统可以动态判断目标分区。

1. 创建静态分区

直接在 **PARTITIONED BY** 后面跟上**分区键**、**类型**即可。（分区键不能和任何列重名）

语法：

```
CREATE [EXTERNAL] TABLE <table_name>
    (<col_name> <data_type> [, <col_name> <data_type> ...])
    -- 指定分区键和数据类型
    PARTITIONED BY (<partition_key> <data_type>, ...)
    [ROW FORMAT <row_format>]
    [STORED AS TEXTFILE|ORC|CSVFILE]
    [LOCATION '<file_path>']
    [TBLPROPERTIES ('<property_name>'='<property_value>', ...)];
```

栗子：

```
-- 分区字段主要是时间，按年分区
CREATE TABLE device_open (
    deviceid varchar(50),
    ...
)
PARTITIONED BY (year varchar(50))
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
```

2. 写入数据

语法：

```
-- 覆盖写入
INSERT OVERWRITE TABLE <table_name>
    PARTITION (<partition_key>=<partition_value>[, <partition_key>=<partition_value>, ...])
    SELECT <select_statement>;

-- 追加写入
```





```
INSERT INTO TABLE <table_name>
```

```
PARTITION (<partition_key>=<partition_value>[, <partition_key>=<partition_value>, ...])
```

```
SELECT <select_statement>;
```

栗子：

```
insert overwrite table device_open partition(year='2020')
```

```
select
```

```
...,
```

```
original_device_open.month as month,
```

```
original_device_open.day as day,
```

```
original_device_open.hour as hour
```

```
FROM original_device_open
```

7.1.2.3 动态分区

1. 创建

创建方式与静态分区表完全一样。

语法：

--分区字段主要是时间，分为年，月，日，时

```
CREATE TABLE device_open (
```

```
    deviceid varchar(50),
```

```
    ...
```

```
)
```

```
    PARTITIONED BY (year varchar(50), month varchar(50), day varchar(50), hour varchar(50))
```

```
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
```

2. 写入

动态分区只需要给出分区键名称。

语法：

-- 开启动态分区支持，并开启非严格模式

```
set hive.exec.dynamic.partition=true;
```




```
set hive.exec.dynamic.partition.mode=nonstrict;

insert overwrite table device_open partition(year,month,day,hour)
select
...
original_device_open.year as year,
original_device_open.month as month,
original_device_open.day as day,
original_device_open.hour as hour
FROM original_device_open
```

set hive.exec.dynamic.partition=true; 是开启动态分区

set hive.exec.dynamic.partition.mode=nonstrict; 这个属性默认值是strict，就是要求分区字段必须有一个是静态的分区值。全部动态分区插入，需要设置为nonstrict非严格模式。

代码中标红的部分，partition(year,month,day,hour) 就是要动态插入的分区。对于大批量数据的插入分区，动态分区相当方便。

7.1.2.4 静态分区和动态分区混用

一张表可同时被静态和动态分区键分区，只是动态分区键需要放在静态分区键的后面（因为HDFS上的动态分区目录下不能包含静态分区的子目录）。

静态分区键要用 <spk>=<value> 指定分区值；动态分区只需要给出分区键名称 <dpk>。

spk 即静态分区static partition key， dpk 即动态分区dynamic partition key。

比如：

```
insert overwrite table device_open partition(year='2017',month='05',day,hour)
select
...
original_device_open.day as day,
original_device_open.hour as hour
FROM original_device_open
where original_device_open.year='2017' and original_device_open.month='05'
```

partition(year='2017', month='05', day, hour)，year和month是静态分区字段，day和hour是动态分区字段，这里指将2017年5月份的数据插入分区表，对应底层的物理操作就是将2017年5月份



的数据load到hdfs上对应2017年5月份下的所有day和hour目录中去。

注意混用的情况下，静态分区上层必须也是静态分区，如果partition(year, month, day='05', hour='08')，则会报错：FAILED: SemanticException [Error 10094]: Line 1:50 Dynamic partition cannot be the parent of a static partition "day"。

7.1.2.5 有序动态分区

注意，如果个人电脑性能不好，出现因为动态分区而导致的内存溢出问题，可以设置hive.optimize.sort.dynamic.partition进行避免：

启用有序动态分区优化程序

hive.optimize.sort.dynamic.partition

☒ HiveServer2 Default Group

设置为true后，当启用动态分区时，reducer仅随时保持一个记录写入程序，从而降低对reducer产生的内存压力。但同时也会使查询性能变慢。

动态分区其他相关属性设置：

名称	默认值	描述
hive.exec.dynamic.partition	false	设置为true用于打开动态分区功能
hive.exec.dynamic.partition.mode	strict	设置为nonstrict能够让所有的分区都动态被设定，否则的话至少需要指定一个分区值
hive.exec.max.dynamic.partitions.pernode	100	能被每个mapper或者reducer创建的最大动态分区的数目，如果一个mapper或者reducer试图创建多余这个值的动态分区数目，会引发错误
hive.exec.max.dynamic.partitions	+1000	被一条带有动态分区的SQL语句所能创建的动态分区总量，如果超出限制会报出错误
hive.exec.max.created.files	100000	全局能被创建文件数目的最大值，专门有一个hadoop计数器来跟踪该值，如果超出会报错

7.1.3 登录记录表 t_user_login





```
DROP TABLE if exists yp_ods.t_user_login;
CREATE TABLE yp_ods.t_user_login(
    id string,
    login_user string,
    login_type string COMMENT '登录类型（登陆时使用）',
    client_id string COMMENT '推送标示id(登录、第三方登录、注册、支付回调、给用户推送消息时使用)',
    login_time string,
    login_ip string,
    logout_time string
)
COMMENT '用户登录记录表'
partitioned by (dt string)
row format delimited fields terminated by '\t'
stored as orc tblproperties ('orc.compress' = 'ZLIB');
```

7.1.4 订单组支付表 t_order_pay

```
DROP TABLE if exists yp_ods.t_order_pay;
CREATE TABLE yp_ods.t_order_pay
(
    id string,
    group_id string COMMENT '关联shop_order_group的group_id,一对多订单',
    order_pay_amount DECIMAL(11,2) COMMENT '订单总金额;',
    create_date string COMMENT '订单创建的时间,需要根据订单创建时间进行判断订单是否已经失效',
    create_user string,
    create_time string,
    update_user string,
    update_time string,
    is_valid TINYINT COMMENT '是否有效 0: false; 1: true; 订单是否有效的标志'
)
comment '订单支付表'
partitioned by (dt string) row format delimited fields terminated by '\t' stored as orc tblproperties
('orc.compress' = 'ZLIB');
```

7.2 Sqoop

7.2.1 首次

```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect 'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_goods_evaluation where 1=1 \${CONDITIONS}" \
--hcatalog-database yp_ods \
```





```
--hcatalog-table t_goods_evaluation \  
-m 1  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_user_login where 1=1 and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_user_login \  
-m 1  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_order_pay where 1=1 and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_order_pay \  
-m 1
```

7.2.2 循环

```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_goods_evaluation where 1=1 and (create_time between  
'${TD_DATE} 00:00:00' and '${TD_DATE} 23:59:59') and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_goods_evaluation \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
```



```
true' \  
  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_user_login where 1=1 and (login_time between '${TD_DATE}' 00:00:00' and '${TD_DATE}' 23:59:59') and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_user_login \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_order_pay where 1=1 and (create_time between '${TD_DATE}' 00:00:00' and '${TD_DATE}' 23:59:59') and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_order_pay \  
-m 1  
wait
```

8. 新增和更新同步

8.1 建表语句

8.1.1 店铺表 t_store

```
DROP TABLE if exists yp_ods.t_store;  
CREATE TABLE yp_ods.t_store  
(  
    `id`                string COMMENT '主键',  
    `user_id`           string,  
    `store_avatar`      string COMMENT '店铺头像',  
    `address_info`      string COMMENT '店铺详细地址',  
    `name`              string COMMENT '店铺名称',  
    `store_phone`       string COMMENT '联系电话',  
    `province_id`       INT COMMENT '店铺所在省份ID',  
    `city_id`           INT COMMENT '店铺所在城市ID',  
    `area_id`           INT COMMENT '店铺所在县ID',  
    `mb_title_img`      string COMMENT '手机店铺 页头背景图',
```



```
`store_description` string COMMENT '店铺描述',
`notice` string COMMENT '店铺公告',
`is_pay_bond` TINYINT COMMENT '是否有交过保证金 1: 是 0: 否',
`trade_area_id` string COMMENT '归属商圈ID',
`delivery_method` TINYINT COMMENT '配送方式 1: 自提; 3: 自提加配送均可; 2: 商家配送',
`origin_price` DECIMAL,
`free_price` DECIMAL,
`store_type` INT COMMENT '店铺类型 22天街网店 23实体店 24直营店铺 33会员专区店',
`store_label` string COMMENT '店铺logo',
`search_key` string COMMENT '店铺搜索关键字',
`end_time` string COMMENT '营业结束时间',
`start_time` string COMMENT '营业开始时间',
`operating_status` TINYINT COMMENT '营业状态 0: 未营业; 1: 正在营业',
`create_user` string,
`create_time` string,
`update_user` string,
`update_time` string,
`is_valid` TINYINT COMMENT '0关闭, 1开启, 3店铺申请中',
`state` string COMMENT '可使用的支付类型: MONEY金钱支付; CASHCOUPON现金券支付',
`idCard` string COMMENT '身份证',
`deposit_amount` DECIMAL(11,2) COMMENT '商圈认购费用总额',
`delivery_config_id` string COMMENT '配送配置表关联ID',
`aip_user_id` string COMMENT '通联支付标识ID',
`search_name` string COMMENT '模糊搜索名称字段: 名称_+真实名称',
`automatic_order` TINYINT COMMENT '是否开启自动接单功能 1: 是 0: 否',
`is_primary` TINYINT COMMENT '是否是总店 1: 是 2: 不是',
`parent_store_id` string COMMENT '父级店铺的id, 只有当is_primary类型为2时有效'
)
comment '店铺表'
partitioned by (dt string) row format delimited fields terminated by '\t' stored as orc tblproperties
('orc.compress'='ZLIB');
```

8.1.2 商圈表 t_trade_area

```
DROP TABLE if exists yp_ods.t_trade_area;
CREATE TABLE yp_ods.t_trade_area
(
`id` string COMMENT '主键',
`user_id` string COMMENT '用户ID',
`user_allinpay_id` string COMMENT '通联用户表id',
`trade_avatar` string COMMENT '商圈logo',
`name` string COMMENT '商圈名称',
`notice` string COMMENT '商圈公告',
`distric_province_id` INT COMMENT '商圈所在省份ID',
`distric_city_id` INT COMMENT '商圈所在城市ID',
`distric_area_id` INT COMMENT '商圈所在县ID',
`address` string COMMENT '商圈地址',
`radius` double COMMENT '半径',
```





```
`mb_title_img`      string COMMENT '手机商圈 页头背景图',
`deposit_amount`    DECIMAL(11,2) COMMENT '商圈认购费用总额',
`hava_deposit`      INT COMMENT '是否有交过保证金 1: 是 0: 否',
`state`             TINYINT COMMENT '申请商圈状态 -1 : 未认购 ; 0 : 申请中; 1 : 已认购 ; ',
`search_key`        string COMMENT '商圈搜索关键字',
`create_user`       string,
`create_time`       string,
`update_user`       string,
`update_time`       string,
`is_valid`          TINYINT COMMENT '是否有效 0: false; 1: true'
)
comment '商圈表'
partitioned by (dt string) row format delimited fields terminated by '\t' stored as orc tblproperties
('orc.compress'='ZLIB');
```

8.1.3 地址信息表 t_location

```
DROP TABLE if exists yp_ods.t_location;
CREATE TABLE yp_ods.t_location
(
  `id` string COMMENT '主键',
  `type` INT COMMENT '类型 1: 商圈地址; 2: 店铺地址; 3. 用户地址管理; 4. 订单买家地址信息; 5. 订单卖家地址信息',
  `correlation_id` string COMMENT '关联表id',
  `address` string COMMENT '地图地址详情',
  `latitude` double COMMENT '纬度',
  `longitude` double COMMENT '经度',
  `street_number` string COMMENT '门牌',
  `street` string COMMENT '街道',
  `district` string COMMENT '区县',
  `city` string COMMENT '城市',
  `province` string COMMENT '省份',
  `business` string COMMENT '百度商圈字段, 代表此点所属的商圈',
  `create_user` string,
  `create_time` string,
  `update_user` string,
  `update_time` string,
  `is_valid` TINYINT COMMENT '是否有效 0: false; 1: true',
  `adcode` string COMMENT '百度adcode, 对应区县code'
)
comment '地址信息'
partitioned by (dt string) row format delimited fields terminated by '\t' stored as orc tblproperties
('orc.compress'='ZLIB');
```

8.1.4 店铺商品表 t_goods

```
DROP TABLE if exists yp_ods.t_goods;
```





```
CREATE TABLE yp_ods.t_goods
(
  `id` string,
  `store_id` string COMMENT '所属商店ID',
  `class_id` string COMMENT '分类id:只保存最后一层分类id',
  `store_class_id` string COMMENT '店铺分类id',
  `brand_id` string COMMENT '品牌id',
  `goods_name` string COMMENT '商品名称',
  `goods_specification` string COMMENT '商品规格',
  `search_name` string COMMENT '模糊搜索名称字段:名称_+真实名称',
  `goods_sort` INT COMMENT '商品排序',
  `goods_market_price` DECIMAL(11,2) COMMENT '商品市场价',
  `goods_price` DECIMAL(11,2) COMMENT '商品销售价格(原价)',
  `goods_promotion_price` DECIMAL(11,2) COMMENT '商品促销价格(售价)',
  `goods_storage` INT COMMENT '商品库存',
  `goods_limit_num` INT COMMENT '购买限制数量',
  `goods_unit` string COMMENT '计量单位',
  `goods_state` TINYINT COMMENT '商品状态 1正常, 2下架, 3违规(禁售)',
  `goods_verify` TINYINT COMMENT '商品审核状态: 1通过, 2未通过, 3审核中',
  `activity_type` TINYINT COMMENT '活动类型: 0无活动 1促销 2秒杀 3折扣',
  `discount` INT COMMENT '商品折扣(%)',
  `seckill_begin_time` string COMMENT '秒杀开始时间',
  `seckill_end_time` string COMMENT '秒杀结束时间',
  `seckill_total_pay_num` INT COMMENT '已秒杀数量',
  `seckill_total_num` INT COMMENT '秒杀总数限制',
  `seckill_price` DECIMAL(11,2) COMMENT '秒杀价格',
  `top_it` TINYINT COMMENT '商品置顶: 1-是, 0-否',
  `create_user` string,
  `create_time` string,
  `update_user` string,
  `update_time` string,
  `is_valid` TINYINT COMMENT '0 : 失效, 1 : 开启'
)
comment '商品表_店铺(SKU)'
partitioned by (dt string) row format delimited fields terminated by '\t' stored as orc tblproperties
('orc.compress'='ZLIB');
```

8.1.5 商品分类表 t_goods_class

```
DROP TABLE if exists yp_ods.t_goods_class;
CREATE TABLE yp_ods.t_goods_class
(
  `id` string,
  `store_id` string COMMENT '店铺id',
  `class_id` string COMMENT '对应的平台分类id',
  `name` string COMMENT '店铺内分类名字',
  `parent_id` string COMMENT '父id',
  `level` TINYINT COMMENT '分类层级',
  `is_parent_node` TINYINT COMMENT '是否为父节点: 1是 0否',

```





```
`background_img` string COMMENT '背景图片',
`img` string COMMENT '分类图片',
`keywords` string COMMENT '关键词',
`title` string COMMENT '搜索标题',
`sort` INT COMMENT '排序',
`note` string COMMENT '类型描述',
`url` string COMMENT '分类的链接',
`is_use` TINYINT COMMENT '是否使用:0否,1是',
`create_user` string,
`create_time` string,
`update_user` string,
`update_time` string,
`is_valid` TINYINT COMMENT '0 : 失效, 1 : 开启'
)
comment '商品分类_店铺'
partitioned by (dt string) row format delimited fields terminated by '\t' stored as orc tblproperties
('orc.compress'='ZLIB');
```

8.1.6 品牌表 t_brand

```
DROP TABLE if exists yp_ods.t_brand;
CREATE TABLE yp_ods.t_brand
(
  `id` string,
  `store_id` string COMMENT '店铺id',
  `brand_pt_id` string COMMENT '平台品牌库品牌id',
  `brand_name` string COMMENT '品牌名称',
  `brand_image` string COMMENT '品牌图片',
  `initial` string COMMENT '品牌首字母',
  `sort` INT COMMENT '排序',
  `is_use` TINYINT COMMENT '0禁用1启用',
  `goods_state` TINYINT COMMENT '商品品牌审核状态 1 审核中,2 通过,3 拒绝',
  `create_user` string,
  `create_time` string,
  `update_user` string,
  `update_time` string,
  `is_valid` TINYINT COMMENT '0 : 失效, 1 : 开启'
)
comment '品牌 (店铺)'
partitioned by (dt string) row format delimited fields terminated by '\t' stored as orc tblproperties
('orc.compress'='ZLIB');
```

8.1.7 订单表 t_shop_order

```
DROP TABLE if exists yp_ods.t_shop_order;
CREATE TABLE yp_ods.t_shop_order
(
```




```

`id`          string COMMENT '根据一定规则生成的订单编号',
`order_num`   string COMMENT '订单序号',
`buyer_id`    string COMMENT '买家的userId',
`store_id`    string COMMENT '店铺id',
`order_from`  TINYINT COMMENT '是来自于app还是小程序,或者pc 1.安卓; 2.ios; 3.小程序H5 ; 4.PC',
`order_state` INT COMMENT '订单状态:1.已下单; 2.已付款; 3. 已确认 ;4. 配送; 5.已完成; 6.退款;7.
已取消',
`create_date` string COMMENT '下单时间',
`finnshed_time` timestamp COMMENT '订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需
要根据订单完成时间,进行自动收货以及自动评价',
`is_settlement` TINYINT COMMENT '是否结算;0.待结算订单; 1.已结算订单;',
`is_delete`    TINYINT COMMENT '订单评价的状态:0.未删除; 1.已删除;(默认0)',
`evaluation_state` TINYINT COMMENT '订单评价的状态:0.未评价; 1.已评价;(默认0)',
`way`          string COMMENT '取货方式:SELF自提;SHOP店铺负责配送',
`is_stock_up`  INT COMMENT '是否需要备货 0: 不需要 1: 需要 2:平台确认备货 3:已完成备货 4平
台已经将货物送至店铺 ',
`create_user`  string,
`create_time`  string,
`update_user`  string,
`update_time`  string,
`is_valid`     TINYINT COMMENT '是否有效 0: false; 1: true; 订单是否有效的标志'
)
comment '订单表'
partitioned by (dt string)
row format delimited fields terminated by '\t'
stored as orc tblproperties ('orc.compress' = 'ZLIB');

```

8.1.8 订单详情表 t_shop_order_address_detail

```

DROP TABLE if exists yp_ods.t_shop_order_address_detail;
CREATE TABLE yp_ods.t_shop_order_address_detail
(
  `id`          string COMMENT '关联订单id',
  `order_amount` DECIMAL(11,2) COMMENT '订单总金额:购买总金额-优惠金额',
  `discount_amount` DECIMAL(11,2) COMMENT '优惠金额',
  `goods_amount` DECIMAL(11,2) COMMENT '用户购买的商品的总金额+运费',
  `is_delivery`  string COMMENT '0.自提; 1.配送',
  `buyer_notes`  string COMMENT '买家备注留言',
  `pay_time`     string,
  `receive_time` string,
  `delivery_begin_time` string,
  `arrive_store_time`  string,
  `arrive_time`       string COMMENT '订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需
要根据订单完成时间,进行自动收货以及自动评价',
  `create_user`  string,
  `create_time`  string,
  `update_user`  string,
  `update_time`  string,
  `is_valid`     TINYINT COMMENT '是否有效 0: false; 1: true; 订单是否有效的标志'
)

```



```
comment '订单详情表'  
partitioned by (dt string) row format delimited fields terminated by '\t' stored as orc  
tblproperties ('orc.compress' = 'ZLIB');
```

8.1.9 商品评价明细表 t_goods_evaluation_detail

```
DROP TABLE if exists yp_ods.t_goods_evaluation_detail;  
CREATE TABLE yp_ods.t_goods_evaluation_detail  
(  
    `id` string,  
    `user_id` string COMMENT '评论人id',  
    `store_id` string COMMENT '店铺id',  
    `goods_id` string COMMENT '商品id',  
    `order_id` string COMMENT '订单id',  
    `order_goods_id` string COMMENT '订单商品表id',  
    `geval_scores_goods` INT COMMENT '商品评分0-10分',  
    `geval_content` string,  
    `geval_content_superaddition` string COMMENT '追加评论',  
    `geval_addtime` string COMMENT '评论时间',  
    `geval_addtime_superaddition` string COMMENT '追加评论时间',  
    `geval_state` TINYINT COMMENT '评价状态 1-正常 0-禁止显示',  
    `geval_remark` string COMMENT '管理员对评价的处理备注',  
    `revert_state` TINYINT COMMENT '回复状态0未回复1已回复',  
    `geval_explain` string COMMENT '管理员回复内容',  
    `geval_explain_superaddition` string COMMENT '管理员追加回复内容',  
    `geval_explaintime` string COMMENT '管理员回复时间',  
    `geval_explaintime_superaddition` string COMMENT '管理员追加回复时间',  
    `create_user` string,  
    `create_time` string,  
    `update_user` string,  
    `update_time` string,  
    `is_valid` TINYINT COMMENT '0 : 失效, 1 : 开启'  
)  
comment '商品评价明细'  
partitioned by (dt string) row format delimited fields terminated by '\t' stored as orc tblproperties  
('orc.compress'='ZLIB');
```



8.1.10 交易记录表 t_trade_record

```
DROP TABLE if exists yp_ods.t_trade_record;
CREATE TABLE yp_ods.t_trade_record
(
    id                string COMMENT '交易单号',
    external_trade_no string COMMENT '(支付, 结算. 退款) 第三方交易单号',
    relation_id       string COMMENT '关联单号',
    trade_type        TINYINT COMMENT '1. 支付订单; 2. 结算订单; 3. 退款订单; 4. 充值单; 5. 提现单; 6. 分销单; 7. 缴纳保证金单; 8. 退还保证金单; 9. 冻结通联订单; 10. 通联通账户余额充值; 11. 扫码单',
    status            TINYINT COMMENT '1. 成功; 2. 失败; 3. 进行中',
    finnshed_time     string COMMENT '订单完成时间, 当配送员点击确认送达时, 进行更新订单完成时间, 后期需要根据订单完成时间, 进行自动收货以及自动评价',
    fail_reason       string COMMENT '交易失败的原因',
    payment_type       string COMMENT '支付方式: 小程序, app, 微信, 支付宝, 快捷支付, 钱包, 银行卡, 消费券',
    trade_before_balance DECIMAL(11,2) COMMENT '交易前余额',
    trade_true_amount  DECIMAL(11,2) COMMENT '交易实际支付金额, 第三方平台扣除优惠以后实际支付金额',
    trade_after_balance DECIMAL(11,2) COMMENT '交易后余额',
    note              string COMMENT '业务说明',
    user_card          string COMMENT '第三方平台账户标识/多钱包用户钱包id',
    user_id            string COMMENT '用户id',
    aip_user_id        string COMMENT '钱包id',
    create_user        string,
    create_time        string,
    update_user        string,
    update_time        string,
    is_valid           TINYINT COMMENT '是否有效 0: false; 1: true; 订单是否有效的标志'
)
comment '所有交易记录信息'
partitioned by (dt string) row format delimited fields terminated by '\t' stored as orc tblproperties
('orc.compress' = 'ZLIB');
```

8.2 Sqoop

8.2.1 首次

```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_store where 1=1 and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_store \
```





```
-m 1
wait

/usr/bin/sqoop import -Dorg.apache.sqoop.splitter.allow_text_splitter=true
-Dorg.apache.sqoop.db.type=mysql \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_trade_area where 1=1 and \${CONDITIONS} order by id" \
--hcatalog-database yp_ods \
--hcatalog-table t_trade_area \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_location where 1=1 and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_location \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_goods where 1=1 and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_goods \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
```





```
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_goods_class where 1=1 and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_goods_class \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_brand where 1=1 and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_brand \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_shop_order where 1=1 and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_shop_order \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_shop_order_address_detail where 1=1 and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_shop_order_address_detail \
-m 1
wait
```





```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_order_settle where 1=1 and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_order_settle \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_refund_order where 1=1 and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_refund_order \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_shop_order_group where 1=1 and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_shop_order_group \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_shop_order_goods_details where 1=1 and \${CONDITIONS}"
```





```
\
--hcatalog-database yp_ods \
--hcatalog-table t_shop_order_goods_details \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_shop_cart where 1=1 and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_shop_cart \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_store_collect where 1=1 and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_store_collect \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_goods_collect where 1=1 and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_goods_collect \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
```





```
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_order_delievery_item where 1=1 and \${CONDITIONS}" \  
\  
--hcatalog-database yp_ods \  
--hcatalog-table t_order_delievery_item \  
-m 1  
wait  
\  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_goods_evaluation_detail where 1=1 and \${CONDITIONS}" \  
\  
--hcatalog-database yp_ods \  
--hcatalog-table t_goods_evaluation_detail \  
-m 1  
wait  
\  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_trade_record where 1=1 and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_trade_record \  
-m 1  
wait
```

8.2.2 循环

```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_trade_record where 1=1 and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_trade_record \  
-m 1  
wait
```





```
--password 123456 \  
  
--query "select *, '${TD_DATE}' as dt from t_store where l=1 and ((create_time between '${TD_DATE}'  
00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}' 00:00:00' and '${TD_DATE}'  
23:59:59')) and \${CONDITIONS}" \  
  
--hcatalog-database yp_ods \  
--hcatalog-table t_store \  
  
-m 1  
wait  
  
/usr/bin/sqoop import -Dorg.apache.sqoop.splitter.allow_text_splitter=true  
-Dorg.apache.sqoop.db.type=mysql \  
  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
  
--username root \  
--password 123456 \  
  
--query "select *, '${TD_DATE}' as dt from t_trade_area where l=1 and ((create_time between  
'${TD_DATE}' 00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}' 00:00:00'  
and '${TD_DATE}' 23:59:59')) and \${CONDITIONS} order by id" \  
  
--hcatalog-database yp_ods \  
--hcatalog-table t_trade_area \  
  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
  
--username root \  
--password 123456 \  
  
--query "select *, '${TD_DATE}' as dt from t_location where l=1 and ((create_time between '${TD_DATE}'  
00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}' 00:00:00' and '${TD_DATE}'  
23:59:59')) and \${CONDITIONS}" \  
  
--hcatalog-database yp_ods \  
--hcatalog-table t_location \  
  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
  
--username root \  
--password 123456 \  
  
--query "select *, '${TD_DATE}' as dt from t_goods where l=1 and ((create_time between '${TD_DATE}'  
00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}' 00:00:00' and '${TD_DATE}'
```





```
23:59:59')) and \${TD_DATE}' as dt from t_goods_class where 1=1 and ((create_time between
'${TD_DATE} 00:00:00' and '${TD_DATE} 23:59:59') or (update_time between '${TD_DATE} 00:00:00'
and '${TD_DATE} 23:59:59')) and \${CONDITIONS" \
--hcatalog-database yp_ods \
--hcatalog-table t_goods \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_goods_class where 1=1 and ((create_time between
'${TD_DATE} 00:00:00' and '${TD_DATE} 23:59:59') or (update_time between '${TD_DATE} 00:00:00'
and '${TD_DATE} 23:59:59')) and \${CONDITIONS" \
--hcatalog-database yp_ods \
--hcatalog-table t_goods_class \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_brand where 1=1 and ((create_time between '${TD_DATE}
00:00:00' and '${TD_DATE} 23:59:59') or (update_time between '${TD_DATE} 00:00:00' and '${TD_DATE}
23:59:59')) and \${CONDITIONS" \
--hcatalog-database yp_ods \
--hcatalog-table t_brand \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_shop_order where 1=1 and ((create_time between
'${TD_DATE} 00:00:00' and '${TD_DATE} 23:59:59') or (update_time between '${TD_DATE} 00:00:00'
and '${TD_DATE} 23:59:59')) and \${CONDITIONS" \
--hcatalog-database yp_ods \
--hcatalog-table t_shop_order
```





```
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_shop_order_address_detail where l=1 and ((create_time
between '${TD_DATE} 00:00:00' and '${TD_DATE} 23:59:59') or (update_time between '${TD_DATE}
00:00:00' and '${TD_DATE} 23:59:59')) and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_shop_order_address_detail \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_order_settle where l=1 and ((create_time between
 '${TD_DATE} 00:00:00' and '${TD_DATE} 23:59:59') or (update_time between '${TD_DATE} 00:00:00'
and '${TD_DATE} 23:59:59')) and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_order_settle \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_refund_order where l=1 and ((create_time between
 '${TD_DATE} 00:00:00' and '${TD_DATE} 23:59:59') or (update_time between '${TD_DATE} 00:00:00'
and '${TD_DATE} 23:59:59')) and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_refund_order \
-m 1
wait
```





```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_shop_order_group where l=1 and ((create_time between  
'${TD_DATE}' 00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}' 00:00:00'  
and '${TD_DATE}' 23:59:59')) and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_shop_order_group \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_shop_order_goods_details where l=1 and ((create_time  
between '${TD_DATE}' 00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}'  
00:00:00' and '${TD_DATE}' 23:59:59')) and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_shop_order_goods_details \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_shop_cart where l=1 and ((create_time between '${TD_DATE}'  
00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}' 00:00:00' and '${TD_DATE}'  
23:59:59')) and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_shop_cart \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  

```





```
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_store_collect where l=1 and ((create_time between  
'${TD_DATE}' 00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}' 00:00:00'  
and '${TD_DATE}' 23:59:59')) and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_store_collect \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_goods_collect where l=1 and ((create_time between  
'${TD_DATE}' 00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}' 00:00:00'  
and '${TD_DATE}' 23:59:59')) and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_goods_collect \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_order_delievery_item where l=1 and ((create_time  
between '${TD_DATE}' 00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}'  
00:00:00' and '${TD_DATE}' 23:59:59')) and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_order_delievery_item \  
-m 1  
wait  
  
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '${TD_DATE}' as dt from t_goods_evaluation_detail where l=1 and ((create_time  
between '${TD_DATE}' 00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}'
```





```
00:00:00' and '${TD_DATE}' 23:59:59')) and \${CONDITIONS" \
--hcatalog-database yp_ods \
--hcatalog-table t_goods_evaluation_detail \
-m 1
wait

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_trade_record where l=1 and ((create_time between
'${TD_DATE}' 00:00:00' and '${TD_DATE}' 23:59:59') or (update_time between '${TD_DATE}' 00:00:00'
and '${TD_DATE}' 23:59:59')) and \${CONDITIONS" \
--hcatalog-database yp_ods \
--hcatalog-table t_trade_record \
-m 1
wait
```

9. 脚本文件

ODS建表: [资料\create_ods_table.sql](#)

Sqoop全量导入脚本: [资料\sqoop_import.sh](#)

DWD层搭建

DWD明细层，数据粒度和ODS层一致，将事实表和维度表进行区分，对数据进行清洗转换。

比如t_date等原始表中的日期格式不统一，需要在DWD层统一转换为yyyy-MM-dd格式：

`concat(substr(dim_date_id,1,4), '-', substr(dim_date_id,5,2), '-', substr(dim_date_id,7,2)) as
dim_date_id`



1. 维度建模流程

掌握了数仓的相关概念和分层后，便可着手开始实际建设数仓。由于ODS层的数据是保持原貌从业务数据库中直接拉取，因此重点考虑DW层和APP层的事实表建设。

Kimball对于维度模型设计提出了四个步骤：选择业务过程、声明粒度、确定维度和确定事实，但是由于目前业务和技术的发展，不可避免地需要把一些维度冗余进事实表中，因此在建设事实表的过程中还需要考虑把哪些通用的维度冗余进去。

1、选择业务过程

与其说选择业务过程，不如说理解清楚需求(事实表)。当从零开始构建数仓时，需要在熟悉业务流程的基础上选择最重要、易实现、对下游数据影响大的业务过程。而当数仓的具备一些最基础的中间表后，无论是业务方从业务需要的角度，还是数仓同学基于整体规划的角度，或者是在建表的过程中发现了某些坑需要去填，其本质都是一个个对数仓的需求(事实表)，需要在理清需要的事实表后不断完善和丰富数仓的内容。

2、声明粒度

即每行是什么含义。在DWD层，其粒度往往是最小粒度，如表中每一行是不可再细分的订单维度、用户的每次操作记录等。而在DWS层，由于会进行一定程度的聚合，因此其粒度要较DWD层大，如每个品类、每个店铺、每一天的相关聚合信息。

3、确定维度

即表中所需要包括的环境信息，如类型、地区、日期、时间等维度。

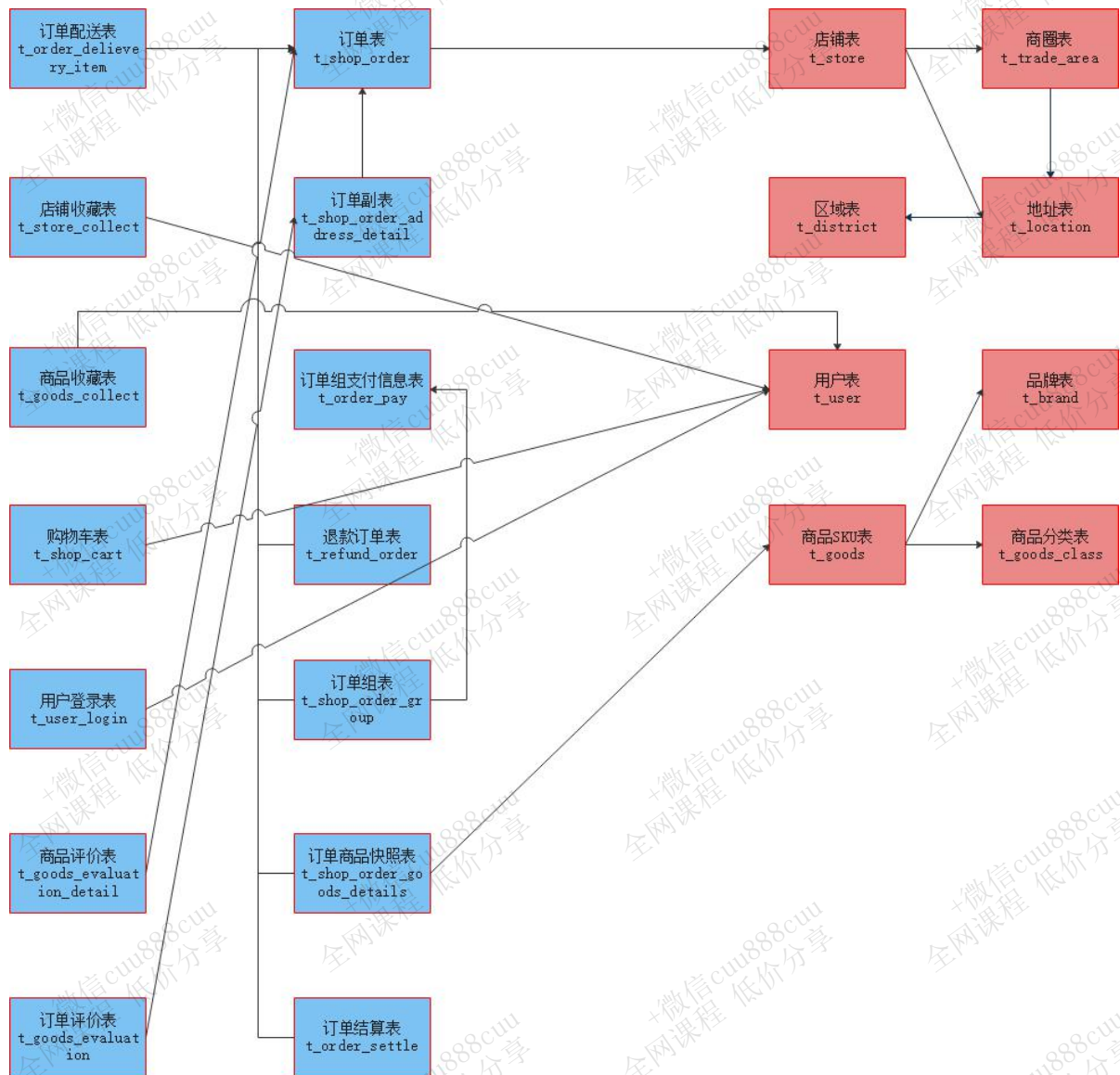
4、确定事实

表中需要度量的属性(字段)，如订单事实表中的金额、数量等，事实与维度共同构成了表中的所有列。一些不可加事实，如比率，记得要拆成可加事实，方便下游了解计算过程以及进行二次加工。

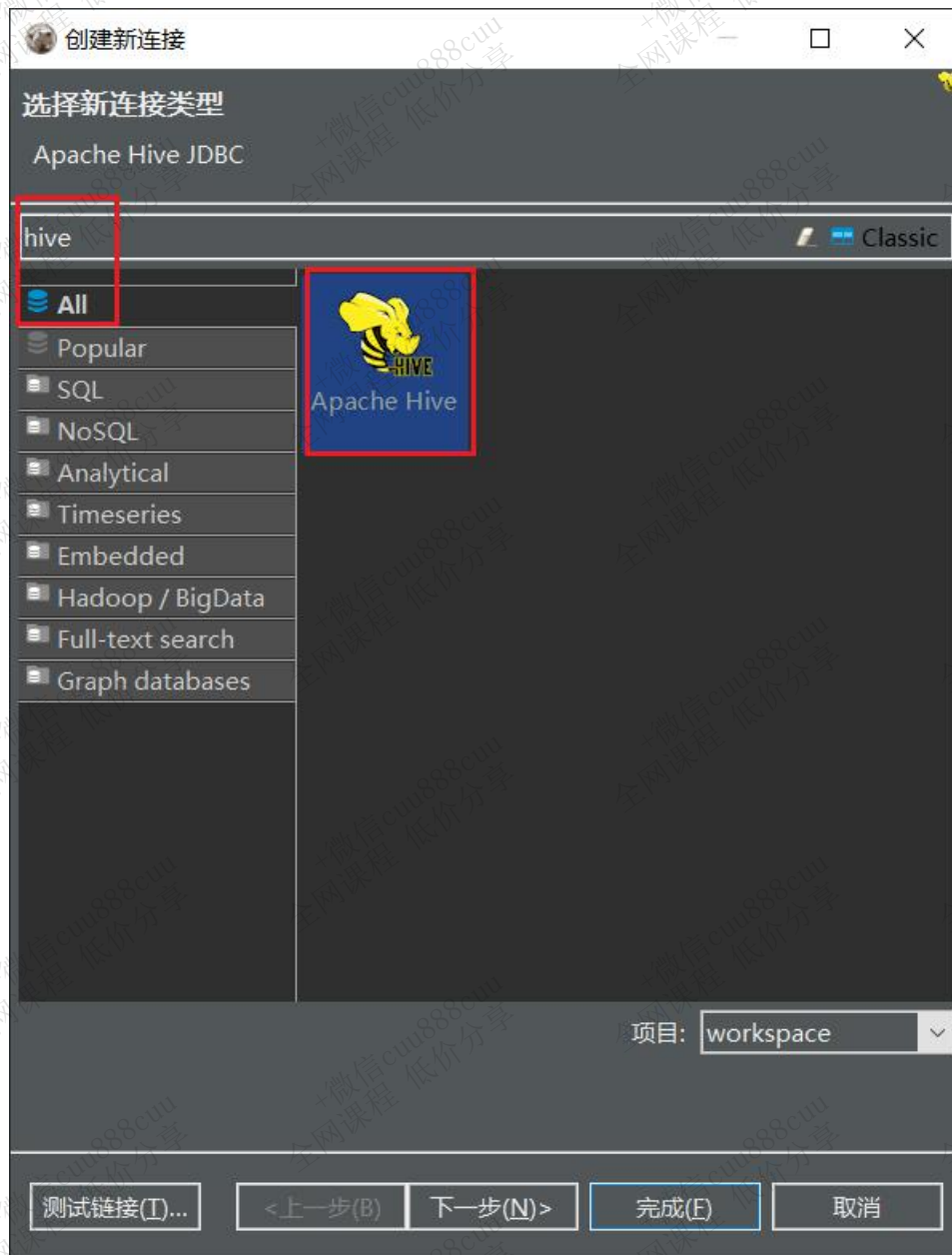
5、冗余维度

完全把事实表和维度表拆开，这是不现实也不符合效率的，因此建设过程中需要把一些通用高频的维度冗余进事实表中，以空间换时间的方法提升效率。





2. DBeaver连接Hive





创建新连接

通用 JDBC 连接设置

Hadoop / Apache Hive 连接设置

常规 驱动属性 SSH Proxy

JDBC URL: jdbc:hive2://localhost:10000

主机: localhost 端口: 10000

数据库/模式:

用户名:

密码: ☒ Save password locally

Advanced settings:

连接详情(名称、类型 ...)

驱动名称: Hadoop / Apache Hive

编辑驱动设置

测试链接(I)... <上一步(B) 下一步(N)> 完成(E) 取消



编辑驱动 'Apache Hive'

设置

驱动名称*: 驱动类型:

类名:

URL 模板:

默认端口: ☐ 嵌入 ☐ 无认证 ☐ Allow Empty Password

描述

目录: ID:

描述:

网址: <https://cwiki.apache.org/confluence/display/Hive/Home>

库 连接属性 高级参数

☐ <https://github.com/timveil/hive-jdbc-uber-jar/releases/download/v1.9-2.6.5/hive-jdbc-uber-jar-v1.9-2.6.5.jar>

☐ H:\JavaSoft\hive\cdh6.2.1\commons-httpclient-3.1.jar

☐ H:\JavaSoft\hive\cdh6.2.1\commons-logging-1.2.jar

☐ H:\JavaSoft\hive\cdh6.2.1\curator-client-2.12.0.jar

☐ H:\JavaSoft\hive\cdh6.2.1\curator-framework-2.12.0.jar

☐ H:\JavaSoft\hive\cdh6.2.1\hadoop-auth-3.0.0-cdh6.2.1.jar

☐ H:\JavaSoft\hive\cdh6.2.1\hadoop-common-3.0.0-cdh6.2.1.jar

☐ H:\JavaSoft\hive\cdh6.2.1\hive-cli-2.1.1-cdh6.2.1.jar

☐ H:\JavaSoft\hive\cdh6.2.1\hive-common-2.1.1-cdh6.2.1.jar

☐ H:\JavaSoft\hive\cdh6.2.1\hive-exec-2.1.1-cdh6.2.1.jar

添加文件(E)

添加文件夹(D)

添加工件(A)

下载/更新(D)

信息(I)

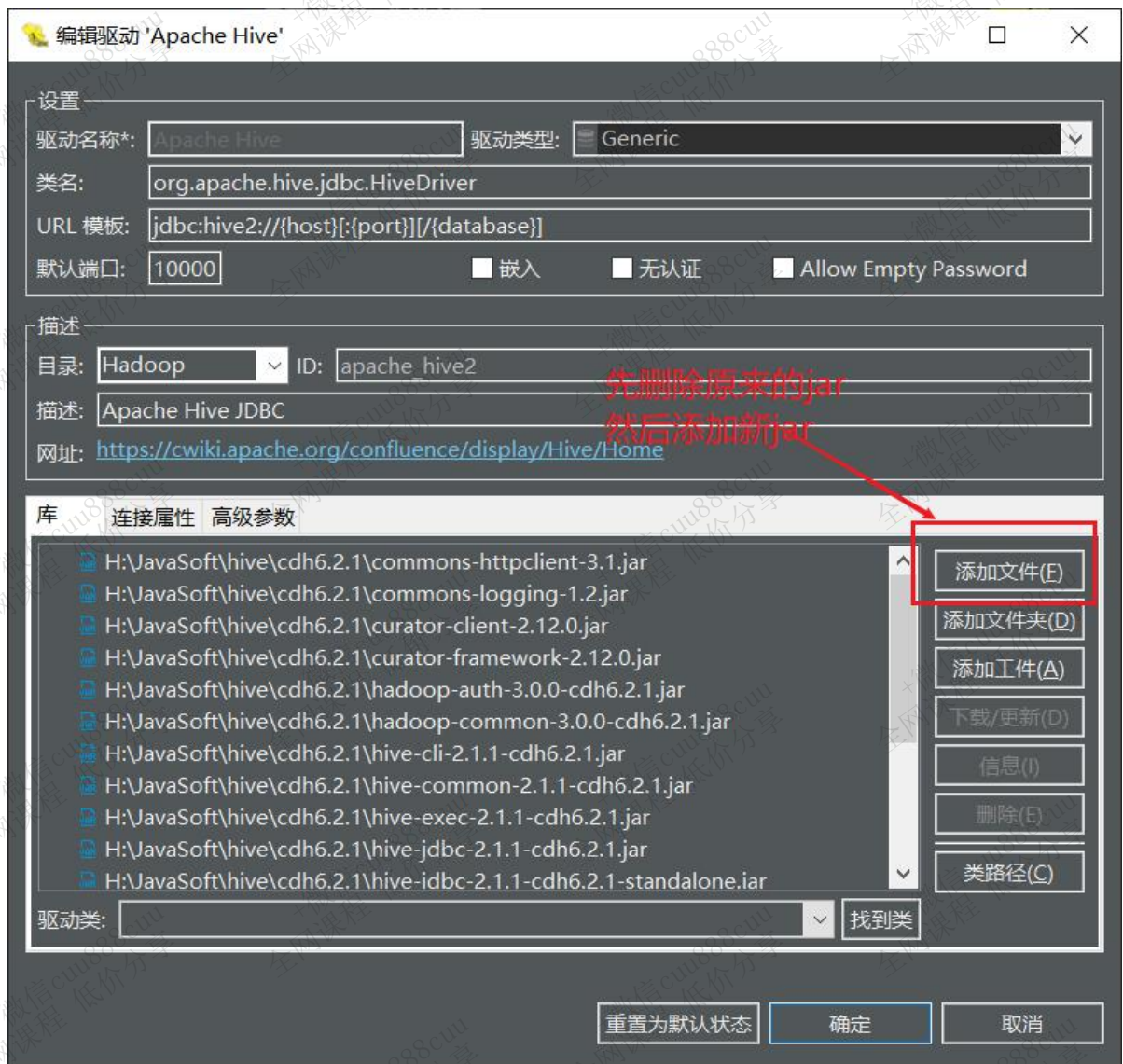
删除(E)

类路径(C)

驱动类: 找到类

重置为默认状态 确定 取消





选择【\亿品新零售\资料\驱动包\hive】下的所有jar包，添加：



编辑驱动 'Apache Hive'

设置

驱动名称*: 驱动类型:

类名:

URL 模板:

默认端口: ☐ 嵌入 ☐ 无认证 ☐ Allow Empty Password

描述

目录: ID:

描述:

网址: <https://cwiki.apache.org/confluence/display/Hive/Home>

库 连接属性 高级参数

☐ H:\JavaSoft\hive\cdh6.2.1\commons-httpclient-3.1.jar
☐ H:\JavaSoft\hive\cdh6.2.1\commons-logging-1.2.jar
☐ H:\JavaSoft\hive\cdh6.2.1\curator-client-2.12.0.jar
☐ H:\JavaSoft\hive\cdh6.2.1\curator-framework-2.12.0.jar
☐ H:\JavaSoft\hive\cdh6.2.1\hadoop-auth-3.0.0-cdh6.2.1.jar
☐ H:\JavaSoft\hive\cdh6.2.1\hadoop-common-3.0.0-cdh6.2.1.jar
☐ H:\JavaSoft\hive\cdh6.2.1\hive-cli-2.1.1-cdh6.2.1.jar
☐ H:\JavaSoft\hive\cdh6.2.1\hive-common-2.1.1-cdh6.2.1.jar
☐ H:\JavaSoft\hive\cdh6.2.1\hive-exec-2.1.1-cdh6.2.1.jar
☐ H:\JavaSoft\hive\cdh6.2.1\hive-jdbc-2.1.1-cdh6.2.1.jar
☐ H:\JavaSoft\hive\cdh6.2.1\hive-idbc-2.1.1-cdh6.2.1-standalone.jar

添加文件(E)
添加文件夹(D)
添加工件(A)
下载/更新(D)
信息(I)
删除(E)
类路径(C)

驱动类: 找到类

重置为默认状态 确定 取消



连接设置

数据库连接设置.

连接设置

初始化

Shell 命令

客户端认证

常规

元数据

错误处理

结果集

编辑器

数据格式化

展示

SQL 编辑器

SQL 处理

常规

驱动属性

SSH

Proxy

JDBC URL: jdbc:hive2://192.168.88.80:10000

主机: 192.168.88.80 端口: 10000

数据库/模式:

用户名:

密码:

☒ Save password locally

驱动名称: Hadoop / Apache Hive

编辑驱动设置

测试链接(T)...

确定

取消

3. 命名规范

3.1 库命名规范

业务简拼_dwd，亿品新零售业务的dwd层，可以命名为yp_dwd。

3.2 表命名规范

dwd层将数据划分为事实表和维度表。

事实表的命名格式为：yp_dwd.fact_xxx，比如订单表t_shop_order，可以命名为yp_dwd.fact_shop_order；



维度表的命名格式为：yp_dwd.dim_xxx，比如区域表t_district，可以命名为yp_dwd.dim_district。

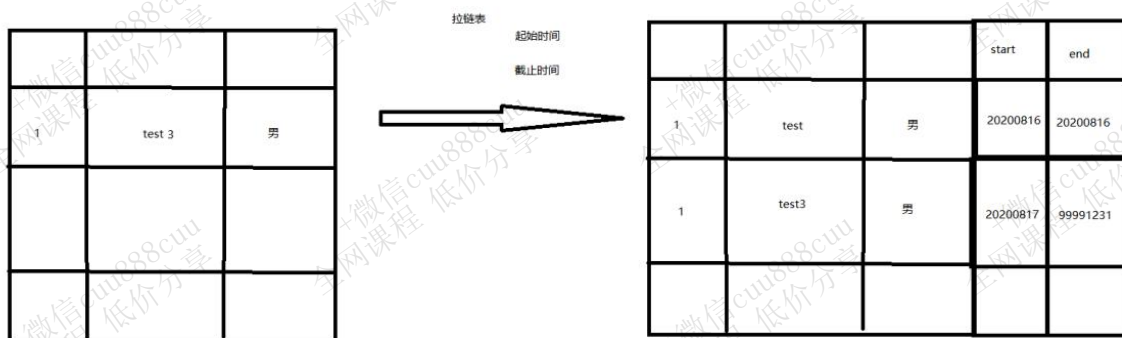
4. 拉链表的应用

4.1 应用场景

当数据存在有变更的情况时，可以采用渐变维（SCD）模型来解决。这里推荐使用SCD2拉链表。拉链表既可以反应数据的历史状态，又能在最大程度上节省存储。

拉链表的实现需要在原始字段基础上增加两个新字段：

- start_date(表示该条记录的生命周期开始时间——周期快照时的状态)
- end_date(该条记录的生命周期结束时间)



ODS层的数据同步方式用到了【全量覆盖】、【仅新增同步】、【新增及更新同步】，其中【新增及更新同步】的方式涉及到了更新数据，为了便于使用，在DWD层采用SCD2拉链表存储。

4.2 拉链表插入数据

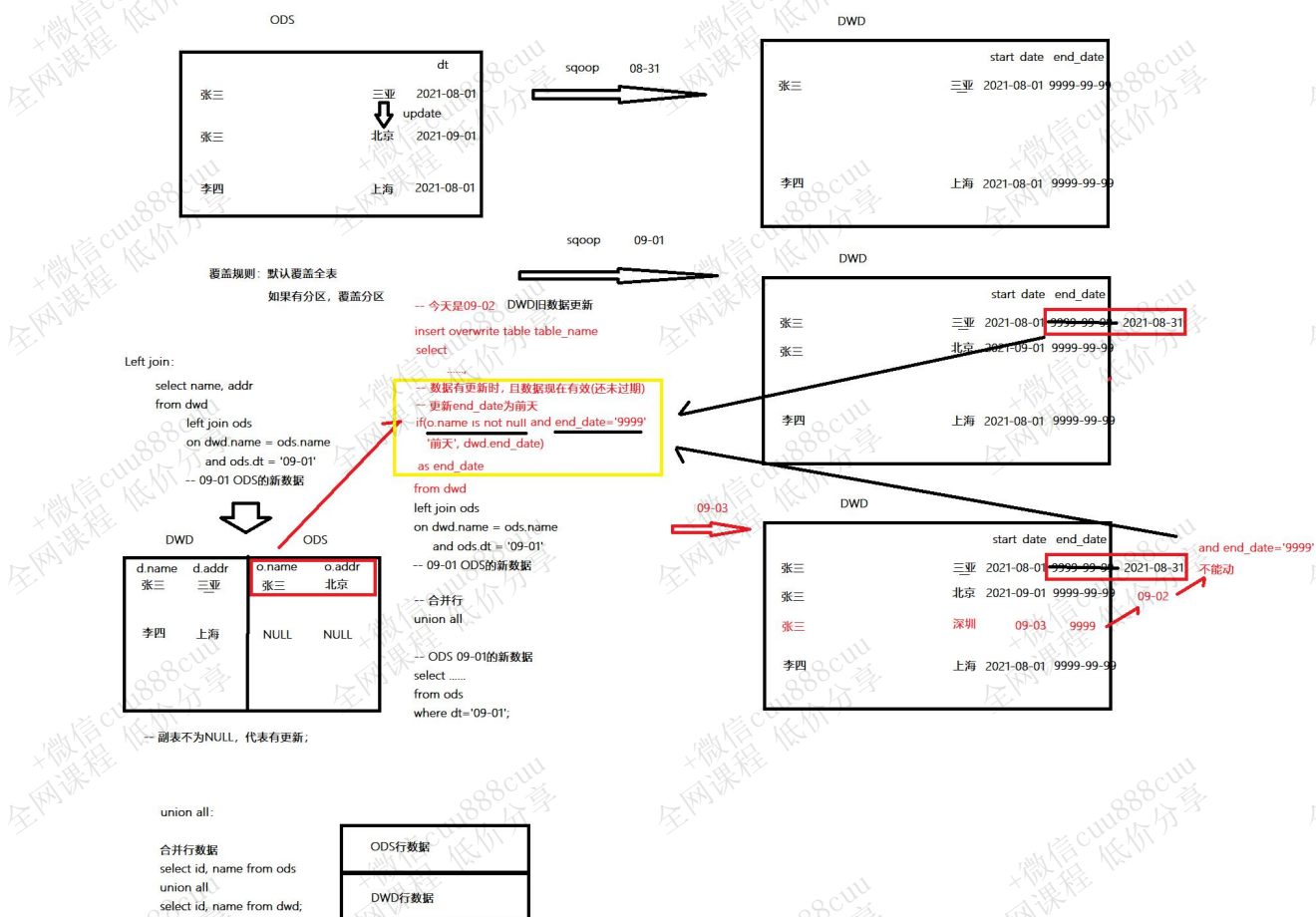
4.2.1 首次

全量抽取数据；

4.2.2 循环



1. ODS抽取新增/变更数据;
2. 重建临时表tmp;
3. 合并昨日增量数据 (ods表) 与历史数据 (拉链表) 到临时表
 - (1) 新数据(员工)end_date设为'9999-99-99', 也就是当前有效;
 - (2) 如果增量数据有重复id的旧数据(未离职老员工), 将旧数据end_date(退休日期)更新为前天 (昨日-1), 也就是从昨天开始不再生效;
 - (3) 合并后的数据写入tmp表;
4. 将临时表的数据, 覆盖到拉链表中; W
5. 第二天再次循环。





4.3 查询拉链表数据

查询拉链表数据时，可以通过start_date和end_date查询出快照数据。

查询当前正在生效的数据：

```
select * from table where end_date='9999-99-99';
```

查询出2021-08-08日生效的数据：

```
select * from table where '2021-08-08' between start_date and end_date;
```

5. 事实表和维度表

5.1 知识点回顾

回顾事实表与维度表：

- 1、数据特性：事实表一般都是行为数据，数据量较大，更新较频繁；维度表相对较小，不更新或更新频率低；
- 2、图表展现：在图表展现中，事实数据体现为x轴，维度数据体现为y轴；
- 3、统计实现：在统计sql中，维度字段体现在groupby分组中，行为指标字段体现在count/sum等聚合函数中。

5.2 事实表

5.2.1 订单事实表（拉链表）

5.2.1.1 建表

```
create database yp_dwd;

DROP TABLE if EXISTS yp_dwd.fact_shop_order;
CREATE TABLE yp_dwd.fact_shop_order (
  id string COMMENT '根据一定规则生成的订单编号',
  order_num string COMMENT '订单序号',
  buyer_id string COMMENT '买家的userId',
  store_id string COMMENT '店铺id',
  order_from string COMMENT '此字段可以转换 1.安卓\; 2.ios\; 3.小程序H5 \; 4.PC',
  order_state int COMMENT '订单状态:1.已下单\; 2.已付款, 3. 已确认 \;4.配送\; 5.已完成\; 6.退款\;7.已
```





```
取消',
create_date string COMMENT '下单时间',
finnshed_time timestamp COMMENT '订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需要根据
订单完成时间,进行自动收货以及自动评价',
is_settlement tinyint COMMENT '是否结算\;0.待结算订单\; 1.已结算订单\;',
is_delete tinyint COMMENT '订单评价的状态:0.未删除\; 1.已删除\;(默认0)',
evaluation_state tinyint COMMENT '订单评价的状态:0.未评价\; 1.已评价\;(默认0)',
way string COMMENT '取货方式:SELF自提\;SHOP店铺负责配送',
is_stock_up int COMMENT '是否需要备货 0: 不需要 1: 需要 2:平台确认备货 3:已完成备货 4平台已经将货
物送至店铺 ',
create_user string,
create_time string,
update_user string,
update_time string,
is_valid tinyint COMMENT '是否有效 0: false\; 1: true\; 订单是否有效的标志',
end_date string COMMENT '拉链结束日期')
COMMENT '订单表'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.1.2 导入数据(全量/增量)

5.2.1.2.1 首次

```
-- 分区
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=150000;
--hive压缩
set hive.exec.compress.intermediate=true;
set hive.exec.compress.output=true;
--写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;

INSERT overwrite TABLE yp_dwd.fact_shop_order PARTITION (start_date)
SELECT
    id,
    order_num,
    buyer_id,
    store_id,
-- 转换
    case order_from
        when 1
        then 'android'
        when 2
        then 'ios'
        when 3
```



```
then 'miniapp'
when 4
then 'pcweb'
else 'other'
end
as order_from,
order_state,
create_date,
finnshed_time,
is_settlement,
is_delete,
evaluation_state,
way,
is_stock_up,
create_user,
create_time,
update_user,
update_time,
is_valid,
'9999-99-99' end_date,
SUBSTRING(create_time, 1, 10) as start_date
FROM yp_ods.t_shop_order
-- 过滤
where id is not null and buyer_id is not null and store_id is not null and create_date is not null;
```

5.2.1.2.2 循环

5.2.1.2.2.1 ODS抽取新增/变更数据

略

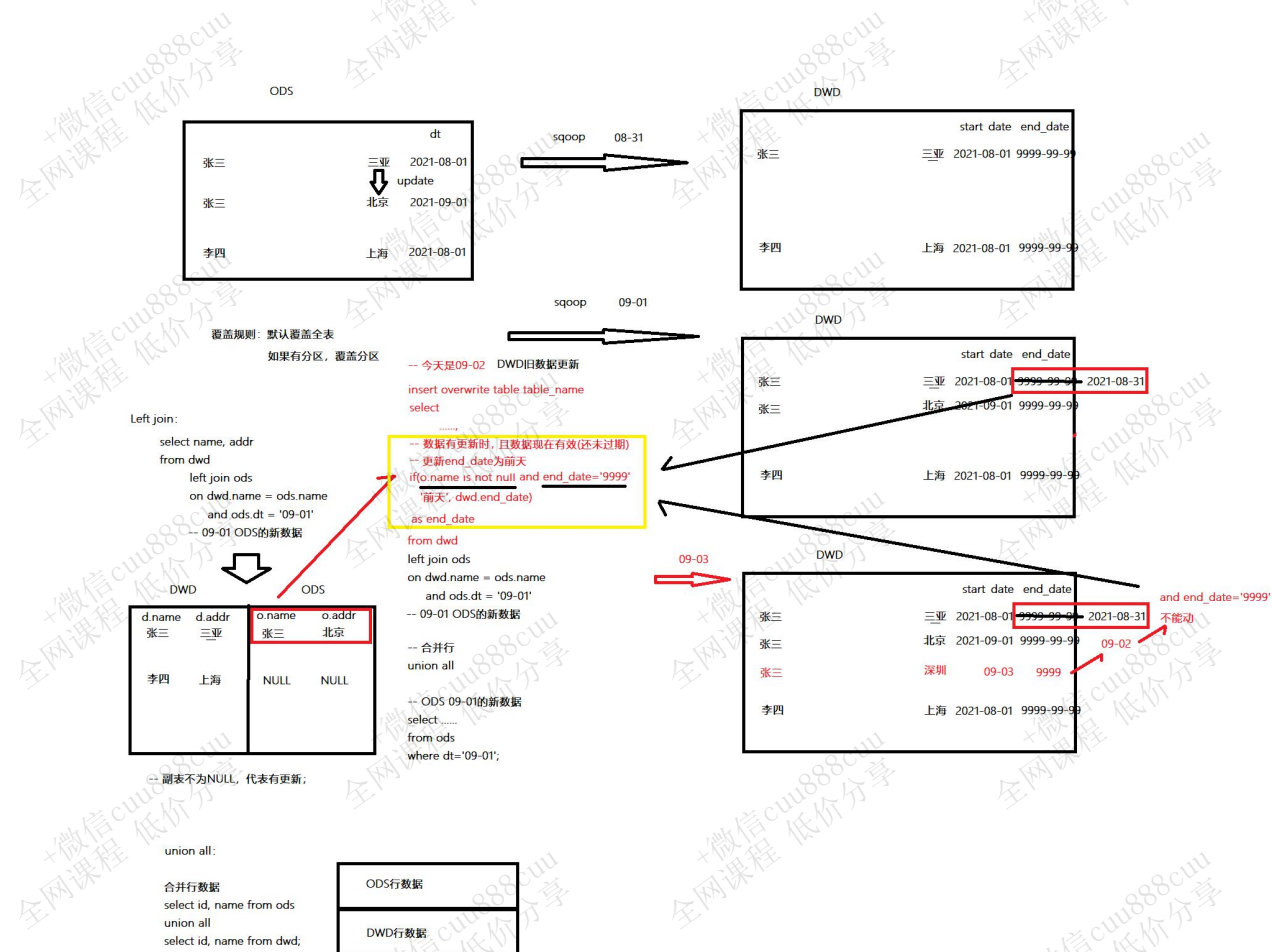
5.2.1.2.2.2 重建临时表

```
DROP TABLE if EXISTS yp_dwd.fact_shop_order_tmp;
CREATE TABLE yp_dwd.fact_shop_order_tmp(
    id string COMMENT '根据一定规则生成的订单编号',
    order_num string COMMENT '订单序号',
    buyer_id string COMMENT '买家的userId',
    store_id string COMMENT '店铺的id',
    order_from string COMMENT '此字段可以转换 1.安卓\; 2.ios\; 3.小程序H5 \; 4.PC',
    order_state int COMMENT '订单状态:1.已下单\; 2.已付款, 3. 已确认 \;4.配送\; 5.已完成\; 6.退款\;7.已取消',
    create_date string COMMENT '下单时间',
    finnshed_time timestamp COMMENT '订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需要根据订单完成时间,进行自动收货以及自动评价',
    is_settlement tinyint COMMENT '是否结算\;0.待结算订单\; 1.已结算订单\;',
    is_delete tinyint COMMENT '订单评价的状态:0.未删除\; 1.已删除\;(默认0)'
```




```
evaluation_state tinyint COMMENT '订单评价的状态:0.未评价\; 1.已评价\;(默认0)',
way string COMMENT '取货方式:SELF自提\;SHOP店铺负责配送',
is_stock_up int COMMENT '是否需要备货 0: 不需要    1: 需要    2:平台确认备货 3:已完成备货 4:平台已经将货物送至店铺 ',
create_user string,
create_time string,
update_user string,
update_time string,
is_valid tinyint COMMENT '是否有效 0: false\; 1: true\;    订单是否有效的标志',
end_date string COMMENT '拉链结束日期')
COMMENT '订单表'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.1.2.2.3 开始合并新旧数据到临时表



1. 获取ods表昨日的更新数据，新数据(新员工)end_date为'9999-99-99'，start_date为昨日日



期，转换字段(order_from)做同样处理；

2. 更新旧数据(老员工)end_date:

(1) 将历史表yp_dwd.fact_shop_order(拉链表)与新增/更新数据表yp_ods.t_shop_order通过id进行关联，如果ods中有与历史表重复的id(新老员工同一岗位)，证明此条id数据(老员工)已有新的变更(新员工)代替；

(2) end_date不变的条件:

1) 没有更新的数据(新员工)保留原始end_date；

2) 历史表已是失效的数据(老员工已退休)，保留原始有效结束日期end_date；

(3) 否则（有新员工代替，且老员工还没有退休），修改end_date为前天（办理退休）；

3. 将 1.ods 与 2.拉链表 合并，覆盖插入到临时表中。

```
--分区
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=150000;
--hive压缩
set hive.exec.compress.intermediate=true;
set hive.exec.compress.output=true;
--写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;

insert overwrite table yp_dwd.fact_shop_order_tmp partition (start_date)
select * from
(
-- 一、ods表的新数据
select
    id,
    order_num,
    buyer_id,
    store_id,
-- 转换
    case order_from
        when 1
            then 'android'
        when 2
            then 'ios'
        when 3
            then 'miniapp'
        when 4
            then 'pcweb'
        else 'other'
    end as order_from
    -- 这里应该还有其他的字段，但被水印遮挡了

```





```
end
as order_from,
order_state,
create_date,
finnshed_time,
is_settlement,
is_delete,
evaluation_state,
way,
is_stock_up,
create_user,
create_time,
update_user,
update_time,
is_valid,
'9999-99-99' end_date,
'${TD_DATE}' as start_date
from yp_ods.t_shop_order where dt='${TD_DATE}';
-- 过滤
and id is not null and buyer_id is not null and store_id is not null and create_date is
not null;
union all

-- 二、历史拉链表数据，并根据up_id判断更新end_time有效期

select
fso.id,
fso.order_num,
fso.buyer_id,
fso.store_id,
fso.order_from,
fso.order_state,
fso.create_date,
fso.finnshed_time,
fso.is_settlement,
fso.is_delete,
fso.evaluation_state,
fso.way,
fso.is_stock_up,
fso.create_user,
fso.create_time,
fso.update_user,
fso.update_time,
```



```
fso.is_valid,

--3、更新end_time: 如果没有匹配到变更数据，或者当前已经是无效的历史数据，则保留原始end_time过期时间；
否则变更end_time时间为前天（昨天之前有效）

if(up.id is null or fso.end_date<'9999-99-99', fso.end_date, date_add(up.dt,-1))
end_time,

fso.start_date

from yp_dwd.fact_shop_order fso left join

(
select
*
from yp_ods.t_shop_order
where dt='${TD_DATE}'
) up
on fso.id=up.id

--4、时间限制：如果订单的变更周期是30天则可加上此条件，结果会按照所属分区进行覆盖插入
--where fso.start_date >= date_add(up.dt,-30)

) his
order by his.id, start_date;
```

5.2.1.2.2.4 临时表覆盖拉链表

```
INSERT OVERWRITE TABLE yp_dwd.fact_shop_order partition (start_date)
SELECT * from yp_dwd.fact_shop_order_tmp;
```

5.2.1.2.2.5 查询表数据

```
SELECT * from yp_dwd.fact_shop_order
WHERE end_date = '9999-99-99';
```

5.2.2 分桶采样

分桶是将数据集分解成更容易管理的若干部分的一个技术，是比分区更为细粒度的数据范围划分。

在真实的大数据分析过程中，由于数据量较大，开发和自测的过程比较慢，严重影响系统的开发进度。此时就可以使用分桶来进行数据采样。采样使用的是一个具有代表性的查询结果而不是全部结果，通过对采样数据的分析，来达到快速开发和自测的目的，节省大量的研发成本。

5.2.2.1 分桶和分区的区别

1. 分桶对数据的处理比分区更加细粒度化：分区针对的是数据的存储路径；分桶针对的是数据文件；
2. 分桶是按照列的哈希函数进行分割的，相对比较平均；而分区是按照列的值来进行分割的，



容易造成数据倾斜；

3. 分桶和分区两者不干扰，可以把分区表进一步分桶。

5.2.2.2 操作

1. 创建分桶表

```
create table test_buck(id int, name string)
clustered by(id) sorted by (id asc) into 6 buckets
row format delimited fields terminated by '\t';
```

CLUSTERED BY来指定划分桶所用列；

SORTED BY对桶中的一个或多个列进行排序；

into 6 buckets指定划分桶的个数。

分桶规则：HIVE对key的hash值除bucket个数取余数，保证数据均匀随机分布在所有bucket里。

查看分桶表信息

itcast_ods_test

表 (3) +

筛选器...

test

web_chat_ems

web_chat_text_ems

Table Type: MANAGED_TABLE

Table Parameters:

COLUMN_STATS_ACCURATE

SORTBUCKETCOLSPREFIX

numFiles

numRows

rawDataSize

totalSize

transient_lastDdlTime

STORAGE INFORMATION

SerDe Library: org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe

InputFormat: org.apache.hadoop.mapred.TextInputFormat

OutputFormat: org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat

Compressed: No

Num Buckets: 10

Bucket Columns: [id]

Sort Columns: [Order(col:id, order:1)]

Storage Desc Params:

field.delim

serialization.format

```
desc formatted test_buck;
```



```
| Num Buckets: | 6  
|  
| Bucket Columns: | [id]
```

2. 插入数据

```
--启用桶表  
set hive.enforce.bucketing=true;  
insert into table test_buck select id, name from temp_buck;
```

hive.enforce.bucketing：启用桶表，数据分桶是否被强制执行，默认false，如果开启，则写入table数据时会启动分桶。

5.2.2.3 文本数据处理

注意：对于分桶表，**不能使用load data的方式进行数据插入操作**，因为load data导入的数据不会有分桶结构。

如何避免针对桶表使用load data插入数据的误操作呢？

```
--限制对桶表进行load操作  
set hive.strict.checks.bucketing = true;
```

也可以在CM的hive配置项中修改此配置，当针对桶表执行load data操作时会报错。



那么对于文本数据如何处理呢？

(1. 先创建**临时表**，通过load data将txt文本导入临时表。



--创建临时表

```
create table temp_buck(id int, name string)
```

row format delimited fields terminated by '\t';

--导入数据

```
load data local inpath '/tools/test_buck.txt' into table temp_buck;
```

(2. 使用insert select语句间接的把数据从临时表导入到分桶表。

--启用桶表

```
set hive.enforce.bucketing=true;
```

--限制对桶表进行load操作

```
set hive.strict.checks.bucketing = true;
```

--insert select

```
insert into table test_buck select id, name from temp_buck;
```

--分桶成功

Browse Directory

/user/hive/warehouse/test_buck

Go!

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rwxrwxr-x	root	supergroup	28 B	2020/3/30 下午5:54:48	3	128 MB	000000_0
-rwxrwxr-x	root	supergroup	28 B	2020/3/30 下午5:54:48	3	128 MB	000001_0
-rwxrwxr-x	root	supergroup	29 B	2020/3/30 下午5:54:48	3	128 MB	000002_0
-rwxrwxr-x	root	supergroup	19 B	2020/3/30 下午5:54:48	3	128 MB	000003_0
-rwxrwxr-x	root	supergroup	19 B	2020/3/30 下午5:54:48	3	128 MB	000004_0
-rwxrwxr-x	root	supergroup	28 B	2020/3/30 下午5:54:48	3	128 MB	000005_0

5.2.2.4 数据采样

对表分桶一般有两个目的，提高数据查询效率、抽样调查。通过前面的讲解，我们已经可以对分桶表进行正常的创建并导入数据了。一般在实际生产中，对于非常大的数据集，有时用户需要使用的是一个具有代表性的查询结果而不是全部结果，比如在开发自测的时候。这个时候Hive就可以通过对表进行抽样来满足这个需求。

语法

```
select * from table tablesample(bucket x out of y on column)
```

hive根据y的大小，决定抽样的比例。y必须是table总bucket数的倍数或者因子。





例如，table总共分了10份bucket，当 $y=2$ 时，抽取 $(10/2=)$ 5个bucket的数据，当 $y=10$ 时，抽取 $(10/10=)$ 1个bucket的数据。

x 表示从哪个bucket开始抽取，如果需要取多个分区，以后的分区号为当前分区号加上 y 。

例如，table总bucket数为6，`tablesample(bucket 1 out of 2)`，表示总共抽取 $(6/2=)$ 3个bucket的数据，从第1个bucket开始，抽取第1(x)个和第3($x+y$)个和第5($x+y$)个bucket的数据。

注意： x 的值必须小于等于 y 的值。否则会抛出异常：FAILED: SemanticException [Error 10061]: Numerator should not be bigger than denominator in sample clause for table stu_bucket.

栗子

```
select * from test_bucket tablesample(bucket 1 out of 10 on id);
```

注意：sqoop不支持分桶表，如果需要从sqoop导入数据到分桶表，可以通过中间临时表进行过度。ODS也可以不做分桶，从DWD明细层开始分桶。

5.2.3 Hive执行计划

5.2.3.1 作用

用户提交HiveQL查询后，Hive会把查询语句转换为MapReduce作业。Hive会自动完成整个执行过程，一般情况下，我们并不知道内部是如何运行的。

执行计划可以告诉我们查询过程的关键信息，用来帮助我们判定优化措施是否已经生效。

5.2.3.2 基础语法

EXPLAIN的使用非常简单，只需要在正常HiveQL前面加上EXPLAIN就可以了。执行计划运行时的HiveQL不会真正执行作业，只是基于优化器生成了最优的执行路径：

```
EXPLAIN [EXTENDED] query
```

extended输出更加详细的信息；



5.2.3.3 执行计划分为两部分

1. stage依赖(STAGE DEPENDENCIES)

- (1) 这部分展示本次查询分为两个stage：Stage-1, Stage-0.
- (2) 一般Stage-0是最终给查询用户展示数据用的，如LIMITE操作就会在这部分。
- (3) Stage-1是mr程序的执行阶段。

```
1 STAGE DEPENDENCIES:  
2 Stage-1 is a root stage  
3 Stage-0 depends on stages: Stage-1
```

2. stage详细执行计划(STAGE PLANS)

- (1) 包含了整个查询所有Stage的大部分处理过程。
- (2) 特定优化是否生效，主要通过此部分内容查看。

3. 名次解释

TableScan:查看表

alias: emp: 所需要的表

Statistics: Num rows: 2 Data size: 820 Basic stats: COMPLETE Column stats: NONE: 这张表的基本统计信息：行数、大小等；

expressions: empno (type: int), ename (type: string), job (type: string), mgr (type: int), hiredate (type: string), sal (type: double), comm (type: double), deptno (type: int): 表中需要输出的字段及类型

outputColumnNames: _col0, _col1, _col2, _col3, _col4, _col5, _col6, _col7: 输出的的字段编号

compressed: true: 输出是否压缩；

input format: org.apache.hadoop.mapred.SequenceFileInputFormat: 文件输入调用的Java类，显示以文本Text格式输入；

output format: org.apache.hadoop.hive ql.io.HiveSequenceFileOutputFormat: 文件输出调用的java类，显示以文本Text格式输出；



5.2.3.4 样例

DWD阶段执行计划：

```

1  STAGE DEPENDENCIES:
2    Stage-1 is a root stage
3    Stage-0 depends on stages: Stage-1
4
5  STAGE PLANS:
6    Stage: Stage-1
7      Map Reduce
8      Map Operator Tree:
9        TableScan
10         alias: rs
11         Statistics: Num rows: 1109147 Data size: 236547154 Basic stats: COMPLETE Column stats: COMPLETE
12         Filter Operator
13           predicate: (((hash(id) & 2147483647) % 10) = 0) (type: boolean)
14           Statistics: Num rows: 554573 Data size: 118273474 Basic stats: COMPLETE Column stats: COMPLETE
15           Select Operator
16             expressions: id (type: int), customer_id (type: int), create_date_time (type: string), if((itcast_school_id
17 is null or (itcast_school_id = 0)), -1, itcast_school_id) (type: int), deleted (type: int), origin_type (type: string),
18 if((itcast_subject_id is null or (itcast_subject_id = 0)), -1, itcast_subject_id) (type: int), substr(create_date_time, 12,
19 2) (type: string), if((origin_type = 'NETSERVICE'), '1', if((origin_type = 'PRESIGNUP'), '1', '0')) (type: string),
20 substr(create_date_time, 1, 4) (type: string), substr(create_date_time, 6, 2) (type: string), substr(create_date_time, 9,
21 2) (type: string)
22             outputColumnNames: _col0, _col1, _col2, _col3, _col4, _col5, _col6, _col7, _col8, _col9, _col10, _col11
23             Statistics: Num rows: 554573 Data size: 631104074 Basic stats: COMPLETE Column stats: COMPLETE
24             File Output Operator
25               compressed: false
26               Statistics: Num rows: 554573 Data size: 631104074 Basic stats: COMPLETE Column stats: COMPLETE
27               table:
28                 input format: org.apache.hadoop.mapred.SequenceFileInputFormat
29                 output format: org.apache.hadoop.hive ql.io.HiveSequenceFileOutputFormat
30                 serde: org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe
31
32    Stage: Stage-0
33      Fetch Operator
34        limit: -1
35      Processor Tree:
36        ListSink

```

5.2.4 订单事实表-分桶重建

5.2.4.1 改为分桶表

```

DROP TABLE if exists yp_ods.t_shop_order;
CREATE TABLE yp_ods.t_shop_order
(
  id                string COMMENT '根据一定规则生成的订单编号',
  order_num         string COMMENT '订单序号',
  buyer_id         string COMMENT '买家的userId',
  store_id          string COMMENT '店铺的id',
  order_from        TINYINT COMMENT '是来自于app还是小程序,或者pc 1.安卓; 2.ios; 3.小程序H5; 4.PC',
  order_state       INT COMMENT '订单状态:1.已下单; 2.已付款; 3. 已确认 ;4.配送; 5.已完成; 6.退款;7.已取消',
  create_date       string COMMENT '下单时间',

```



```
finnshed_time    timestamp COMMENT '订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需要
根据订单完成时间,进行自动收货以及自动评价',

is_settlement     TINYINT COMMENT '是否结算;0.待结算订单; 1.已结算订单;',

is_delete        TINYINT COMMENT '订单评价的状态:0.未删除; 1.已删除;(默认0)',

evaluation_state  TINYINT COMMENT '订单评价的状态:0.未评价; 1.已评价;(默认0)',

way              string COMMENT '取货方式:SELF自提;SHOP店铺负责配送',

is_stock_up      INT COMMENT '是否需要备货 0: 不需要    1: 需要    2:平台确认备货 3:已完成备货 4平台
已经将货物送至店铺 ',

create_user      string,

create_time      string,

update_user      string,

update_time      string,

is_valid         TINYINT COMMENT '是否有效 0: false; 1: true;  订单是否有效的标志'

)

comment '订单表'

partitioned by (dt string)

clustered by(id) sorted by(id) into 10 buckets

row format delimited fields terminated by '\t'

stored as orc tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.4.2 通过临时表采集数据

创建临时表(不要分桶):

```
DROP TABLE if exists yp_ods.t_shop_order_tmp;

CREATE TABLE yp_ods.t_shop_order_tmp (

`id` string COMMENT '根据一定规则生成的订单编号',

`order_num` string COMMENT '订单序号',

`buyer_id` string COMMENT '买家的userId',

`store_id` string COMMENT '店铺id',

`order_from` TINYINT COMMENT '是来自于app还是小程序,或者pc 1.安卓; 2.ios; 3.小程序H5 ; 4.PC',

`order_state` INT COMMENT '订单状态:1.已下单; 2.已付款, 3. 已确认 ;4.配送; 5.已完成; 6.退款;7. 已取消',

`create_date` string COMMENT '下单时间',

`finnshed_time` timestamp COMMENT '订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需 要根据
订单完成时间,进行自动收货以及自动评价',

`is_settlement` TINYINT COMMENT '是否结算;0.待结算订单; 1.已结算订单;',

`is_delete` TINYINT COMMENT '订单评价的状态:0.未删除; 1.已删除;(默认0)',
```





```
`evaluation_state` TINYINT COMMENT '订单评价的状态:0.未评价; 1.已评价;(默认0)',
`way` string COMMENT '取货方式:SELF自提;SHOP店铺负责配送',
`is_stock_up` INT COMMENT '是否需要备货 0: 不需要 1: 需要 2:平台确认备货 3:已完成备货 4平 台已经将货物送至
店铺 ',
`create_user` string,
`create_time` string,
`update_user` string,
`update_time` string,
`is_valid` TINYINT COMMENT '是否有效 0: false; 1: true; 订单是否有效的标志' )
comment '订单表' partitioned by (dt string)
row format delimited fields terminated by '\t' stored as orc tblproperties ('orc.compress' = 'ZLIB');
```

sqoop采集:

```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '2021-10-22' as dt from t_shop_order where 1=1 and \$CONDITIONS" \
--hcatalog-database yp_ods \
--hcatalog-table t_shop_order_tmp \
-m 1
```

覆盖插入:

```
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
insert overwrite table yp_ods.t_shop_order partition (dt)
select * from yp_ods.t_shop_order_tmp;
```

5.2.4.3 分桶采样

改为分桶表后，分别对比采样前后的执行计划：

```
--分区
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
```





```
set hive.exec.max.created.files=150000;
--hive压缩
set hive.exec.compress.intermediate=true;
set hive.exec.compress.output=true;
--写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;
--分桶
set hive.enforce.bucketing=true;
set hive.enforce.sorting=true;

-- INSERT overwrite TABLE yp_dwd.fact_shop_order PARTITION (start_date)
-- 采样前
EXPLAIN
SELECT
    id,
    order_num,
    buyer_id,
    store_id,
    case order_from
        when 1
            then 'android'
        when 2
            then 'ios'
        when 3
            then 'miniapp'
        when 4
            then 'pcweb'
        else 'other'
    end
    as order_from,
    order_state,
    create_date,
    finnshed_time,
    is_settlement,
    is_delete,
    evaluation_state,
    way,
    is_stock_up,
    create_user,
    create_time,
    update_user,
    update_time,
    is_valid,
    '9999-99-99' end_date,
    SUBSTRING(create_time, 1, 10) as start_date
FROM yp_ods.t_shop_order
where id is not null and buyer_id is not null and store_id is not null and create_date is not null;

-- 采样后
-- INSERT overwrite TABLE yp_dwd.fact_shop_order PARTITION (start_date)
EXPLAIN
SELECT
    id,
    order_num,
```





```
buyer_id,  
store_id,  
case order_from  
  when 1  
  then 'android'  
  when 2  
  then 'ios'  
  when 3  
  then 'miniapp'  
  when 4  
  then 'pcweb'  
  else 'other'  
  end  
  as order_from,  
order_state,  
create_date,  
finnshed_time,  
is_settlement,  
is_delete,  
evaluation_state,  
way,  
is_stock_up,  
create_user,  
create_time,  
update_user,  
update_time,  
is_valid,  
'9999-99-99' end_date,  
SUBSTRING(create_time, 1, 10) as start_date  
FROM yp_ods.t_shop_order TABLESAMPLE (BUCKET 1 OUT OF 10 ON id)  
where id is not null and buyer_id is not null and store_id is not null and create_date is not null;
```



5.2.4.4 对比执行计划

1 Explain	1 Explain
2 STAGE DEPENDENCIES:	2 STAGE DEPENDENCIES:
3 Stage-1 is a root stage	3 Stage-1 is a root stage
4 Stage-0 depends on stages: Stage-1	4 Stage-0 depends on stages: Stage-1
5	5
6 STAGE PLANS:	6 STAGE PLANS:
7 Stage: Stage-1	7 Stage: Stage-1
8 Map Reduce	8 Map Reduce
9 Map Operator Tree:	9 Map Operator Tree:
10 TableScan	10 TableScan
11 alias: t_shop_order	11 alias: t_shop_order
12 Statistics: Num rows: 126 Data size: 158384 Basic stats: COMPLETE Column stats: PAR	12 Statistics: Num rows: 126 Data size: 158384 Basic stats: COMPLETE Column stats: PAR
13	13
14 Select Operator	14 Select Operator
15 expressions: id (type: string), order_num (type: string), buyer_id (type: string)	15 expressions: id (type: string), order_num (type: string), buyer_id (type: string)
16 outputColumnNames: col0, col1, col2, col3, col4, col5, col6, col7, col8	16 outputColumnNames: col0, col1, col2, col3, col4, col5, col6, col7, col
17 Statistics: Num rows: 126 Data size: 58212 Basic stats: COMPLETE Column stats: PA	17 Statistics: Num rows: 63 Data size: 11592 Basic stats: COMPLETE Column stats: PAR
18 File Output Operator	18 File Output Operator
19 compressed: false	19 compressed: false
20 Statistics: Num rows: 126 Data size: 58212 Basic stats: COMPLETE Column stats: I	20 Statistics: Num rows: 63 Data size: 29106 Basic stats: COMPLETE Column stats: P
21 table:	21 table:
22 input format: org.apache.hadoop.mapred.SequenceFileInputFormat	22 input format: org.apache.hadoop.mapred.SequenceFileInputFormat
23 output format: org.apache.hadoop.hive.q1.io.HiveSequenceFileOutputFormat	23 output format: org.apache.hadoop.hive.q1.io.HiveSequenceFileOutputFormat
24 serde: org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe	24 serde: org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe
25 Execution mode: vectorized	25
26	26
27 Stage: Stage-0	27 Stage: Stage-0
28 Fetch Operator	28 Fetch Operator
29 limit: -1	29 limit: -1
30 Processor Tree:	30 Processor Tree:
31 ListSink	31 ListSink
32	32
33	33

5.2.4.5 导入数据

5.2.4.5.1 首次

在数据量较大时，可以通过采样的方式提高开发和测试效率。数据量较小时不会有明显提升。

```
--分区
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=150000;
--hive压缩
set hive.exec.compress.intermediate=true;
set hive.exec.compress.output=true;
--写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;
--分桶
set hive.enforce.bucketing=true;
set hive.enforce.sorting=true;

INSERT overwrite TABLE yp_dwd.fact_shop_order PARTITION (start_date)
SELECT
  id,
  order_num,
  buyer_id,
  store_id,
  case order_from
    when 1
    then 'android'
    when 2
    then 'ios'
    when 3
    then 'miniapp'
```





```
when 4
then 'pcweb'
else 'other'
end
as order_from,
order_state,
create_date,
finnshed_time,
is_settlement,
is_delete,
evaluation_state,
way,
is_stock_up,
create_user,
create_time,
update_user,
update_time,
is_valid,
'9999-99-99' end_date,
SUBSTRING(create_time, 1, 10) as start_date
FROM yp_ods.t_shop_order
where id is not null and buyer_id is not null and store_id is not null and create_date is not null;
```

5.2.4.5.2 循环

略。

5.2.5 订单详情表（拉链表）

5.2.5.1 建表

```
DROP TABLE if EXISTS yp_dwd.fact_shop_order_address_detail;
CREATE TABLE yp_dwd.fact_shop_order_address_detail (
  id string COMMENT '关联订单的id',
  order_amount decimal(36,2) COMMENT '订单总金额:购买总金额-优惠金额',
  discount_amount decimal(36,2) COMMENT '优惠金额',
  goods_amount decimal(36,2) COMMENT '用户购买的商品的总金额+运费',
  is_delivery string COMMENT '0.自提; 1.配送',
  buyer_notes string COMMENT '买家备注留言',
  pay_time string,
  receive_time string,
  delivery_begin_time string,
  arrive_store_time string,
  arrive_time string COMMENT '订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需要根据订单完成时间,进行自动收货以及自动评价',
  create_user string,
  create_time string,
  update_user string,
  update_time string,
  is_valid tinyint COMMENT '是否有效 0: false\; 1: true\; 订单是否有效的标志',
```




```
end_date string COMMENT '拉链结束日期')
COMMENT '订单详情表'
partitioned by (start_date string)
clustered by(id) sorted by(id) into 10 buckets
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.5.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.fact_shop_order_address_detail PARTITION (start_date)
SELECT
    id,
    order_amount,
    discount_amount,
    goods_amount,
    is_delivery,
    buyer_notes,
    pay_time,
    receive_time,
    delivery_begin_time,
    arrive_store_time,
    arrive_time,
    create_user,
    create_time,
    update_user,
    update_time,
    is_valid,
    '9999-99-99' end_date,
    SUBSTRING(create_time, 1, 10) as start_date
FROM yp_ods.t_shop_order_address_detail;
```

5.2.6 订单结算表（拉链表）

5.2.6.1 建表

```
DROP TABLE if exists yp_dwd.fact_order_settle;
CREATE TABLE yp_dwd.fact_order_settle(
    id string COMMENT '结算单号',
    order_id string,
    settlement_create_date string COMMENT '用户申请结算的时间',
    settlement_amount decimal(36,2) COMMENT '如果发生退款,则结算的金额 = 订单的总金额 - 退款的金额',
    dispatcher_user_id string COMMENT '配送员id',
    dispatcher_money decimal(36,2) COMMENT '配送员的配送费(配送员的运费(如果退货方式为1:则买家支付配送费))',
    circle_master_user_id string COMMENT '圈主id',
    circle_master_money decimal(36,2) COMMENT '圈主分润的金额',
    plat_fee decimal(36,2) COMMENT '平台应得的分润',
    store_money decimal(36,2) COMMENT '商家应得的订单金额',
```





```
status tinyint COMMENT '0.待结算; 1.待审核 \; 2.完成结算; 3.拒绝结算',
note string COMMENT '原因',
settle_time string COMMENT ' 结算时间',
create_user string,
create_time string,
update_user string,
update_time string,
is_valid tinyint COMMENT '是否有效 0: false\; 1: true\;  订单是否有效的标志',
first_commission_user_id string COMMENT '一级分佣用户',
first_commission_money decimal(36,2) COMMENT '一级分佣金额',
second_commission_user_id string COMMENT '二级分佣用户',
second_commission_money decimal(36,2) COMMENT '二级分佣金额',
end_date string COMMENT '拉链结束日期')
COMMENT '订单结算表'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.6.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.fact_order_settle PARTITION (start_date)
SELECT
  id
  ,order_id
  ,settlement_create_date
  ,settlement_amount
  ,dispatcher_user_id
  ,dispatcher_money
  ,circle_master_user_id
  ,circle_master_money
  ,plat_fee
  ,store_money
  ,status
  ,note
  ,settle_time
  ,create_user
  ,create_time
  ,update_user
  ,update_time
  ,is_valid
  ,first_commission_user_id
  ,first_commission_money
  ,second_commission_user_id
  ,second_commission_money
  ,'9999-99-99' end_date,
  SUBSTRING(create_time, 1, 10) as start_date
FROM yp_ods.t_order_settle;
```



5.2.7 退款订单表（拉链表）

5.2.7.1 建表

```
DROP TABLE if exists yp_dwd.fact_refund_order;
CREATE TABLE yp_dwd.fact_refund_order
(
    id                string COMMENT '退款单号',
    order_id          string COMMENT '订单的id',
    apply_date        string COMMENT '用户申请退款的时间',
    modify_date       string COMMENT '退款订单更新时间',
    refund_reason     string COMMENT '买家退款原因',
    refund_amount     DECIMAL(11,2) COMMENT '订单退款的金额',
    refund_state      TINYINT COMMENT '1.申请退款;2.拒绝退款; 3.同意退款,配送员配送; 4:商家同意退款,
    用户亲自送货;5.退款完成',
    refuse_refund_reason string COMMENT '商家拒绝退款原因',
    refund_goods_type  string COMMENT '1.上门取货(买家承担运费); 2.买家送达;',
    refund_shipping_fee DECIMAL(11,2) COMMENT '配送员的运费(如果退货方式为1:则买家支付配送费)',
    create_user        string,
    create_time        string,
    update_user        string,
    update_time        string,
    is_valid           TINYINT COMMENT '是否有效 0: false;1: true;  订单是否有效的标志',
    end_date           string COMMENT '拉链结束日期'
)
comment '退款订单表'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.7.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.fact_refund_order PARTITION (start_date)
SELECT
    id
    ,order_id
    ,apply_date
    ,modify_date
    ,refund_reason
    ,refund_amount
    ,refund_state
    ,refuse_refund_reason
    ,refund_goods_type
    ,refund_shipping_fee
    ,create_user
    ,create_time
    ,update_user
    ,update_time
    ,is_valid
```





```
, '9999-99-99' end_date  
, SUBSTRING(create_time, 1, 10) as start_date  
FROM yp_ods.t_refund_order;
```

5.2.8 订单组表（拉链表）

5.2.8.1 建表

```
DROP TABLE if EXISTS yp_dwd.fact_shop_order_group;  
CREATE TABLE yp_dwd.fact_shop_order_group(  
    id string,  
    order_id string COMMENT '订单id',  
    group_id string COMMENT '订单分组id',  
    is_pay tinyint COMMENT '是否已支付, 0未支付, 1已支付',  
    create_user string,  
    create_time string,  
    update_user string,  
    update_time string,  
    is_valid tinyint,  
    end_date string COMMENT '拉链结束日期')  
COMMENT '订单组'  
partitioned by (start_date string)  
row format delimited fields terminated by '\t'  
stored as orc  
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.8.2 导入数据

```
--全量  
INSERT overwrite TABLE yp_dwd.fact_shop_order_group PARTITION (start_date)  
SELECT  
    id,  
    order_id,  
    group_id,  
    is_pay,  
    create_user,  
    create_time,  
    update_user,  
    update_time,  
    is_valid,  
    '9999-99-99' end_date,  
    SUBSTRING(create_time, 1, 10) as start_date  
FROM yp_ods.t_shop_order_group;
```





5.2.9 订单商品快照(拉链表)

5.2.9.1 建表

```
DROP TABLE if EXISTS yp_dwd.fact_shop_order_goods_details;
CREATE TABLE yp_dwd.fact_shop_order_goods_details(
  id string COMMENT 'id主键',
  order_id string COMMENT '对应订单表的id',
  shop_store_id string COMMENT '卖家店铺ID',
  buyer_id string COMMENT '购买用户ID',
  goods_id string COMMENT '购买商品的id',
  buy_num int COMMENT '购买商品的数量',
  goods_price decimal(36,2) COMMENT '购买商品的价格',
  total_price decimal(36,2) COMMENT '购买商品的价格 = 商品的数量 * 商品的单价',
  goods_name string COMMENT '商品的名称',
  goods_image string COMMENT '商品的图片',
  goods_specification string COMMENT '商品规格',
  goods_weight int,
  goods_unit string COMMENT '商品计量单位',
  goods_type string COMMENT '商品分类      ytgj:进口商品      ytsc:普通商品      hots:爆品',
  refund_order_id string COMMENT '退款订单的id',
  goods_brokerage decimal(36,2) COMMENT '商家设置的商品分润的金额',
  is_refund tinyint COMMENT '0.不退款\; 1.退款',
  create_user string,
  create_time string,
  update_user string,
  update_time string,
  is_valid tinyint COMMENT '是否有效 0: false\; 1: true',
  end_date string COMMENT '拉链结束日期')
COMMENT '订单商品快照'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.9.2 导入数据

--全量

```
INSERT overwrite TABLE yp_dwd.fact_shop_order_goods_details PARTITION (start_date)
SELECT
  id,
  order_id,
  shop_store_id,
  buyer_id,
  goods_id,
  buy_num,
```





```
goods_price,  
total_price,  
goods_name,  
goods_image,  
goods_specification,  
goods_weight,  
goods_unit,  
goods_type,  
refund_order_id,  
goods_brokerage,  
is_refund,  
create_user,  
create_time,  
update_user,  
update_time,  
is_valid,  
'9999-99-99' end_date,  
SUBSTRING(create_time, 1, 10) as start_date
```

FROM

yp_ods.t_shop_order_goods_details;

--2.重建临时表

--3.开始合并新旧数据到临时表

--4.临时表覆盖拉链表

--5.查询表数据

5.2.10 购物车（拉链表）

5.2.10.1 建表

```
DROP TABLE if exists yp_dwd.fact_shop_cart;  
CREATE TABLE yp_dwd.fact_shop_cart  
(  
    id string COMMENT '主键id',  
    shop_store_id string COMMENT '卖家店铺ID',  
    buyer_id string COMMENT '购买用户ID',  
    goods_id string COMMENT '购买商品的id',  
    buy_num INT COMMENT '购买商品的数量',  
    create_user string,  
    create_time string,  
    update_user string,  
    update_time string,  
    is_valid TINYINT COMMENT '是否有效 0: false; 1: true',  
    end_date string COMMENT '拉链结束日期')  
comment '购物车'
```





```
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.10.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.fact_shop_cart PARTITION (start_date)
SELECT
    id,
    shop_store_id,
    buyer_id,
    goods_id,
    buy_num,
    create_user,
    create_time,
    update_user,
    update_time,
    is_valid,
    '9999-99-99' end_date,
    SUBSTRING(create_time, 1, 10) as start_date
FROM
    yp_ods.t_shop_cart;
```

5.2.11 店铺收藏（拉链表）

5.2.11.1 建表

```
DROP TABLE if exists yp_dwd.fact_store_collect;
CREATE TABLE yp_dwd.fact_store_collect
(
    id string,
    user_id string COMMENT '收藏人id',
    store_id string COMMENT '店铺id',
    create_user string,
    create_time string,
    update_user string,
    update_time string,
    is_valid TINYINT COMMENT '0：失效，1：开启',
    end_date string COMMENT '拉链结束日期')
comment '收藏店铺记录表'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```





5.2.11.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.fact_store_collect PARTITION (start_date)
SELECT
    id,
    user_id,
    store_id,
    create_user,
    create_time,
    update_user,
    update_time,
    is_valid,
    '9999-99-99' end_date,
    SUBSTRING(create_time, 1, 10) as start_date
FROM yp_ods.t_store_collect;
```

5.2.12 商品收藏（拉链表）

5.2.12.1 建表

```
DROP TABLE if exists yp_dwd.fact_goods_collect;
CREATE TABLE yp_dwd.fact_goods_collect
(
    id string,
    user_id string COMMENT '收藏人id',
    goods_id string COMMENT '商品id',
    store_id string COMMENT '通过哪个店铺收藏的（因主店分店概念存在需要）',
    create_user string,
    create_time string,
    update_user string,
    update_time string,
    is_valid TINYINT COMMENT '0：失效，1：开启',
    end_date string COMMENT '拉链结束日期')
comment '收藏商品记录表'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.12.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.fact_goods_collect PARTITION (start_date)
SELECT
    id,
```




```
user_id,  
goods_id,  
store_id,  
create_user,  
create_time,  
update_user,  
update_time,  
is_valid,  
'9999-99-99' end_date,  
SUBSTRING(create_time, 1, 10) as start_date  
FROM yp_ods.t_goods_collect;
```

5.2.13 配送表（拉链表）

5.2.13.1 建表

```
DROP TABLE if EXISTS yp_dwd.fact_order_delievery_item;  
CREATE TABLE yp_dwd.fact_order_delievery_item(  
    id string COMMENT '主键id',  
    shop_order_id string COMMENT '订单表ID',  
    refund_order_id string,  
    dispatcher_order_type tinyint COMMENT '配送订单类型1.支付单\; 2.退款单',  
    shop_store_id string COMMENT '卖家店铺ID',  
    buyer_id string COMMENT '购买用户ID',  
    circle_master_user_id string COMMENT '圈主ID',  
    dispatcher_user_id string COMMENT '配送员ID',  
    dispatcher_order_state tinyint COMMENT '配送订单状态:0.待接单.1.已接单,2.已到店.3.配送中 4.商家普通  
提货码完成订单.5.商家万能提货码完成订单.6, 买家完成订单',  
    order_goods_num tinyint COMMENT '订单商品的个数',  
    delivery_fee decimal(36,2) COMMENT '配送员的运费',  
    distance int COMMENT '配送距离',  
    dispatcher_code string COMMENT '收货码',  
    receiver_name string COMMENT '收货人姓名',  
    receiver_phone string COMMENT '收货人电话',  
    sender_name string COMMENT '发货人姓名',  
    sender_phone string COMMENT '发货人电话',  
    create_user string,  
    create_time string,  
    update_user string,  
    update_time string,  
    is_valid tinyint COMMENT '是否有效 0: false\; 1: true',  
    end_date string COMMENT '拉链结束日期')  
COMMENT '订单配送详细信息表'  
partitioned by (start_date string)  
row format delimited fields terminated by '\t'  
stored as orc  
tblproperties ('orc.compress' = 'SNAPPY');
```





5.2.13.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.fact_order_delievery_item PARTITION(start_date)
select
    id,
    shop_order_id,
    refund_order_id,
    dispatcher_order_type,
    shop_store_id,
    buyer_id,
    circle_master_user_id,
    dispatcher_user_id,
    dispatcher_order_state,
    order_goods_num,
    delivery_fee,
    distance,
    dispatcher_code,
    receiver_name,
    receiver_phone,
    sender_name,
    sender_phone,
    create_user,
    create_time,
    update_user,
    update_time,
    is_valid,
    '9999-99-99' end_date,
    substr(create_time, 1, 10) as start_date
FROM yp_ods.t_order_delievery_item;
```

5.2.14 商品评价表（拉链表）

5.2.14.1 建表

```
DROP TABLE if EXISTS yp_dwd.fact_goods_evaluation_detail;
CREATE TABLE yp_dwd.fact_goods_evaluation_detail(
    id string,
    user_id string COMMENT '评论人id',
    store_id string COMMENT '店铺id',
    goods_id string COMMENT '商品id',
    order_id string COMMENT '订单id',
    order_goods_id string COMMENT '订单商品表id',
    geval_scores_goods int COMMENT '商品评分0-10分',
    geval_content string,
    geval_content_superaddition string COMMENT '追加评论',
    geval_addtime string COMMENT '评论时间',
    geval_addtime_superaddition string COMMENT '追加评论时间',
    geval_state tinyint COMMENT '评价状态 1-正常 0-禁止显示',
    geval_remark string COMMENT '管理员对评价的处理备注',
    revert_state tinyint COMMENT '回复状态0未回复1已回复',
```





```
geval_explain string COMMENT '管理员回复内容',
geval_explain_superaddition string COMMENT '管理员追加回复内容',
geval_explaintime string COMMENT '管理员回复时间',
geval_explaintime_superaddition string COMMENT '管理员追加回复时间',
create_user string,
create_time string,
update_user string,
update_time string,
is_valid tinyint COMMENT '0：失效，1：开启',
end_date string COMMENT '拉链结束日期')
COMMENT '商品评价明细'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.14.2 导入数据

```
INSERT overwrite TABLE yp_dwd.fact_goods_evaluation_detail PARTITION(dt)
select
  id,
  user_id,
  store_id,
  goods_id,
  order_id,
  order_goods_id,
  GEVAL_scores_goods,
  geval_content,
  geval_content_superaddition,
  geval_addtime,
  geval_addtime_superaddition,
  geval_state,
  geval_remark,
  revert_state,
  geval_explain,
  geval_explain_superaddition,
  geval_explaintime,
  geval_explaintime_superaddition,
  create_user,
  create_time,
  update_user,
  update_time,
  is_valid,
  '9999-99-99' end_date,
  substr(create_time, 1, 10) as start_date
from yp_ods.t_goods_evaluation_detail;
```

5.2.15 交易记录（拉链表）



5.2.15.1 建表

```
DROP TABLE if exists yp_dwd.fact_trade_record;
CREATE TABLE yp_dwd.fact_trade_record
(
    id                string COMMENT '交易单号',
    external_trade_no string COMMENT '(支付, 结算, 退款) 第三方交易单号',
    relation_id       string COMMENT '关联单号',
    trade_type        TINYINT COMMENT '1. 支付订单; 2. 结算订单; 3. 退款订单; 4. 充值单; 5. 提现单; 6. 分销单; 7. 缴纳保证金单 8. 退还保证金单 9. 冻结通联订单, 10. 通联通账户余额充值, 11. 扫码单',
    status            TINYINT COMMENT '1. 成功; 2. 失败; 3. 进行中',
    finnshed_time     string COMMENT '订单完成时间, 当配送员点击确认送达时, 进行更新订单完成时间, 后期需要根据订单完成时间, 进行自动收货以及自动评价',
    fail_reason       string COMMENT '交易失败的原因',
    payment_type       string COMMENT '支付方式: 小程序, app 微信, 支付宝, 快捷支付, 钱包, 银行卡, 消费券',
    trade_before_balance DECIMAL(11,2) COMMENT '交易前余额',
    trade_true_amount  DECIMAL(11,2) COMMENT '交易实际支付金额, 第三方平台扣除优惠以后实际支付金额',
    trade_after_balance DECIMAL(11,2) COMMENT '交易后余额',
    note              string COMMENT '业务说明',
    user_card         string COMMENT '第三方平台账户标识/多钱包用户钱包id',
    user_id           string COMMENT '用户id',
    aip_user_id       string COMMENT '钱包id',
    create_user       string,
    create_time       string,
    update_user       string,
    update_time       string,
    is_valid          TINYINT COMMENT '是否有效 0: false; 1: true; 订单是否有效的标志',
    end_date string COMMENT '拉链结束日期')
comment '交易记录'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.15.2 导入数据

```
INSERT overwrite TABLE yp_dwd.fact_trade_record PARTITION (dt)
SELECT
    id,
    external_trade_no,
    relation_id,
    trade_type,
    status,
    finnshed_time,
    fail_reason,
    payment_type,
    trade_before_balance,
    trade_true_amount,
    trade_after_balance,
    note,
    user_card,
    user_id,
```





```
aip_user_id,  
create_user,  
create_time,  
update_user,  
update_time,  
is_valid,  
'9999-99-99' end_date,  
substr(create_time, 1, 10) as start_date  
FROM yp_ods.t_trade_record;
```

5.2.16 订单评价表（增量表）

5.2.16.1 建表

```
DROP TABLE if EXISTS yp_dwd.fact_goods_evaluation;  
CREATE TABLE yp_dwd.fact_goods_evaluation(  
    id string,  
    user_id string COMMENT '评论人id',  
    store_id string COMMENT '店铺id',  
    order_id string COMMENT '订单id',  
    geval_scores int COMMENT '综合评分',  
    geval_scores_speed int COMMENT '送货速度评分0-5分(配送评分)',  
    geval_scores_service int COMMENT '服务评分0-5分',  
    geval_isanony tinyint COMMENT '0-匿名评价, 1-非匿名',  
    create_user string,  
    create_time string,  
    update_user string,  
    update_time string,  
    is_valid tinyint COMMENT '0 : 失效, 1 : 开启')  
COMMENT '订单评价表'  
partitioned by (dt string)  
row format delimited fields terminated by '\t'  
stored as orc  
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.16.2 导入数据（首次/循环）

首次

```
INSERT overwrite TABLE yp_dwd.fact_goods_evaluation PARTITION(dt)  
select  
    id,  
    user_id,  
    store_id,  
    order_id,  
    geval_scores,  
    geval_scores_speed,  
    geval_scores_service,  
    geval_isanony,  
    create_user,  
    create_time,
```





```
update_user,  
update_time,  
is_valid,  
substr(create_time, 1, 10) as dt  
from yp_ods.t_goods_evaluation;
```

循环

```
INSERT overwrite TABLE yp_dwd.fact_goods_evaluation PARTITION(dt)  
select  
    id,  
    user_id,  
    store_id,  
    order_id,  
    geval_scores,  
    geval_scores_speed,  
    geval_scores_service,  
    geval_isanony,  
    create_user,  
    create_time,  
    update_user,  
    update_time,  
    is_valid,  
    substr(create_time, 1, 10) as dt  
from yp_ods.t_goods_evaluation  
where dt='${TD_DATE}';
```

5.2.17 登录记录表（增量表）

5.2.17.1 建表

```
DROP TABLE if exists yp_dwd.fact_user_login;  
CREATE TABLE yp_dwd.fact_user_login(  
    id string,  
    login_user string,  
    login_type string COMMENT '登录类型（登陆时使用）',  
    client_id string COMMENT '推送标示id(登录、第三方登录、注册、支付回调、给用户推送消息时使用)',  
    login_time string,  
    login_ip string,  
    logout_time string  
)  
COMMENT '用户登录记录表'  
partitioned by(dt string)  
row format delimited fields terminated by '\t'  
stored as orc  
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.17.2 导入数据

```
INSERT overwrite TABLE yp_dwd.fact_user_login PARTITION(dt)
```





```
select
  id,
  login_user,
  login_type,
  client_id,
  login_time,
  login_ip,
  logout_time,
  SUBSTRING(login_time, 1, 10) as dt
FROM yp_ods.t_user_login;
```

5.2.18 订单组支付（增量表）

5.2.18.1 建表

```
DROP TABLE if EXISTS yp_dwd.fact_order_pay;
CREATE TABLE yp_dwd.fact_order_pay(
  id string,
  group_id string COMMENT '关联shop_order_group的group_id,一对多订单',
  order_pay_amount decimal(36,2) COMMENT '订单总金额;',
  create_date string COMMENT '订单创建的时间,需要根据订单创建时间进行判断订单是否已经失效',
  create_user string,
  create_time string,
  update_user string,
  update_time string,
  is_valid tinyint COMMENT '是否有效 0: false; 1: true;  订单是否有效的标志')
COMMENT '订单组支付表'
partitioned by (dt string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.2.18.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.fact_order_pay PARTITION (start_date)
SELECT
  id
  ,group_id
  ,order_pay_amount
  ,create_date
  ,create_user
  ,create_time
  ,update_user
  ,update_time
  ,is_valid
  ,dt
FROM yp_ods.t_order_pay;
```





5.3 维度表

5.3.1 区域字典表（全量覆盖）

5.3.1.1 建表

```
DROP TABLE IF EXISTS yp_dwd.dim_district;
CREATE TABLE yp_dwd.dim_district(
  id string COMMENT '主键ID',
  code string COMMENT '区域编码',
  name string COMMENT '区域名称',
  pid string COMMENT '父级ID',
  alias string COMMENT '别名')
COMMENT '区域字典表'
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.3.1.2 导入数据

```
INSERT overwrite TABLE yp_dwd.dim_district
select * from yp_ods.t_district
WHERE code IS NOT NULL AND name IS NOT NULL;
```

5.3.2 时间维度表（全量覆盖）

5.3.2.1 建表

```
-- 时间维度表
drop table yp_dwd.dim_date;
CREATE TABLE yp_dwd.dim_date
(
  dim_date_id      string COMMENT '日期',
  date_code        string COMMENT '日期编码',
  lunar_calendar   string COMMENT '农历',
  year_code        string COMMENT '年code',
  year_name        string COMMENT '年名称',
  month_code       string COMMENT '月份编码',
  month_name       string COMMENT '月份名称',
  quarter_code     string COMMENT '季度编码',
  quarter_name     string COMMENT '季度名称',
  year_month       string COMMENT '年月',
```





```

year_week_code      string COMMENT '一年中第几周',
year_week_name      string COMMENT '一年中第几周名称',
year_week_code_cn   string COMMENT '一年中第几周 (中国)',
year_week_name_cn   string COMMENT '一年中第几周名称 (中国)',
week_day_code       string COMMENT '周几code',
week_day_name       string COMMENT '周几名称',
day_week            string COMMENT '周',
day_week_cn         string COMMENT '周 (中国)',
day_week_num        string COMMENT '一周第几天',
day_week_num_cn     string COMMENT '一周第几天 (中国)',
day_month_num       string COMMENT '一月第几天',
day_year_num        string COMMENT '一年第几天',
date_id_wow         string COMMENT '与本周环比的上周日期',
date_id_mom         string COMMENT '与本月环比的上月日期',
date_id_wyw         string COMMENT '与本周同比的上年日期',
date_id_mym         string COMMENT '与本月同比的上年日期',
first_date_id_month string COMMENT '本月第一天日期',
last_date_id_month  string COMMENT '本月最后一天日期',
half_year_code      string COMMENT '半年code',
half_year_name      string COMMENT '半年名称',
season_code         string COMMENT '季节编码',
season_name         string COMMENT '季节名称',
is_weekend          string COMMENT '是否周末 (周六和周日)',
official_holiday_code string COMMENT '法定节假日编码',
official_holiday_name string COMMENT '法定节假日',
festival_code       string COMMENT '节日编码',
festival_name       string COMMENT '节日',
custom_festival_code string COMMENT '自定义节日编码',
custom_festival_name string COMMENT '自定义节日',
update_time         string COMMENT '更新时间'
)
COMMENT '时间维度表'
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');

```

5.3.2.2 导入数据

```

INSERT overwrite TABLE yp_dwd.dim_date
select
    concat(substr(dim_date_id,1,4), '-', substr(dim_date_id,5,2), '-', substr(dim_date_id,7,2))
as dim_date_id,
    date_code,
    concat(substr(lunar_calendar,1,4), '-', substr(lunar_calendar,5,2), '-',
substr(lunar_calendar,7,2)) as lunar_calendar,
    year_code,
    year_name,

```





```
month_code,  
month_name,  
quarter_code,  
quarter_name,  
concat(substr(year_month,1,4), '-', substr(year_month,5,2)) as year_month,  
year_week_code,  
concat(substr(year_week_name,1,4), '-', substr(year_week_name,5,2)) as year_week_name,  
year_week_code_cn,  
concat(substr(year_week_name_cn,1,4), '-', substr(year_week_name_cn,5,2)) as  
year_week_name_cn,  
week_day_code,  
week_day_name,  
day_week,  
day_week_cn,  
day_week_num,  
day_week_num_cn,  
day_month_num,  
day_year_num,  
concat(substr(date_id_wow,1,4), '-', substr(date_id_wow,5,2), '-', substr(date_id_wow,7,2))  
as date_id_wow,  
concat(substr(date_id_mom,1,4), '-', substr(date_id_mom,5,2), '-', substr(date_id_mom,7,2))  
as date_id_mom,  
date_id_wyw,  
concat(substr(date_id_mym,1,4), '-', substr(date_id_mym,5,2), '-', substr(date_id_mym,7,2))  
as date_id_mym,  
concat(substr(first_date_id_month,1,4), '-', substr(first_date_id_month,5,2), '-',  
substr(first_date_id_month,7,2)) as first_date_id_month,  
concat(substr(last_date_id_month,1,4), '-', substr(last_date_id_month,5,2), '-',  
substr(last_date_id_month,7,2)) as last_date_id_month,  
half_year_code,  
half_year_name,  
season_code,  
season_name,  
is_weekend,  
official_holiday_code,  
official_holiday_name,  
festival_code,  
festival_name,  
custom_festival_code,  
custom_festival_name,  
update_time  
from yp_ods.t_date;
```





5.3.3 店铺 (拉链表)

5.3.3.1 建表

```
DROP TABLE if EXISTS yp_dwd.dim_store;
CREATE TABLE yp_dwd.dim_store(
  id string COMMENT '主键',
  user_id string,
  store_avatar string COMMENT '店铺头像',
  address_info string COMMENT '店铺详细地址',
  name string COMMENT '店铺名称',
  store_phone string COMMENT '联系电话',
  province_id int COMMENT '店铺所在省份ID',
  city_id int COMMENT '店铺所在城市ID',
  area_id int COMMENT '店铺所在县ID',
  mb_title_img string COMMENT '手机店铺 页头背景图',
  store_description string COMMENT '店铺描述',
  notice string COMMENT '店铺公告',
  is_pay_bond tinyint COMMENT '是否有交过保证金 1: 是 0: 否',
  trade_area_id string COMMENT '归属商圈ID',
  delivery_method tinyint COMMENT '配送方式 1：自提；3：自提加配送均可；2：商家配送',
  origin_price decimal(36,2),
  free_price decimal(36,2),
  store_type int COMMENT '店铺类型 22天街网店 23实体店 24直营店铺 33会员专区店',
  store_label string COMMENT '店铺logo',
  search_key string COMMENT '店铺搜索关键字',
  end_time string COMMENT '营业结束时间',
  start_time string COMMENT '营业开始时间',
  operating_status tinyint COMMENT '营业状态 0：未营业；1：正在营业',
  create_user string,
  create_time string,
  update_user string,
  update_time string,
  is_valid tinyint COMMENT '0关闭, 1开启, 3店铺申请中',
  state string COMMENT '可使用的支付类型:MONEY金钱支付\CASHCOUPON现金券支付',
  idcard string COMMENT '身份证',
  deposit_amount decimal(36,2) COMMENT '商圈认购费用总额',
  delivery_config_id string COMMENT '配送配置表关联ID',
  aip_user_id string COMMENT '通联支付标识ID',
  search_name string COMMENT '模糊搜索名称字段:名称_真实名称',
  automatic_order tinyint COMMENT '是否开启自动接单功能 1: 是 0: 否',
  is_primary tinyint COMMENT '是否是总店 1: 是 2: 不是',
  parent_store_id string COMMENT '父级店铺的id, 只有当is_primary类型为2时有效',
  end_date string COMMENT '拉链结束日期')
COMMENT '店铺表'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```





5.3.3.2 导入数据

```
INSERT overwrite TABLE yp_dwd.dim_store PARTITION (start_date)
select
  id,
  user_id,
  store_avatar,
  address_info,
  name,
  store_phone,
  province_id,
  city_id,
  area_id,
  mb_title_img,
  store_description,
  notice,
  is_pay_bond,
  trade_area_id,
  delivery_method,
  origin_price,
  free_price,
  store_type,
  store_label,
  search_key,
  end_time,
  start_time,
  operating_status,
  create_user,
  create_time,
  update_user,
  update_time,
  is_valid,
  state,
  idcard,
  deposit_amount,
  delivery_config_id,
  aip_user_id,
  search_name,
  automatic_order,
  is_primary,
  parent_store_id,
  '9999-99-99' end_date,
  SUBSTRING(create_time, 1, 10) as start_date
from yp_ods.t_store;
```

5.3.4 商圈（拉链表）

5.3.4.1 建表

```
DROP TABLE if EXISTS yp_dwd.dim_trade_area;
```





```
CREATE TABLE yp_dwd.dim_trade_area(  
    id string COMMENT '主键',  
    user_id string COMMENT '用户ID',  
    user_allinpay_id string COMMENT '通联用户表id',  
    trade_avatar string COMMENT '商圈logo',  
    name string COMMENT '商圈名称',  
    notice string COMMENT '商圈公告',  
    distric_province_id int COMMENT '商圈所在省份ID',  
    distric_city_id int COMMENT '商圈所在城市ID',  
    distric_area_id int COMMENT '商圈所在县ID',  
    address string COMMENT '商圈地址',  
    radius double COMMENT '半径',  
    mb_title_img string COMMENT '手机商圈 页头背景图',  
    deposit_amount decimal(36,2) COMMENT '商圈认购费用总额',  
    hava_deposit int COMMENT '是否有交过保证金 1: 是 0: 否',  
    state tinyint COMMENT '申请商圈状态 -1: 未认购; 0: 申请中; 1: 已认购;',  
    search_key string COMMENT '商圈搜索关键字',  
    create_user string,  
    create_time string,  
    update_user string,  
    update_time string,  
    is_valid tinyint COMMENT '是否有效 0: false; 1: true',  
    end_date string COMMENT '拉链结束日期')  
COMMENT '商圈表'  
partitioned by (start_date string)  
row format delimited fields terminated by '\t'  
stored as orc  
tblproperties ('orc.compress' = 'SNAPPY');
```

5.3.4.2 导入数据

```
--全量  
INSERT overwrite TABLE yp_dwd.dim_trade_area PARTITION(start_date)  
SELECT  
    id,  
    user_id,  
    user_allinpay_id,  
    trade_avatar,  
    name,  
    notice,  
    distric_province_id,  
    distric_city_id,  
    distric_area_id,  
    address,  
    radius,  
    mb_title_img,  
    deposit_amount,  
    hava_deposit,  
    state,  
    search_key,  
    create_user,  
    create_time,
```





```
update_user,  
update_time,  
is_valid,  
'9999-99-99' end_date,  
SUBSTRING(create_time, 1, 10) as start_date  
FROM yp_ods.t_trade_area;
```

5.3.5 地址信息表（拉链表）

5.3.5.1 建表

```
DROP TABLE if EXISTS yp_dwd.dim_location;  
CREATE TABLE yp_dwd.dim_location(  
    id string COMMENT '主键',  
    type int COMMENT '类型 1: 商圈地址; 2: 店铺地址; 3.用户地址管理\; 4.订单买家地址信息\; 5.订单卖家地址信息',  
    correlation_id string COMMENT '关联表id',  
    address string COMMENT '地图地址详情',  
    latitude double COMMENT '纬度',  
    longitude double COMMENT '经度',  
    street_number string COMMENT '门牌',  
    street string COMMENT '街道',  
    district string COMMENT '区县',  
    city string COMMENT '城市',  
    province string COMMENT '省份',  
    business string COMMENT '百度商圈字段，代表此点所属的商圈',  
    create_user string,  
    create_time string,  
    update_user string,  
    update_time string,  
    is_valid tinyint COMMENT '是否有效 0: false\; 1: true',  
    adcode string COMMENT '百度adcode,对应区县code',  
    end_date string COMMENT '拉链表结束日期')  
COMMENT '地址信息'  
partitioned by (start_date string)  
row format delimited fields terminated by '\t'  
stored as orc  
tblproperties ('orc.compress' = 'SNAPPY');
```

5.3.5.2 导入数据

```
--全量  
INSERT overwrite TABLE yp_dwd.dim_location PARTITION(start_date)  
SELECT  
    id,  
    type,  
    correlation_id,
```





```
address,  
latitude,  
longitude,  
street_number,  
street,  
district,  
city,  
province,  
business,  
create_user,  
create_time,  
update_user,  
update_time,  
is_valid,  
adcode,  
'9999-99-99' end_date,  
SUBSTRING(create_time, 1, 10) as start_date  
FROM yp_ods.t_location;
```

5.3.6 商品SKU表 (拉链表)

5.3.6.1 建表

```
DROP TABLE if EXISTS yp_dwd.dim_goods;  
CREATE TABLE yp_dwd.dim_goods(  
  id string,  
  store_id string COMMENT '所属商店ID',  
  class_id string COMMENT '分类id:只保存最后一层分类id',  
  store_class_id string COMMENT '店铺分类id',  
  brand_id string COMMENT '品牌id',  
  goods_name string COMMENT '商品名称',  
  goods_specification string COMMENT '商品规格',  
  search_name string COMMENT '模糊搜索名称字段:名称_+真实名称',  
  goods_sort int COMMENT '商品排序',  
  goods_market_price decimal(36,2) COMMENT '商品市场价',  
  goods_price decimal(36,2) COMMENT '商品销售价格(原价)',  
  goods_promotion_price decimal(36,2) COMMENT '商品促销价格(售价)',  
  goods_storage int COMMENT '商品库存',  
  goods_limit_num int COMMENT '购买限制数量',  
  goods_unit string COMMENT '计量单位',  
  goods_state tinyint COMMENT '商品状态 1正常, 2下架, 3违规(禁售)',  
  goods_verify tinyint COMMENT '商品审核状态: 1通过, 2未通过, 3审核中',  
  activity_type tinyint COMMENT '活动类型: 0无活动 1促销 2秒杀 3折扣',  
  discount int COMMENT '商品折扣(%)',  
  seckill_begin_time string COMMENT '秒杀开始时间',  
  seckill_end_time string COMMENT '秒杀结束时间',  
  seckill_total_pay_num int COMMENT '已秒杀数量',  
  seckill_total_num int COMMENT '秒杀总数限制',  
  seckill_price decimal(36,2) COMMENT '秒杀价格',
```





```
top_it tinyint COMMENT '商品置顶: 1-是, 0-否',
create_user string,
create_time string,
update_user string,
update_time string,
is_valid tinyint COMMENT '0 : 失效, 1 : 开启',
end_date string COMMENT '拉链结束日期')
COMMENT '商品表_店铺(SKU)'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.3.6.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.dim_goods PARTITION(start_date)
SELECT
  id,
  store_id,
  class_id,
  store_class_id,
  brand_id,
  goods_name,
  goods_specification,
  search_name,
  goods_sort,
  goods_market_price,
  goods_price,
  goods_promotion_price,
  goods_storage,
  goods_limit_num,
  goods_unit,
  goods_state,
  goods_verify,
  activity_type,
  discount,
  seckill_begin_time,
  seckill_end_time,
  seckill_total_pay_num,
  seckill_total_num,
  seckill_price,
  top_it,
  create_user,
  create_time,
  update_user,
  update_time,
  is_valid,
  '9999-99-99' end_date,
  SUBSTRING(create_time, 1, 10) as start_date
FROM yp_ods.t_goods;
```





5.3.7 商品分类（拉链表）

5.3.7.1 建表

```
DROP TABLE if EXISTS yp_dwd.dim_goods_class;
CREATE TABLE yp_dwd.dim_goods_class (
  id string,
  store_id string COMMENT '店铺id',
  class_id string COMMENT '对应的平台分类表id',
  name string COMMENT '店铺内分类名字',
  parent_id string COMMENT '父id',
  level tinyint COMMENT '分类层级',
  is_parent_node tinyint COMMENT '是否为父节点:1是0否',
  background_img string COMMENT '背景图片',
  img string COMMENT '分类图片',
  keywords string COMMENT '关键词',
  title string COMMENT '搜索标题',
  sort int COMMENT '排序',
  note string COMMENT '类型描述',
  url string COMMENT '分类的链接',
  is_use tinyint COMMENT '是否使用:0否,1是',
  create_user string,
  create_time string,
  update_user string,
  update_time string,
  is_valid tinyint COMMENT '0 : 失效, 1 : 开启',
  end_date string COMMENT '拉链表结束日期')
COMMENT '商品分类表'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.3.7.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.dim_goods_class PARTITION(start_date)
SELECT
  id,
  store_id,
  class_id,
  name,
  parent_id,
  level,
  is_parent_node,
  background_img,
  img,
  keywords,
  title,
```



```
sort,
note,
url,
is_use,
create_user,
create_time,
update_user,
update_time,
is_valid,
'9999-99-99' end_date,
SUBSTRING(create_time, 1, 10) as start_date
FROM yp_ods.t_goods_class;
```

5.3.8 品牌表（拉链表）

5.3.8.1 建表

```
DROP TABLE if EXISTS yp_dwd.dim_brand;
CREATE TABLE yp_dwd.dim_brand(
  id string,
  store_id string COMMENT '店铺id',
  brand_pt_id string COMMENT '平台品牌库品牌Id',
  brand_name string COMMENT '品牌名称',
  brand_image string COMMENT '品牌图片',
  initial string COMMENT '品牌首字母',
  sort int COMMENT '排序',
  is_use tinyint COMMENT '0禁用1启用',
  goods_state tinyint COMMENT '商品品牌审核状态 1 审核中,2 通过,3 拒绝',
  create_user string,
  create_time string,
  update_user string,
  update_time string,
  is_valid tinyint COMMENT '0 : 失效, 1 : 开启',
  end_date string COMMENT '拉链结束日期')
COMMENT '品牌（店铺）'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.3.8.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.dim_brand PARTITION(start_date)
SELECT
  id,
  store_id,
  brand_pt_id,
```



```
brand_name,  
brand_image,  
initial,  
sort,  
is_use,  
goods_state,  
create_user,  
create_time,  
update_user,  
update_time,  
is_valid,  
'9999-99-99' end_date,  
SUBSTRING(create_time, 1, 10) as start_date  
FROM yp_ods.t_brand;
```

5.3.9 用户表 (拉链表)

5.3.9.1 建表

```
DROP TABLE if exists yp_dwd.dim_user;  
CREATE TABLE yp_dwd.dim_user  
(  
    id                string COMMENT '主键',  
    phone             string COMMENT '会员电话',  
    phone_bind        INT COMMENT '会员手机号绑定: 0未绑定1已绑定',  
    avatar             string COMMENT '会员头像',  
    birthday           string COMMENT '会员生日',  
    email              string COMMENT '会员邮箱',  
    email_bind         INT COMMENT '会员邮箱绑定: 0未绑定1已绑定',  
    password           string COMMENT '会员登录密码',  
    salt              string COMMENT '盐 (密码加密用)',  
    sex                INT COMMENT '会员性别 1男 2女',  
    truename           string COMMENT '会员真实姓名',  
    inviter_id         string COMMENT '邀请人',  
    idcard             string COMMENT '身份证号',  
    create_user        string,  
    create_time        string,  
    update_user        string,  
    update_time        string,  
    is_valid           TINYINT COMMENT '是否有效 0: false; 1: true',  
    nickname           string COMMENT '会员昵称',  
    register_source    string COMMENT '注册来源',  
    user_allinpay_id   string COMMENT '通联用户表ID',  
    spreader_id        string COMMENT '推广者id',  
    union_id           string,  
    promo_code         string COMMENT '推广码',  
    end_date           string COMMENT '拉链结束日期')  
COMMENT '用户表'  
partitioned by (start_date string)  
row format delimited fields terminated by '\t'
```



```
stored as orc
tblproperties ('orc.compress' = 'SNAPPY');
```

5.3.9.2 导入数据

```
--全量
INSERT overwrite TABLE yp_dwd.dim_user PARTITION(start_date)
SELECT
  id,
  phone,
  phone_bind,
  avatar,
  birthday,
  email,
  email_bind,
  password,
  salt,
  sex,
  truename,
  inviter_id,
  idcard,
  create_user,
  create_time,
  update_user,
  update_time,
  is_valid,
  nickname,
  register_source,
  user_allinpay_id,
  spreader_id,
  union_id,
  promo_code,
  '9999-99-99' end_date,
  SUBSTRING(create_time, 1, 10) as start_date
FROM yp_ods.t_user;
```

6. 脚本文件

6.1 建库SQL

[资料\create_dwd_db.sql](#)

6.2 事实表

建表SQL: [资料\create_fact_table.sql](#)





导入SQL：[资料\insert-fact.sql](#)

6.3 维度表

建表SQL：[资料\create_dim_table.sql](#)

导入SQL：[资料\insert-dim.sql](#)

